2026-10-08 09:01:00 You

Elokinab-Health-V3.pdf
1.7 MB

Pharmacie-en-ligne-V1.pdf
14.2 MB

DeepSeek

Fix complet — Upload preuve via JSON base64 (contourne ERR_HTTP2_PROTOCOL_ERROR)

Le tunnel Cloudflare gratuit coupe les connexions HTTP/2 lors des uploads multipart > 200 Ko. La solution : envoyer l'image encodée en base64 dans un JSON classique au lieu d'un `multipart/form-data`.

Voici le script complet à coller d'un seul bloc (la réponse précédente avait été coupée).

Script complet

```bat
cd /d "C:\Users\ELOGE KEY\Desktop\pharma\pharmacie"
notepad fix_upload_json.py
```

Colle exactement ceci, puis Ctrl+S :

```python
"""
fix_upload_json.py
Contourne ERR_HTTP2_PROTOCOL_ERROR :
- Backend : nouvel endpoint /payment/proof-json (JSON + base64)
- Frontend : compression agressive + upload JSON base64
"""
from pathlib import Path
import datetime
import re
import shutil

BASE = Path(__file__).resolve().parent
PAY_JS = BASE / "static" / "payment.js"
PAY_PY = BASE / "v3_modules" / "payments.py"
```

```
stamp = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
backup = BASE / ("_backup_json_upload_" + stamp)
backup.mkdir(parents=True, exist_ok=True)
shutil.copy2(PAY_JS, backup / "payment.js")
shutil.copy2(PAY_PY, backup / "payments.py")
print("[OK] Backup ->", backup.name)

# ============================================================
# 1. BACKEND : endpoint JSON base64
# ============================================================
src = PAY_PY.read_text(encoding="utf-8")

if "import base64" not in src:
    src = src.replace("import uuid", "import base64\nimport uuid", 1)
    print("[OK] payments.py -> import base64")

if "class ProofUpload" not in src:
    src = src.replace(
        "class PaymentRejectRequest(BaseModel):",
        '''class ProofUpload(BaseModel):
    filename: str = "proof.jpg"
    content_type: str = "image/jpeg"
    data_base64: str


class PaymentRejectRequest(BaseModel):''',
        1,
    )
    print("[OK] payments.py -> modele ProofUpload")

if "/payment/proof-json" not in src:
    json_endpoint = ''' @app.post("/api/orders/{reference}/payment/proof-json")
 def upload_proof_json(reference: str, payload: ProofUpload,
                       user=Depends(require_role("PATIENT", "PROFESSIONNEL"))):
     """Upload preuve via JSON + base64. Contourne les bugs HTTP/2 multipart."""
     conn = get_db()
     try:
         order = conn.execute(
             "SELECT * FROM orders WHERE reference = ? AND user_id = ?",
             (reference, user["id"]),
         ).fetchone()
         if order is None:
             raise HTTPException(404, "Commande introuvable")
         payment = conn.execute(
             "SELECT * FROM payments WHERE order_id = ? ORDER BY id DESC LIMIT 1",
             (order["id"],),
         ).fetchone()
         if payment is None:
```

```
            raise HTTPException(400, "Initialisez le paiement d'abord")
        if payment["status"] == "VERIFIED":
            raise HTTPException(400, "Paiement deja verifie")
        if payment["status"] == "REJECTED":
            conn.execute(
                "UPDATE payments SET status='PENDING', rejection_reason='' WHERE id=?",
                (payment["id"],),
            )
        try:
            raw = payload.data_base64
            if "," in raw and raw.strip().startswith("data:"):
                raw = raw.split(",", 1)[1]
            data = base64.b64decode(raw)
        except Exception:
            raise HTTPException(400, "Donnees image invalides")
        if len(data) > MAX_PROOF_MB * 1024 * 1024:
            raise HTTPException(
                400, "Fichier trop volumineux (max " + str(MAX_PROOF_MB) + " Mo)"
            )
        ext = Path(payload.filename or "proof").suffix.lower()
        if ext not in (".jpg", ".jpeg", ".png", ".webp", ".heic", ".heif", ".pdf"):
            ext = ".jpg"
        fname = uuid.uuid4().hex + ext
        target = UPLOAD_DIR / fname
        target.write_bytes(data)
        now = datetime.now().isoformat(timespec="seconds")
        conn.execute(
            "UPDATE payments SET proof_file=?, status='PROOF_SUBMITTED', submitted_at=? WHERE
id=?",
            ("/uploads/proofs/" + fname, now, payment["id"]),
        )
        conn.execute(
            "UPDATE orders SET payment_status='PROOF_SUBMITTED' WHERE id=?",
            (order["id"],),
        )
        conn.commit()
        updated = conn.execute(
            "SELECT * FROM payments WHERE id = ?", (payment["id"],)
        ).fetchone()
    finally:
        conn.close()
    return _row(updated)


@app.post("/api/orders/{reference}/payment/proof")'''
    anchor = ' @app.post("/api/orders/{reference}/payment/proof")'
    if anchor in src:
        src = src.replace(anchor, json_endpoint, 1)
        print("[OK] payments.py -> endpoint /payment/proof-json")
```

```
    else:
        print("[!] anchor /payment/proof introuvable")

PAY_PY.write_text(src, encoding="utf-8")

# ============================================================
# 2. FRONTEND : compression plus agressive + upload JSON
# ============================================================
js = PAY_JS.read_text(encoding="utf-8")

# Compression : 1000px max, qualite 0.65
js = js.replace(
    "function compressImage(file, maxDim, quality) {",
    "function compressImage(file, maxDim, quality) {\n"
    "    maxDim = maxDim || 1000;\n"
    "    quality = quality || 0.65;",
    1,
)

# Helper : lire un fichier en base64
if "function readAsBase64" not in js:
    helper = '''
 function readAsBase64(file) {
     return new Promise(function (resolve, reject) {
         var reader = new FileReader();
         reader.onerror = function () { reject(new Error("Lecture impossible")); };
         reader.onload = function (e) {
             var result = e.target.result || "";
             // Retirer le prefixe 
             if (result.indexOf(",") >= 0) result = result.split(",", 1)[1];
             resolve(result);
         };
         reader.readAsDataURL(file);
     });
 }
'''
    # Inserer avant "function send()"
    idx = js.find(" function send() {")
    if idx > 0:
        js = js[:idx] + helper + js[idx:]
        print("[OK] payment.js -> helper readAsBase64")

# Remplacer la fonction send complete
old_send_pat = re.compile(r' function send$$ \{.*?\n \}', re.DOTALL)

new_send = r''' function send() {
     if (!state.file) { toast("Choisissez un fichier."); return; }
     var btn = $("submit-btn");
     btn.disabled = true;
```

```
    btn.innerHTML = "Preparation...";

    // Etape 0 : compresser
    compressImage(state.file)
    .then(function (compressedFile) {
        var originalKB = Math.round(state.file.size / 1024);
        var finalKB = Math.round(compressedFile.size / 1024);
        console.log("[payment] taille finale : " + finalKB + " Ko (original : " + originalKB +
" Ko)");
        if (finalKB < originalKB) {
            toast("Image compressee : " + originalKB + " -> " + finalKB + " Ko");
        }
        return readAsBase64(compressedFile).then(function (b64) {
            return { file: compressedFile, base64: b64, sizeKB: finalKB };
        });
    })
    .then(function (payload) {
        // Etape 1 : init paiement
        btn.innerHTML = "Initialisation...";
        console.log("[payment] Etape 1 - init paiement");
        var initController = new AbortController();
        var initTimeout = setTimeout(function () { initController.abort(); }, 30000);
        return fetch(API + "/orders/" + encodeURIComponent(state.ref) + "/payment", {
            method: "POST",
            credentials: "same-origin",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify({ payment_method: "MANUAL" }),
            signal: initController.signal
        })
        .then(function (r) {
            clearTimeout(initTimeout);
            console.log("[payment] init status:", r.status);
            return r.json().then(function (d) {
                if (!r.ok) throw new Error(d.detail || "Erreur init paiement");
                return d;
            });
        })
        .then(function () {
            // Etape 2 : upload JSON base64
            btn.innerHTML = "Envoi de la preuve... (" + payload.sizeKB + " Ko)";
            console.log("[payment] Etape 2 - upload JSON (" + payload.sizeKB + " Ko)");
            var upController = new AbortController();
            var upTimeout = setTimeout(function () { upController.abort(); }, 120000);
            return fetch(API + "/orders/" + encodeURIComponent(state.ref) + "/payment/proof-
json", {
                method: "POST",
                credentials: "same-origin",
                headers: { "Content-Type": "application/json" },
                body: JSON.stringify({
```

```
                    filename: payload.file.name || "proof.jpg",
                    content_type: payload.file.type || "image/jpeg",
                    data_base64: payload.base64
                }),
                signal: upController.signal
            })
            .then(function (r) {
                clearTimeout(upTimeout);
                console.log("[payment] upload status:", r.status);
                return r.json().then(function (d) {
                    if (!r.ok) throw new Error(d.detail || "Erreur upload preuve");
                    return d;
                });
            });
        });
    })
    .then(function (result) {
        console.log("[payment] OK:", result);
        toast("Preuve envoyee. En attente de verification.", true);
        btn.innerHTML = "&#10003; Envoyee ! Redirection...";
        setTimeout(function () {
            window.location.href = "/suivi?ref=" + encodeURIComponent(state.ref);
        }, 1800);
    })
    .catch(function (e) {
        console.error("[payment] erreur:", e);
        var msg = e.message || "Erreur";
        if (e.name === "AbortError") {
            msg = "Delai depasse. Verifiez votre connexion puis reessayez.";
        }
        toast(msg);
        btn.disabled = false;
        btn.innerHTML = '<span>&#128172;</span> Reessayer l\'envoi';
    });
}'''

if old_send_pat.search(js):
    js = js.replace(old_send_pat.search(js).group(0), new_send, 1)
    print("[OK] payment.js -> send() avec upload JSON base64")
else:
    print("[!] fonction send introuvable")

PAY_JS.write_text(js, encoding="utf-8")

print()
print("=" * 60)
print("FIX UPLOAD JSON LIVRE")
print("=" * 60)
print()
```

```
print("Backend :")
print(" - POST /api/orders/{ref}/payment/proof-json")
print(" - Accepte JSON avec data_base64")
print()
print("Frontend :")
print(" - Compression : 1000px max, qualite 65% (au lieu de 1600px/80%)")
print(" - Upload JSON base64 (pas multipart)")
print(" - Logs detailles dans la console")
print()
print("PROCHAINE ETAPE :")
print(" taskkill /F /IM python.exe")
print(" uvicorn main:app --reload --host 0.0.0.0 --port 8000")
print(" Puis Ctrl+Shift+R sur iPhone")
```

Lancer

bat

```
python fix_upload_json.py
taskkill /F /IM python.exe
cd /d "C:\Users\ELOGE KEY\Desktop\pharma\pharmacie"
call venv\Scripts\activate
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

Tester

1. **iPhone** : ouvre la page paiement (navigation privée pour éviter le cache)
2. Choisis une photo de reçu (2-3 Mo)
3. Clique **Envoyer la preuve**
4. Regarde la console si tu veux (F12 sur PC, ou Safari Inspecteur sur Mac)

Ce que tu verras

```
[payment] compression : 2430 Ko -> 180 Ko
[payment] Etape 1 - init paiement
[payment] init status: 200
[payment] Etape 2 - upload JSON (180 Ko)
[payment] upload status: 200
[payment] OK: {status: "PROOF_SUBMITTED", ...}
```

Puis redirection automatique vers `/suivi?ref=CMD-xxx`.

Pourquoi ça marche maintenant

Élément	Avant	Après
Format envoi	multipart/form-data	JSON + base64

Élément	Avant	Après
Taille image	344 Ko	~180 Ko
Résolution	1600 px	1000 px
Qualité JPEG	80 %	65 %
HTTP/2 multipart	❌ ERR_HTTP2_PROTOCOL_ERROR	✅ Ignoré
Retry timeout	60 s	120 s

Les requêtes JSON passent sans problème à travers le tunnel Cloudflare — c'est spécifiquement le multipart qui pose problème en HTTP/2.

Si le tunnel coupe encore

Dis-moi ce que la console affiche après le clic :

```
[payment] taille finale : X Ko
[payment] Etape 1 - init paiement
[payment] init status: ???
[payment] Etape 2 - upload JSON (X Ko)
[payment] upload status: ???
[payment] erreur: ...
```

Avec ces logs, je saurai exactement où ça bloque.

2026-10-08 09:13:49 **You**

on a reglé ce probleme depuis

DeepSeek

Parfait 👍 Si l'upload fonctionne maintenant, on peut passer à la suite.

Voici où on en était, dis-moi ce que tu veux prioriser :

1. Vérifier l'accueil pro (refonte V6)

- Le guide démarrage apparaît si tu n'as pas de commande ?
- Le bloc « Reprendre une commande » s'affiche avec ta dernière commande ?
- La section Notifications apparaît après validation d'un paiement côté pharmacien ?

2. Test end-to-end complet

Le parcours de bout en bout :

1. Connexion pro (lien magic)
2. Accueil → Nouvelle demande
3. Catalogue → ajout produits
4. Demande → review → Confirmer
5. Confirmer → GPS auto depuis profil → Continuer vers paiement
6. Paiement → upload preuve (JSON base64)
7. Suivi → timeline dynamique
8. Pharmacien → valide paiement
9. Retour pro → notification + statut mis à jour

3. Points restants identifiés

- Fix auto-localisation (utiliser GPS du profil sans redemander) — déjà codé, à tester
- Vérifier que la timeline dynamique affiche bien les vraies dates
- Nettoyage des fichiers `.bak` et scripts temporaires

4. Autres chantiers possibles

- SMS notifications (pour quand l'app est fermée)
- VPS + domaine fixe (fin du tunnel instable)
- Mixx automatique
- Multi-pharmacies

Dis-moi :

1. Qu'est-ce qui marche / ne marche pas dans l'accueil pro actuellement ?
2. Tu veux lancer le test end-to-end maintenant, ou d'abord peaufiner un écran ?

2026-10-08 09:20:48 **You**

le plus grnd moment maintenant c'est que lorsqu'on clique sur l'url de base, ça affiche le vieux interface avant redirection vers le nouveau mais on redemarre le cloudfare je vais refaire l'essaie voir si ça persiste tu me donne un script de diagnostic du code

Script de diagnostic — Flash vieille interface avant redirection

Ce script vérifie les 6 causes possibles du bug en une seule passe.

Créer le script

```bat
cd /d "C:\Users\ELOGE KEY\Desktop\pharma\pharmacie"
notepad diag_flash.py
```

Colle exactement ceci, puis Ctrl+S :

```python
"""
diag_flash.py
Diagnostic du bug "vieille interface avant redirection".
Verifie : service worker, cache HTML, middleware, headers, contenu servi.
"""
import sqlite3
import urllib.request
import urllib.error
import re
from pathlib import Path

BASE = Path(__file__).resolve().parent
MAIN = BASE / "main.py"
STATIC = BASE / "static"
DB = BASE / "database.db"
API = "http://127.0.0.1:8000"


def section(t):
    print()
    print("=" * 72)
    print(" " + t)
    print("=" * 72)
    print()


def get(path, method="GET"):
    try:
        req = urllib.request.Request(API + path, method=method)
        with urllib.request.urlopen(req, timeout=5) as r:
            return r.status, dict(r.headers), r.read().decode("utf-8", errors="replace")
```

```
    except urllib.error.HTTPError as e:
        return e.code, dict(e.headers), e.read().decode("utf-8", errors="replace")
    except Exception as e:
        return 0, {}, "ERREUR: " + str(e)


# ============================================================
# 1. Fichiers HTML sur disque
# ============================================================
section("1. FICHIERS HTML SUR DISQUE")

for name in ("index.html", "professional.html", "suivi.html", "payment.html"):
    p = STATIC / name
    if not p.exists():
        print(f"  [!] {name} MANQUANT")
        continue
    content = p.read_text(encoding="utf-8")
    size = len(content)
    flags = []
    if "eh-loader" in content: flags.append("loader")
    if "safe_locale" in content: flags.append("safe_locale")
    if "sw_kill" in content.lower(): flags.append("sw-kill")
    if "serviceworker" in content.lower(): flags.append("SW-REG")
    if "Cache-Control" in content: flags.append("meta-cache")
    print(f"  {name:25s} {size:>7d} o  [{', '.join(flags) if flags else 'basique'}]")

# ============================================================
# 2. Service Worker
# ============================================================
section("2. SERVICE WORKER")

sw_files = list(STATIC.glob("sw*.js")) + list(STATIC.glob("service-worker*.js"))
if sw_files:
    for f in sw_files:
        print(f"  [!] {f.name} present ({f.stat().st_size} o)")
else:
    print("  [OK] Aucun service worker detecte sur disque")

# ============================================================
# 3. Routes HTML dans main.py
# ============================================================
section("3. ROUTES HTML DANS main.py")

src = MAIN.read_text(encoding="utf-8")

routes = re.findall(r'@app\.(get|api_route)\("(/[^"]*)"', src)
html_routes = [(m, p) for m, p in routes if not p.startswith("/api/") and not
p.startswith("/static/")]
for method, path in html_routes:
```

```
    print(f"  {method.upper():10s} {path}")

# ============================================================
# 4. Middlewares actifs
# ============================================================
section("4. MIDDLEWARES DANS main.py")

middlewares = [
    "NoCacheAllStatic",
    "NoCacheHTMLMiddleware",
    "ClearSiteDataMiddleware",
    "StaticCacheMiddleware",
    "GZipMiddleware",
    "NoCacheStaticFiles",
]
for mw in middlewares:
    count = src.count("add_middleware(" + mw)
    status = "OK" if count >= 1 else "absent"
    print(f"  {mw:30s} {status} ({count}x)")

# ============================================================
# 5. Headers servis par le serveur
# ============================================================
section("5. HEADERS SERVIS (localhost)")

for route in ("/", "/professionnel", "/connexion", "/inscription"):
    status, headers, _ = get(route)
    cc = headers.get("cache-control", headers.get("Cache-Control", ""))
    csd = headers.get("clear-site-data", headers.get("Clear-Site-Data", ""))
    ctype = headers.get("content-type", headers.get("Content-Type", ""))[:40]
    print(f"  {route:20s} status={status} ctype={ctype}")
    print(f"    cache-control={cc[:70]}")
    print(f"    clear-site-data={csd}")

# ============================================================
# 6. Contenu HTML servi vs disque
# ============================================================
section("6. CONTENU SERVI vs DISQUE")

status, headers, served = get("/")
on_disk = (STATIC / "index.html").read_text(encoding="utf-8")

print(f"  Taille disque : {len(on_disk)}")
print(f"  Taille servi  : {len(served)}")
print(f"  Identiques    : {served == on_disk}")

if served != on_disk:
    min_len = min(len(served), len(on_disk))
    diff_at = -1
```

```
    for i in range(min_len):
        if served[i] != on_disk[i]:
            diff_at = i
            break
    if diff_at >= 0:
        print(f"  Premiere diff a l'offset {diff_at}")
        print(f"    disque : {repr(on_disk[diff_at:diff_at+80])}")
        print(f"    servi  : {repr(served[diff_at:diff_at+80])}")

# ============================================================
# 7. Detecter le vieux contenu
# ============================================================
section("7. MARQUEURS VIEUX vs NOUVEAU")

status, _, served = get("/")
markers = {
    "gateway (V6)": "gw-card" in served or "acc-welcome" in served,
    "guide V6": "acc-guide" in served,
    "loader (eh-loader)": "eh-loader" in served,
    "old placeholder style": "Votre pharmacie, en ligne" in served,
    "old design (vert)": "#0f9d76" in served and "1a4a7f" not in served,
    "nouveau bleu": "1a4a7f" in served or "--blue" in served,
    "catalogue direct link": "pro-quick" in served,
}
for label, ok in markers.items():
    print(f"  {'OK ' if ok else '[!]'} {label}: {ok}")

# ============================================================
# 8. Magic link + redirection
# ============================================================
section("8. MAGIC LINK (test de redirection)")

try:
    import secrets, datetime
    conn = sqlite3.connect(DB)
    conn.row_factory = sqlite3.Row
    user = conn.execute("SELECT id FROM users WHERE email='pro@test.tg'").fetchone()
    if user:
        token = secrets.token_urlsafe(16)
        now = datetime.datetime.now()
        exp = (now + datetime.timedelta(hours=1)).isoformat(timespec="seconds")
        conn.execute(
            "INSERT INTO magic_links (token, user_id, created_at, expires_at, max_uses,
used_count, revoked, note)"
            " VALUES (?, ?, ?, ?, 0, 0, 0, 'diag')",
            (token, user["id"], now.isoformat(timespec="seconds"), exp),
        )
        conn.commit()
        print(f"  Magic link cree : /acces/{token[:20]}...")
```

```
        status, headers, body = get("/acces/" + token)
        print(f"  status={status}")
        print(f"  location={headers.get('location', headers.get('Location', ''))}")
        print(f"  cache-control={headers.get('cache-control', headers.get('Cache-Control', ''))
[:60]}")
        print(f"  ctype={headers.get('content-type', headers.get('Content-Type', ''))[:40]}")
        print(f"  taille={len(body)}")
        print()
        print("  Extrait du body :")
        print("  " + body[:300].replace("\n", "\n  "))
    else:
        print("  [!] pro@test.tg introuvable")
    conn.close()
except Exception as e:
    print("  Erreur magic link :", e)

# ============================================================
# 9. Fichiers suspects
# ============================================================
section("9. FICHIERS SUSPECTS")

baks = list(STATIC.glob("*.bak")) + list(STATIC.glob("*.bak*"))
print(f"  {len(baks)} fichier(s) .bak dans static/")
for f in baks[:10]:
    print(f"    {f.name}")

# Chercher d'autres index.html (doublons)
dups = list(BASE.rglob("index.html"))
print(f"\n  {len(dups)} index.html trouves :")
for d in dups:
    print(f"    {d.relative_to(BASE)} ({d.stat().st_size} o)")

# ============================================================
section("FIN")
print("  Colle-moi l'integralite du resultat ci-dessus.")
```

Lancer

```
bat

python diag_flash.py
```

Ce que le script vérifie

#	Test	Cause possible si échec
1	Fichiers HTML disque	Fichier non patché

#	Test	Cause possible si échec
2	Service Worker	SW qui intercepte et sert du vieux cache
3	Routes dans main.py	Route mal enregistrée
4	Middlewares	No-cache absent
5	Headers HTTP	Cache-Control manquant ou permissif
6	Disque vs servi	Middleware qui altère le HTML
7	Marqueurs V6	Détection ancien/nouveau design
8	Magic link	Redirection 302 vs HTML
9	Fichiers suspects	Doublons `.bak` confondus par erreur

Ce que je vais regarder dans ta sortie

- **Section 2** : s'il y a un Service Worker → c'est lui la cause n°1
- **Section 5** : si `cache-control` n'est pas `no-store` → middleware absent
- **Section 6** : si disque ≠ servi → middleware qui altère
- **Section 8** : si magic link renvoie 302 (mauvaise URL) → cache navigateur sur URL fixe
- **Section 9** : si plusieurs `index.html` existent → confusion de route

En attendant, teste sur iPhone en navigation privée

Pour confirmer ou infirmer la théorie du cache Safari :

1. Safari iPhone → onglets → **Privé**
2. Ouvre l'URL de base
3. Si le flash disparaît en privé → **c'est bien du cache Safari**
4. Si ça persiste → **c'est côté serveur/route**

Colle-moi :

1. La sortie complète de `python diag_flash.py`
2. Le résultat du test en navigation privée sur iPhone

2026-10-08 09:22:57 **You**

commande pour lancer cloudfare et uvicorn

Lancer Cloudflare + Uvicorn

Terminal 1 — Uvicorn

Ouvre un CMD et colle :

```bat
cd /d "C:\Users\ELOGE KEY\Desktop\pharma\pharmacie"
call venv\Scripts\activate
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

Attends de voir :

`INFO: Application startup complete.`

Laisse cette fenêtre ouverte.

Terminal 2 — Cloudflared

Ouvre un **deuxième** CMD et colle :

```bat
"C:\Program Files (x86)\cloudflared\cloudflared.exe" tunnel --url http://127.0.0.1:8000 --
retries 5
```

Attends 15-20 secondes, cherche dans la sortie :

```
+--------------------------------------------------------------------------------------------
+
| Your quick Tunnel has been created! Visit it at (it may take some time to be reachabl
e):
| https://xxxxx-xxxxx-xxxxx-xxxx.trycloudflare.com                                          |
+--------------------------------------------------------------------------------------------
+
```

Copie cette URL.

Lancer les 2 d'un coup (recommandé)

Si tu veux lancer tout automatiquement avec le tunnel + génération du lien pro :

```bat
cd /d "C:\Users\ELOGE KEY\Desktop\pharma\pharmacie"
python restart_and_link.py
```

Ce script fait tout :

1. Vérifie qu'Uvicorn tourne
2. Tue l'ancien cloudflared
3. Relance cloudflared
4. Capture la nouvelle URL
5. Met à jour `generate_link.py`
6. Génère le magic link pour pro@test.tg
7. **Copie le lien dans le presse-papier**

⚠️ **Uvicorn doit déjà tourner** dans une autre fenêtre avant de lancer ce script.

Résumé

Fenêtre	Commande
1 — Uvicorn	`uvicorn main:app --reload --host 0.0.0.0 --port 8000`
2 — Cloudflared	cloudflared.exe tunnel --url http://127.0.0.1:8000 --retries 5
OU un seul script	`python restart_and_link.py`

Colle-moi la nouvelle URL dès qu'elle s'affiche, je te donnerai les liens de test prêts à ouvrir.

2026-10-08 09:24:45 **You**

Microsoft Windo***19045.6466].txt
5.9 KB

Microsoft Windo***19045.6466].txt
4.9 KB

DeepSeek

Diagnostic — Serveur Uvicorn pas lancé

Le diagnostic est clair : **toutes les sections HTTP renvoient `status=0` et `WinError 10061`**. Uvicorn ne tourne pas sur le port 8000.

Preuve :

- Section 5 : tous les `status=0`
- Section 6 : `ERREUR: <urlopen error [WinError 10061] Aucune connexion n'a pu être établie>`
- Section 7 : tous les marqueurs à `False` (car rien n'est servi)

C'est pour ça que tu vois le vieux flash : **Cloudflared pointe vers `127.0.0.1:8000` mais rien ne répond**, donc l'erreur 502 est mise en cache par le navigateur.

Bonne nouvelle : tout le reste est OK.

- ✅ Middlewares présents (`NoCacheHTMLMiddleware` , `ClearSiteDataMiddleware`)
- ✅ Routes `/connexion` , `/inscription` en `api_route` (support HEAD)
- ✅ Aucun Service Worker sur disque
- ✅ Fichiers HTML patchés (`safe_locale` , `meta-cache`)

Fix immédiat

Terminal 1 — Lancer Uvicorn

```bat
cd /d "C:\Users\ELOGE KEY\Desktop\pharma\pharmacie"
call venv\Scripts\activate
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

Attends de voir :

```
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000
```

Laisse cette fenêtre ouverte.

Terminal 2 — Vérifier que ça répond

Dans un nouveau CMD :

```bat
curl -s -o nul -w "HTTP %%{http_code}\n" http://127.0.0.1:8000/
```

Attendu : `HTTP 200`

Terminal 3 — Relancer le diagnostic

```bat
cd /d "C:\Users\ELOGE KEY\Desktop\pharma\pharmacie"
python diag_flash.py
```

Cette fois les sections 5, 6, 7, 8 devraient toutes avoir des valeurs correctes.

Cloudflared tourne toujours ?

Ton tunnel est actif :

`https://linked-cycles-extras-non.trycloudflare.com`

Il reste valide tant que la fenêtre cloudflared reste ouverte. Pas besoin de le relancer.

Une fois Uvicorn lancé — générer le lien pro

```bat
cd /d "C:\Users\ELOGE KEY\Desktop\pharma\pharmacie"
python generate_link.py pro@test.tg --days 30
```

⚠️ **Avant**, mets à jour `generate_link.py` :

```bat
notepad generate_link.py
```

Ctrl+H → Remplacer :

- Rechercher : `PUBLIC_URL = "https://<ancienne-URL>"`
- Remplacer par : `PUBLIC_URL = "https://linked-cycles-extras-non.trycloudflare.com"`

Ctrl+S puis lance `python generate_link.py pro@test.tg –days 30` .

Test du flash après

1. **iPhone en navigation privée** (Safari → onglets → Privé)
2. Colle le lien `/acces/...`
3. Attendu : pas de flash, redirection directe

Si le flash persiste APRÈS qu'Uvicorn tourne, alors c'est vraiment du cache Safari sur l'URL `/professionnel` . Dans ce cas, le `?_v=<timestamp>` sur le magic link réglera le problème (déjà en place dans `magic_links.py`).

Colle-moi :

1. La sortie d'Uvicorn au démarrage
2. La sortie de `curl -s -o nul -w "HTTP %%{http_code}\n" http://127.0.0.1:8000/`
3. La nouvelle sortie de `python diag_flash.py` (sections 5-8 surtout)

2026-10-08 09:26:42 You

Microsoft Windows [version 10.0.19045.6...
5.3 KB

DeepSeek

Trouvé — Cause réelle du flash

Le diagnostic révèle **2 problèmes** :

Problème 1 — `/connexion` renvoie `status=0` 🚨

```
/connexion    status=0   ← KO
/inscription  status=200 ← OK
```

La route `/connexion` plante silencieusement. À corriger.

Problème 2 — Le vrai coupable du flash

Regarde la section 7 :

```
OK  old placeholder style: True     ← "Votre pharmacie, en ligne"
OK  old design (vert): True         ← #0f9d76 (ancien vert)
[!] nouveau bleu: False             ← #1a4a7f (bleu V6 absent)
[!] gateway (V6): False             ← pas de nouveau design
```

`static/index.html` (15987 o) est l'ANCIEN fichier patient (design vert, ancienne structure).

Ce qui se passe quand tu cliques sur l'URL de base

1. Tu ouvres `https://xxx.trycloudflare.com/`
2. Le serveur renvoie `index.html` → **l'ancien design vert**
3. Le navigateur affiche cette vieille page pendant 200-500 ms
4. Pendant ce temps, un script JS détecte que tu es PRO connecté et fait `window.location.replace("/professionnel")`
5. La page bleue V6 apparaît enfin

Ce n'est pas du cache. C'est le mauvais fichier servi sur `/`.

Fix — Route `/` intelligente + correction `/connexion`

Un seul script qui règle les 2 problèmes.

Créer le script

```bat
cd /d "C:\Users\ELOGE KEY\Desktop\pharma\pharmacie"
notepad fix_route_root.py
```

Colle exactement ceci, puis Ctrl+S :

```python
"""
fix_route_root.py
1. Rend la route / intelligente : redirige les pros et admins
2. Corrige /connexion qui plante
"""
from pathlib import Path
import re
import shutil
import datetime

BASE = Path(__file__).resolve().parent
MAIN = BASE / "main.py"

stamp = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
backup = BASE / ("_backup_root_" + stamp)
backup.mkdir(parents=True, exist_ok=True)
shutil.copy2(MAIN, backup / "main.py")
print("[OK] Backup ->", backup.name)

src = MAIN.read_text(encoding="utf-8")

# ============================================================
# 1. Route / intelligente
# ============================================================
# Chercher la route / existante
pattern = re.compile(
    r'@app\.(get|api_route)$"/"[^)]*$\s*\ndef\s+(\w+)\s*$[^)]*$:.*?(?=\n@app\.|\n\nclass
|\n\ndef |\Z)',
    re.DOTALL
)

new_root = '''@app.get("/", include_in_schema=False)
def root_page(request: Request):
    """Redirige les pros/admins vers leur espace, sinon sert index.html."""
    token = request.cookies.get("elokinab_session")
    if token:
        try:
```

```
            conn = get_db()
            try:
                row = conn.execute(
                    "SELECT u.role FROM sessions s "
                    "JOIN users u ON u.id = s.user_id "
                    "WHERE s.token = ? AND s.expires_at > ?",
                    (token, datetime.now().isoformat(timespec="seconds")),
                ).fetchone()
                if row:
                    if row["role"] == "PROFESSIONNEL":
                        from fastapi.responses import RedirectResponse
                        return RedirectResponse("/professionnel", status_code=302)
                    if row["role"] in ("PHARMACIEN", "ADMIN"):
                        from fastapi.responses import RedirectResponse
                        return RedirectResponse("/pharmacien", status_code=302)
            finally:
                conn.close()
        except Exception:
            pass  # En cas d'erreur, on sert index.html normalement
    return FileResponse(STATIC_DIR / "index.html")
'''

if pattern.search(src):
    src = pattern.sub(new_root, src, count=1)
    print("[OK] main.py -> route / remplacee par version intelligente")
else:
    print("[!] route / introuvable, insertion manuelle")
    anchor = "\ninit_db()\n"
    if anchor in src:
        src = src.replace(anchor, "\n" + new_root + anchor, 1)
        print("[OK] main.py -> route / inseree avant init_db()")

# ============================================================
# 2. Verifier import Request
# ============================================================
if "from fastapi import" in src:
    first_import = src.split("from fastapi import")[1].split("\n")[0]
    if "Request" not in first_import:
        src = src.replace(
            "from fastapi import",
            "from fastapi import Request,",
            1,
        )
        print("[OK] main.py -> import Request ajoute")

MAIN.write_text(src, encoding="utf-8")

# ============================================================
# 3. Verifier la route /connexion dans main.py
```

```
# ============================================================
src = MAIN.read_text(encoding="utf-8")
print()
print("Route /connexion actuelle :")
connexion_pattern = re.compile(
    r'(@app\.(get|api_route)$"/connexion"[^)]*$\s*\ndef\s+(\w+)\s*$[^)]*$:.*?)(?
=\n@app\.|\n\nclass |\n\ndef |\Z)',
    re.DOTALL
)
m = connexion_pattern.search(src)
if m:
    print(m.group(1)[:400])
else:
    print("[!] Aucune route /connexion trouvee !")

print()
print("=" * 60)
print("FIX ROUTE ROOT APPLIQUE")
print("=" * 60)
print()
print("Changements :")
print(" 1. / -> Redirige PRO vers /professionnel")
print("      Redirige ADMIN/PHARMACIEN vers /pharmacien")
print("      Sinon -> index.html (patient)")
print(" 2. Import Request ajoute")
print()
print("PROCHAINE ETAPE :")
print(" taskkill /F /IM python.exe")
print(" uvicorn main:app --reload --host 0.0.0.0 --port 8000")
print()
print("Puis teste :")
print(" 1. Ouvre l'URL de base connecte en PRO")
print(" 2. Doit rediriger directement vers /professionnel")
print(" 3. Aucun flash d'ancienne interface")
```

Lancer

```bat
python fix_route_root.py
taskkill /F /IM python.exe
cd /d "C:\Users\ELOGE KEY\Desktop\pharma\pharmacie"
call venv\Scripts\activate
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

Tester

1. **Ouvre l'URL de base** : https://linked-cycles-extras-non.trycloudflare.com/

2. **Attendu** : redirection immédiate vers `/professionnel` — plus de flash
3. Si tu n'es PAS connecté → tu vois `index.html` (page patient, normal)

À propos de `/connexion` qui renvoie 0

Le diagnostic montre un `status=0` — c'est probablement un crash silencieux. Teste-le manuellement :

```bat
curl -s -o nul -w "HTTP %%{http_code}\n" http://127.0.0.1:8000/connexion
```

Si `HTTP 500` → regarde les logs Uvicorn (colle-moi les 5 dernières lignes). Si `HTTP 200` → c'était un timeout temporaire du script diag. Si rien → la route n'est pas correctement enregistrée.

Optionnel — refonte de `index.html` (patient)

`index.html` utilise encore le vieux design vert. Si tu veux que la page patient ait aussi le design bleu V6, dis-le et je te livre un script.

Colle-moi :

1. La sortie de `python fix_route_root.py`
2. Ce qui se passe quand tu ouvres l'URL de base (flash ou pas)
3. Le résultat de `curl -s -o nul -w "HTTP %%{http_code}\n" http://127.0.0.1:8000/connexion`

2026-10-08 09:30:09 **You**

Requesting new quick Tunnel on trycloudflare.com...
2026-10-08T09:24:11Z INF +--+
2026-10-08T09:24:11Z INF | Your quick Tunnel has been created! Visit it at (it may take some time to be reachable): |
2026-10-08T09:24:11Z INF | https://linked-cycles-extras-non.trycloudflare.com |
2026-10-08T09:24:11Z INF +

DeepSeek

Tunnel actif

URL de base : https://linked-cycles-extras-non.trycloudflare.com

Liens de test

Page	Lien
Accueil (patient)	https://linked-cycles-extras-non.trycloudflare.com/
Connexion	https://linked-cycles-extras-non.trycloudflare.com/connexion
Inscription	https://linked-cycles-extras-non.trycloudflare.com/inscription
Espace pro	https://linked-cycles-extras-non.trycloudflare.com/professionnel
Catalogue pro	https://linked-cycles-extras-non.trycloudflare.com/professionnel/catalogue
Demande pro	https://linked-cycles-extras-non.trycloudflare.com/professionnel/demande
Commandes pro	https://linked-cycles-extras-non.trycloudflare.com/professionnel/commandes
Établissement	https://linked-cycles-extras-non.trycloudflare.com/professionnel/etablissement
Dashboard pharmacien	https://linked-cycles-extras-non.trycloudflare.com/pharmacien
Suivi commande	https://linked-cycles-extras-non.trycloudflare.com/suivi

Identifiants

Email	Mot de passe	Rôle
pro@test.tg	Test1234	Professionnel
pharmacien@test.tg	Test1234	Pharmacien
admin@test.tg	Test1234	Admin
patient@test.tg	Test1234	Patient

Ce qu'il faut vérifier

Avant tout — Uvicorn doit tourner dans l'autre fenêtre :

```
INFO: Application startup complete.
```

Si non → lance-le :

```bat
cd /d "C:\Users\ELOGE KEY\Desktop\pharma\pharmacie"
call venv\Scripts\activate
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

Test immédiat du flash :

1. Connecte-toi en pro via `/connexion`
2. Ferme Safari complètement
3. Rouvre `https://linked-cycles-extras-non.trycloudflare.com/`
4. **Attendu** : redirection directe vers `/professionnel`, plus de flash

Si le flash persiste — on applique le fix `fix_route_root.py` que je t'ai donné. Uvicorn doit être lancé pour tester.

Mettre à jour generate_link.py

```bat
cd /d "C:\Users\ELOGE KEY\Desktop\pharma\pharmacie"
notepad generate_link.py
```

Ctrl+H → Remplacer :

- Rechercher : `PUBLIC_URL = "https://<ancienne-URL>"`
- Remplacer par : `PUBLIC_URL = "https://linked-cycles-extras-non.trycloudflare.com"`

Ctrl+S, puis :

```bat
python generate_link.py pro@test.tg --days 30
```

Le lien `/acces/...` te permettra d'accéder à l'espace pro sans mot de passe.

Rappels

- ⚠️ Les 2 fenêtres (Uvicorn + Cloudflared) doivent rester ouvertes
- ⚠️ Si tu fermes Cloudflared → nouvelle URL, ancien lien mort
- ⚠️ Si tu fermes Uvicorn → erreur 502 Bad Gateway sur toutes les pages

Colle-moi :

1. Confirmation que Uvicorn tourne
2. Ce qui se passe quand tu ouvres l'URL de base en tant que pro

3. La sortie de `python generate_link.py pro@test.tg --days 30`

2026-10-08 09:32:27 **You**

l'affichage du vieux interface persiste on fait tout les diagnostique possible en un script

DeepSeek

Diagnostic complet — Flash vieille interface

Un script qui teste tout d'un coup : local + tunnel, headers, contenu, redirection, timing, magic link.

Créer le script

```bat
cd /d "C:\Users\ELOGE KEY\Desktop\pharma\pharmacie"
notepad diag_flash_full.py
```

Colle exactement ceci, puis Ctrl+S :

```python
"""
diag_flash_full.py
Diagnostic complet du bug "vieille interface avant redirection".
Teste local + tunnel, headers, contenu, redirection, timing, magic link.
"""
import json
import re
import secrets
import sqlite3
import sys
import time
import urllib.request
import urllib.error
import http.cookiejar
from datetime import datetime, timedelta
from pathlib import Path

# ============================================================
# CONFIG
# ============================================================
```

```
BASE = Path(__file__).resolve().parent
DB = BASE / "database.db"
STATIC = BASE / "static"
LOCAL = "http://127.0.0.1:8000"
TUNNEL = "https://linked-cycles-extras-non.trycloudflare.com"
PRO_EMAIL = "pro@test.tg"

PASS = 0
FAIL = 0
WARN = 0


def sec(title):
    print()
    print("=" * 74)
    print(" " + title)
    print("=" * 74)
    print()


def check(label, ok, detail=""):
    global PASS, FAIL, WARN
    if ok == "warn":
        WARN += 1
        print(f"  [!] {label}")
    elif ok:
        PASS += 1
        print(f"  [OK] {label}")
    else:
        FAIL += 1
        print(f"  [X] {label}")
    if detail:
        print(f"        {detail}")


def fetch(url, cookie_jar=None, method="GET", follow=True, timeout=15):
    """Retourne (status, headers, body, redirect_chain, elapsed_ms)."""
    start = time.time()
    chain = []
    if cookie_jar:
        opener = urllib.request.build_opener(
            urllib.request.HTTPCookieProcessor(cookie_jar),
            urllib.request.HTTPRedirectHandler() if follow else NoRedirectHandler()
        )
    else:
        opener = urllib.request.build_opener(
            urllib.request.HTTPRedirectHandler() if follow else NoRedirectHandler()
        )
    req = urllib.request.Request(url, method=method)
```

```
    try:
        resp = opener.open(req, timeout=timeout)
        body = resp.read().decode("utf-8", errors="replace")
        elapsed = int((time.time() - start) * 1000)
        for h in resp.history:
            chain.append((h.status, h.headers.get("Location", "")))
        return resp.status, dict(resp.headers), body, chain, elapsed
    except urllib.error.HTTPError as e:
        elapsed = int((time.time() - start) * 1000)
        body = e.read().decode("utf-8", errors="replace")
        return e.code, dict(e.headers), body, chain, elapsed
    except Exception as e:
        elapsed = int((time.time() - start) * 1000)
        return 0, {}, f"ERREUR: {e}", chain, elapsed


class NoRedirectHandler(urllib.request.HTTPRedirectHandler):
    def redirect_request(self, *args, **kwargs):
        return None


def has_old_design(html):
    """Detecte l'ancien design vert."""
    markers = ["#0f9d76", "Votre pharmacie, en ligne", "prototype de demonstration"]
    count = sum(1 for m in markers if m in html)
    return count >= 1


def has_new_design(html):
    """Detecte le nouveau design bleu V6."""
    markers = ["#1a4a7f", "--blue", "acc-welcome", "acc-guide", "gw-card"]
    return any(m in html for m in markers)


def has_redirect_script(html):
    """Detecte un JS de redirection."""
    patterns = [
        r'window\.location\.(replace|href)\s*=\s*["\']',
        r'location\.replace\s*\(',
    ]
    for p in patterns:
        if re.search(p, html):
            return True
    return False


def extract_redirects(html):
    """Extrait les URLs de redirection trouvees dans le JS."""
    urls = re.findall(r'window\.location\.(?:replace|href)\s*=\s*["\']([^"\']+)["\']', html)
```

```
    return urls[:5]


# ============================================================
# 1. VERIFIER UVICORN
# ============================================================
sec("1. UVICORN (local)")

status, headers, body, _, ms = fetch(LOCAL + "/")
check("Uvicorn repond sur /", status == 200, f"status={status} ({ms}ms)")
if status != 200:
    print()
    print("  >>> Uvicorn n'est pas lance. Arrete ici.")
    sys.exit(1)

# ============================================================
# 2. FICHIERS DISQUE
# ============================================================
sec("2. FICHIERS DISQUE")

for name in ("index.html", "professional.html", "login.html", "register.html"):
    p = STATIC / name
    if not p.exists():
        check(f"{name} existe", False)
        continue
    content = p.read_text(encoding="utf-8")
    size = len(content)
    old = has_old_design(content)
    new = has_new_design(content)
    redir = has_redirect_script(content)
    flags = []
    if old: flags.append("VIEUX")
    if new: flags.append("nouveau")
    if redir: flags.append("JS-redirect")
    check(f"{name:22s} {size:>6}o [{', '.join(flags) or 'neutre'}]", True)

# ============================================================
# 3. HEADERS LOCAL vs TUNNEL
# ============================================================
sec("3. HEADERS / (LOCAL)")

status, headers, body_local, chain, ms = fetch(LOCAL + "/")
cc = headers.get("cache-control", headers.get("Cache-Control", ""))
csd = headers.get("clear-site-data", headers.get("Clear-Site-Data", ""))
loc = headers.get("location", headers.get("Location", ""))
ctype = headers.get("content-type", headers.get("Content-Type", ""))[:40]

print(f"  status      = {status}  ({ms}ms)")
print(f"  ctype       = {ctype}")
```

```
print(f"  cache-control = {cc}")
print(f"  clear-site-data = {csd}")
print(f"  location    = {loc}")
print(f"  taille body = {len(body_local)}")
print(f"  redirect chain = {chain}")

check("cache-control contient no-store", "no-store" in cc.lower())
check("clear-site-data present", "cache" in csd.lower())
check("/ ne fait PAS de 30x", not (300 <= status < 400),
      f"status={status}")

# ============================================================
# 4. CONTENU SERVI SUR /
# ============================================================
sec("4. CONTENU SERVI SUR / (LOCAL)")

old_in_served = has_old_design(body_local)
new_in_served = has_new_design(body_local)
redir_in_served = has_redirect_script(body_local)
redirects = extract_redirects(body_local)

check("Nouveau design (bleu) servi", new_in_served)
check("Ancien design (vert) ABSENT", not old_in_served,
      "Le vieux design est servi sur / - C'EST LA CAUSE DU FLASH")
check("Script de redirection present", redir_in_served)
if redirects:
    print(f"  Redirections JS detectees :")
    for r in redirects:
        print(f"    -> {r}")

if old_in_served and not new_in_served:
    print()
    print("  >>> DIAGNOSTIC : / sert l'ANCIEN index.html.")
    print("      Le navigateur affiche ce vieux contenu pendant que le JS")
    print("      de redirection fait son travail (300-800 ms).")
    print("      C'EST LE FLASH.")

# ============================================================
# 5. HEADERS / TUNNEL
# ============================================================
sec("5. HEADERS / (TUNNEL)")

status_t, headers_t, body_tunnel, chain_t, ms_t = fetch(TUNNEL + "/")
cc_t = headers_t.get("cache-control", headers_t.get("Cache-Control", ""))
csd_t = headers_t.get("clear-site-data", headers_t.get("Clear-Site-Data", ""))
ctype_t = headers_t.get("content-type", headers_t.get("Content-Type", ""))[:40]

print(f"  status      = {status_t}  ({ms_t}ms)")
print(f"  ctype       = {ctype_t}")
```

```
print(f"  cache-control = {cc_t}")
print(f"  clear-site-data = {csd_t}")
print(f"  taille body = {len(body_tunnel)}")
print(f"  redirect chain = {chain_t}")

check("Tunnel repond", status_t == 200, f"status={status_t} ({ms_t}ms)")
check("Tunnel < 3s", ms_t < 3000, f"{ms_t}ms")
check("Cache-Control no-store via tunnel", "no-store" in cc_t.lower())
check("Clear-Site-Data via tunnel", "cache" in csd_t.lower())

# Comparer local vs tunnel
if body_local == body_tunnel:
    check("Contenu local == tunnel", True)
else:
    check("Contenu local == tunnel", False,
          f"local={len(body_local)}o tunnel={len(body_tunnel)}o")

# ============================================================
# 6. ROUTE / POUR UN PRO CONNECTE
# ============================================================
sec("6. ROUTE / AVEC COOKIE PRO")

# Creer une session pour pro@test.tg
conn = sqlite3.connect(DB)
conn.row_factory = sqlite3.Row
try:
    user = conn.execute(
        "SELECT id, email FROM users WHERE email=?", (PRO_EMAIL,)
    ).fetchone()
    if not user:
        check(f"{PRO_EMAIL} existe en base", False)
    else:
        check(f"{PRO_EMAIL} existe", True, f"id={user['id']}")
        token = secrets.token_urlsafe(48)
        now = datetime.now()
        exp = now + timedelta(hours=1)
        conn.execute(
            "INSERT INTO sessions (token, user_id, created_at, expires_at)"
            " VALUES (?, ?, ?, ?)",
            (token, user["id"],
             now.isoformat(timespec="seconds"),
             exp.isoformat(timespec="seconds")),
        )
        conn.commit()
        print(f"  Session de test : token={token[:16]}...")
        print()

        # Se connecter avec cookie au tunnel (sans suivre les redirects)
        jar = http.cookiejar.CookieJar()
```

```
        cookie = http.cookiejar.Cookie(
            version=0, name="elokinab_session", value=token,
            port=None, port_specified=False,
            domain="linked-cycles-extras-non.trycloudflare.com",
            domain_specified=True, domain_initial_dot=False,
            path="/", path_specified=True,
            secure=True, expires=None, discard=False,
            comment=None, comment_url=None, rest={},
        )
        jar.set_cookie(cookie)

        # Test 1 : sans suivi de redirect
        status_pro, headers_pro, body_pro, chain_pro, ms_pro = fetch(
            TUNNEL + "/", cookie_jar=jar, follow=False
        )
        loc_pro = headers_pro.get("location", headers_pro.get("Location", ""))

        print(f"  Sans suivre les redirects :")
        print(f"    status      = {status_pro}")
        print(f"    location    = {loc_pro}")
        print(f"    taille      = {len(body_pro)}")
        print(f"    elapsed     = {ms_pro}ms")
        print()

        if 300 <= status_pro < 400:
            check("Pro -> redirection 30x cote serveur", True,
                  f"-> {loc_pro}")
            if loc_pro == "/professionnel":
                check("Redirection vers /professionnel", True)
            else:
                check(f"Redirection vers {loc_pro}", False,
                      "Attendu : /professionnel")
        else:
            check("Pro -> redirection 30x cote serveur", False,
                  f"status={status_pro} (pas de redirection serveur)")
            print()
            print("  >>> PROBLEME : le serveur renvoie la page / sans rediriger.")
            print("      Le flash est INEVITABLE : le navigateur affiche")
            print("      / puis un JS doit le rediriger vers /professionnel.")
            print()
            print("      Solution : faire une redirection SERVEUR (302) sur /")
            print("      pour les utilisateurs PRO connectes.")
finally:
    conn.close()

# ============================================================
# 7. MAGIC LINK FLOW
# ============================================================
sec("7. MAGIC LINK FLOW")
```

```
conn = sqlite3.connect(DB)
conn.row_factory = sqlite3.Row
try:
    user = conn.execute(
        "SELECT id FROM users WHERE email=?", (PRO_EMAIL,)
    ).fetchone()
    if user:
        token = secrets.token_urlsafe(32)
        now = datetime.now()
        exp = (now + timedelta(hours=1)).isoformat(timespec="seconds")
        conn.execute(
            "INSERT INTO magic_links (token, user_id, created_at, expires_at,"
            " max_uses, used_count, revoked, note)"
            " VALUES (?, ?, ?, ?, 0, 0, 0, 'diag_full')",
            (token, user["id"], now.isoformat(timespec="seconds"), exp),
        )
        conn.commit()

        # Tester /acces/{token} sans suivre
        jar = http.cookiejar.CookieJar()
        url_acces = TUNNEL + "/acces/" + token
        status_m, headers_m, body_m, chain_m, ms_m = fetch(
            url_acces, cookie_jar=jar, follow=False
        )
        loc_m = headers_m.get("location", headers_m.get("Location", ""))

        print(f"  /acces/{{token}} :")
        print(f"    status   = {status_m}  ({ms_m}ms)")
        print(f"    location = {loc_m}")
        print(f"    taille   = {len(body_m)}")
        print()

        if 300 <= status_m < 400:
            check("Magic link redirige par 302 (mauvais pour cache)", "warn",
                  f"-> {loc_m}")
            print("        >>> Recupere une URL FIXE -> le navigateur peut")
            print("        >>> resservir un vieux /professionnel en cache.")
            print("        >>> Solution : renvoyer un HTML + JS avec ?_v=timestamp")
        elif status_m == 200:
            has_js = has_redirect_script(body_m)
            redirects_m = extract_redirects(body_m)
            check("Magic link redirige via HTML+JS (bon)", has_js)
            if redirects_m:
                print(f"    Redirection JS : {redirects_m[0][:80]}")
                if "_v=" in redirects_m[0]:
                    check("Cache-bust ?_v= dans l'URL", True)
                else:
                    check("Cache-bust ?_v= dans l'URL", False,
```

```
                        "Ajoute ?_v=<timestamp> dans magic_links.py")
        else:
            check(f"Magic link status", False, f"status={status_m}")

        # Cookie pose ?
        cookie_names = [c.name for c in jar]
        check("Cookie session pose par /acces/",
              "elokinab_session" in cookie_names,
              f"cookies: {cookie_names}")
finally:
    conn.close()

# ============================================================
# 8. TIMING TUNNEL
# ============================================================
sec("8. TIMING TUNNEL (5 requetes sur /)")

times = []
for i in range(5):
    _, _, _, _, ms = fetch(TUNNEL + "/")
    times.append(ms)
print(f"  Temps : {times}")
print(f"  Min : {min(times)}ms / Max : {max(times)}ms / Moy : {sum(times)//len(times)}ms")
print()
if max(times) > 3000:
    check("Tunnel stable (< 3s)", False, f"max={max(times)}ms")
elif max(times) - min(times) > 1000:
    check("Tunnel stable", "warn", f"variance={max(times)-min(times)}ms")
else:
    check("Tunnel stable", True)

# ============================================================
# 9. DETECTION SERVICE WORKER + CACHE
# ============================================================
sec("9. SERVICE WORKER")

sw_files = list(STATIC.glob("sw*.js")) + list(STATIC.glob("service-worker*.js"))
if sw_files:
    for f in sw_files:
        check(f"SW sur disque : {f.name}", "warn", "Peut causer du cache persistant")
else:
    check("Aucun Service Worker sur disque", True)

# ============================================================
# 10. DOUBLONS DE FICHIERS
# ============================================================
sec("10. DOUBLONS")

# Compter les .bak
```

```
baks = list(STATIC.glob("*.bak")) + list(STATIC.glob("*.bak*"))
print(f"  {len(baks)} fichiers .bak dans static/")

# Doublons index.html
idxs = sorted(BASE.rglob("index.html"))
print(f"  {len(idxs)} fichiers index.html dans le projet :")
for i in idxs:
    rel = i.relative_to(BASE)
    print(f"    {str(rel):60s}  {i.stat().st_size:>7} o")

# ============================================================
# RESUME
# ============================================================
sec("RESUME FINAL")

print(f"  PASS  : {PASS}")
print(f"  FAIL  : {FAIL}")
print(f"  WARN  : {WARN}")
print()

if FAIL == 0 and WARN == 0:
    print("  >>> AUCUN probleme detecte.")
else:
    print("  >>> Corrige les [X] et [i] ci-dessus.")
    print()

print("=" * 74)
print(" Colle-moi l'integralite de la sortie.")
print("=" * 74)
```

Lancer

```
bat

cd /d "C:\Users\ELOGE KEY\Desktop\pharma\pharmacie"
python diag_flash_full.py
```

Ce que le script teste

#	Test	Ce que ça vérifie
1	Uvicorn répond	Local 200 OK
2	Fichiers disque	index.html contient vieux ou nouveau design
3	Headers local	cache-control, clear-site-data
4	Contenu servi sur /	Vieux design = cause du flash

#	Test	Ce que ça vérifie
5	Headers via tunnel	Même chose, à travers Cloudflared
6	Route `/` avec cookie PRO	Redirige 302 ou renvoie HTML ?
7	Magic link flow	302 ou HTML+JS avec `?_v=` ?
8	Timing tunnel	5 requêtes pour mesurer la stabilité
9	Service Worker	Présence de SW qui cache
10	Doublons	Nombre de .bak et index.html

Ce que le résultat va nous dire

Cause A — Section 4 : `Ancien design servi sur /` = le flash vient du serveur qui sert `index.html` au lieu de rediriger

Cause B — Section 6 : `/` avec cookie PRO renvoie `status=200` (pas de 302) = le serveur ne redirige pas, tout dépend du JS

Cause C — Section 7 : magic link renvoie `302` sans `?_v=` = le navigateur peut resservir un vieux `/professionnel`

Cause D — Section 9 : Service Worker trouvé = cache persistant même en navigation privée

Cause E — Section 8 : tunnel lent ou instable = l'affichage du vieux HTML est long

---

Colle-moi :

1. La sortie complète de `python diag_flash_full.py`
2. Le résultat du résumé final (`PASS` / `FAIL` / `WARN`)

Avec ça, je te livre le fix exact, en une seule commande.

2026-10-08 09:33:56 **You**

```
:\Users\ELOGE KEY\Desktop\pharma\pharmacie>python
diag_flash_full.py

============================================================
====================
 1. UVICORN (local)
============================================================
====================
```

```
  [X] Uvicorn repond sur /
      status=0 (248ms)

  >>> Uvicorn n'est pas lance. Arrete ici.

C:\Users\ELOGE KEY\Desktop\pharma\pharmacie>cd /d
"C:\Users\ELOGE KEY\Desktop\pharma\pharmacie"

C:\Users\ELOGE KEY\Desktop\pharma\pharmacie>python
diag_flash_full.py

============================================================
====================
 1. UVICORN (local)
============================================================
====================

  [X] Uvicorn repond sur /
      status=0 (56ms)

  >>> Uvicorn n'est pas lance. Arrete ici.

C:\Users\ELOGE KEY\Desktop\pharma\pharmacie>
```

DeepSeek

Uvicorn n'est pas lancé

Le diagnostic s'arrête dès le test 1 : `status=0` = rien n'écoute sur `127.0.0.1:8000`.

C'est **impossible de tester quoi que ce soit** sans Uvicorn. Le tunnel Cloudflared pointe vers `127.0.0.1:8000` mais rien ne répond → le navigateur reçoit un 502 mis en cache → **c'est pour ça que tu vois le vieux flash**.

Fix — Lancer Uvicorn d'abord

Étape 1 — Ouvrir un CMD (Terminal 1)

```bat
```

```
cd /d "C:\Users\ELOGE KEY\Desktop\pharma\pharmacie"
call venv\Scripts\activate
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

Attends de voir ces 3 lignes :

```
INFO: Started server process [xxxx]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000
```

Laisse cette fenêtre ouverte. Ne la ferme pas.

Étape 2 — Vérifier dans un autre CMD

Ouvre un **deuxième CMD** et colle :

```
bat

curl -s -o nul -w "HTTP %%{http_code}\n" http://127.0.0.1:8000/
```

Attendu : `HTTP 200`

Si tu vois autre chose → colle-moi les 10 dernières lignes de la fenêtre Uvicorn.

Étape 3 — Relancer le diagnostic

Une fois qu'Uvicorn répond, dans le CMD 2 :

```
bat

cd /d "C:\Users\ELOGE KEY\Desktop\pharma\pharmacie"
python diag_flash_full.py
```

Cette fois le script ira au bout et nous dira **exactement** pourquoi le flash persiste.

Si Uvicorn plante au démarrage

Colle-moi les 10 dernières lignes de la fenêtre Uvicorn (avec les erreurs éventuelles). Erreurs fréquentes :

Erreur	Cause	Fix
`ModuleNotFoundError`	package manquant	`pip install -r requirements.txt`
SyntaxError	main.py cassé par un patch	on restaure depuis .bak
`Address already in use`	autre Python sur le port	`taskkill /F /IM python.exe`
sqlite3.OperationalError	DB verrouillée	ferme les autres sessions

Erreur	Cause	Fix
Rien du tout (silencieux)	venv cassé	recréer venv

Vérifie aussi que Cloudflared tourne

Dans la fenêtre Cloudflared, tu dois toujours voir :

`INF Registered tunnel connection connIndex=0 ... protocol=quic`

Si cette fenêtre est fermée, relance :

```bat
"C:\Program Files (x86)\cloudflared\cloudflared.exe" tunnel --url http://127.0.0.1:8000 --retries 5
```

⚠️ Ça générera une **nouvelle URL** — il faudra mettre à jour `generate_link.py` .

Résumé

Tu ne peux pas tester le flash tant qu'Uvicorn ne tourne pas. Le flash que tu vois est probablement juste le navigateur qui affiche un 502 mis en cache (le tunnel ne trouve personne à `127.0.0.1:8000`).

Lance Uvicorn, vérifie avec curl, puis relance `diag_flash_full.py` . Colle-moi la sortie complète.

2026-10-08 09:34:59 **You**

pourtant uvicorn etait lancé: [33mWARNING←[0m: StatReload detected changes in 'diag_flash_full.py'. Reloading...
 ←[32mINFO←[0m: Shutting down
←[32mINFO←[0m: Waiting for application shutdown.
←[32mINFO←[0m: Application shutdown complete.
←[32mINFO←[0m: Finished server process [←[36m7556←[0m]
←[32mINFO←[0m: Started server process [←[36m8160←[0m]
←[32mINFO←[0m: Waiting for application startup.
←[32mINFO←[0m: Application startup complete.
←[32mINFO←[0m: 127.0.0.1:60745 - "←[1mGET / HTTP/1.1←[0m" ←[32m200 OK←[0m
←[32mINFO←[0m: 127.0.0.1:60746 - "←[1mGET / HTTP/1.1←[0m" ←[32m200 OK←[0m

DeepSeek

Trouvé — `--reload` tue Uvicorn pendant le diagnostic

Regarde tes logs :

```
StatReload detected changes in 'diag_flash_full.py'. Reloading...
Shutting down
Application shutdown complete.
```

Uvicorn redémarre à chaque fois que tu crées ou lances `diag_flash_full.py` parce que `–reload` surveille **tous les `.py`** du dossier — y compris ton script de diagnostic.

Résultat :

1. Tu lances `python diag_flash_full.py`
2. Uvicorn détecte le fichier → commence à redémarrer
3. Le diagnostic essaie de contacter `127.0.0.1:8000` → **status=0** (serveur down)
4. Uvicorn finit de redémarrer → 200 OK (trop tard)

Fix — Placer le diagnostic hors du dossier surveillé

Étape 1 — Déplacer le script

```bat
cd /d "C:\Users\ELOGE KEY\Desktop\pharma\pharmacie"
move diag_flash_full.py "C:\Users\ELOGE KEY\Desktop\pharma\diag_flash_full.py"
```

Étape 2 — Éditer le chemin du projet

```bat
notepad "C:\Users\ELOGE KEY\Desktop\pharma\diag_flash_full.py"
```

Cherche cette ligne :

```python
BASE = Path(__file__).resolve().parent
```

Remplace par :

```python
BASE = Path(r"C:\Users\ELOGE KEY\Desktop\pharma\pharmacie")
```

Ctrl+S.

Étape 3 — Lancer le diagnostic depuis l'extérieur

Ouvre un CMD (peu importe où) :

```bat
cd /d "C:\Users\ELOGE KEY\Desktop\pharma"
python diag_flash_full.py
```

Cette fois, aucun fichier `.py` n'est créé dans `pharmacie/`, donc Uvicorn ne recharge pas.

Alternative — Vérifier manuellement qu'Uvicorn répond AVANT de lancer le diag

Ouvre un CMD et teste :

```bat
curl -s -o nul -w "HTTP %%{http_code}\n" http://127.0.0.1:8000/
```

- `HTTP 200` → Uvicorn est OK, lance le diagnostic immédiatement (moins de 3 secondes après)
- `HTTP 000` ou rien → Uvicorn est en train de redémarrer, attends 2 sec et reteste

Alternative définitive — Redémarrer Uvicorn sans `--reload`

Pour éviter définitivement ce problème :

1. **Ferme la fenêtre Uvicorn** (Ctrl+C dedans)
2. Relance **sans** `--reload` :

```bat
cd /d "C:\Users\ELOGE KEY\Desktop\pharma\pharmacie"
call venv\Scripts\activate
uvicorn main:app --host 0.0.0.0 --port 8000
```

Attends :

`INFO: Application startup complete.`

Sans `--reload`, Uvicorn ne redémarrera pas quand tu crées des fichiers `.py`. Tu perds la recharge auto, mais pour tester le diagnostic, c'est parfait.

Tu redémarreras manuellement quand tu changes `main.py` (ce qui est rare maintenant).

Résumé

Problème	Cause	Fix
`status=0`	`--reload` tue Uvicorn quand tu crées le script	Place le diag hors du dossier, ou lance Uvicorn sans `--reload`

Prochaine action :

1. Ferme la fenêtre Uvicorn (Ctrl+C)
2. Relance **sans** **`--reload`** :

```bat
cd /d "C:\Users\ELOGE KEY\Desktop\pharma\pharmacie"
call venv\Scripts\activate
uvicorn main:app --host 0.0.0.0 --port 8000
```

3. **Attends** **`INFO: Application startup complete.`**
4. **NE TOUCHE PLUS à cette fenêtre**
5. Dans un autre CMD :

```bat
cd /d "C:\Users\ELOGE KEY\Desktop\pharma\pharmacie"
python diag_flash_full.py
```

Colle-moi la sortie complète du diagnostic. Cette fois Uvicorn sera stable.

2026-10-08 09:37:10 **You**

C:\Users\ELOGE KEY\Desktop\pharma\pharmacie>cd /d "C:\Users\ELOGE KEY\Desktop\pharma\pharmacie"

C:\Users\ELOGE KEY\Desktop\pharma\pharmacie>move diag_flash_full.py "C:\Users\ELOGE KEY\Desktop\pharma\diag_flash_full.py"
1 fichier(s) déplacé(s).

C:\Users\ELOGE KEY\Desktop\pharma\pharmacie>notepad "C:\Users\ELOGE KEY\Desktop\pharma\diag_flash_full.py"

C:\Users\ELOGE KEY\Desktop\pharma\pharmacie>cd /d

```
"C:\Users\ELOGE KEY\Desktop\pharma"

C:\Users\ELOGE KEY\Desktop\pharma>python diag_flash_full.py

================================================================
 1. UVICORN (local)
================================================================

  [X] Uvicorn repond sur /
      status=0 (57ms)

  >>> Uvicorn n'est pas lance. Arrete ici.

C:\Users\ELOGE KEY\Desktop\pharma>
```

DeepSeek

Vérifier si Uvicorn tourne vraiment

Le diagnostic dit `status=0` = personne n'écoute sur `127.0.0.1:8000`.

Test 1 — Vérifier manuellement

Ouvre un **nouveau CMD** et colle :

```bat
curl -s -o nul -w "HTTP %%{http_code}\n" http://127.0.0.1:8000/
```

Réponds-moi avec ce que ça affiche :

Résultat	Signification
`HTTP 200`	Uvicorn tourne → le diag a un bug
`HTTP 000`	Uvicorn ne répond pas
Rien du tout	Uvicorn ne répond pas
`curl: (7) Failed to connect`	Rien n'écoute sur le port 8000

Test 2 — Vérifier que Uvicorn écoute

```bat
netstat -ano | findstr :8000
```

Réponds-moi avec ce que ça affiche :

Résultat	Signification
`TCP 0.0.0.0:8000 ... LISTENING`	Uvicorn écoute bien
`TCP 127.0.0.1:8000 ... LISTENING`	Uvicorn écoute en local seulement
Rien	Uvicorn n'est PAS lancé

Test 3 — Vérifier les processus Python

```bat
tasklist | findstr python.exe
```

Réponds-moi avec ce que ça affiche :

Résultat	Signification
1 ou 2 lignes `python.exe`	Uvicorn tourne
Rien	Aucun Python → Uvicorn n'est pas lancé

Test 4 — Ouvrir dans le navigateur

Sur ton PC, ouvre **Edge** et tape :

`http://127.0.0.1:8000/`

Réponds-moi :

- ✅ La page s'affiche → Uvicorn tourne → le diag a un bug (probablement un problème de proxies Windows)
- ❌ Erreur 502/refus de connexion → Uvicorn n'est pas lancé
- ❌ Page blanche → Uvicorn répond mais plante

Si Uvicorn n'est pas lancé

Ouvre la fenêtre Uvicorn et dis-moi ce qu'elle affiche. Colle-moi les 20 dernières lignes.

Erreurs possibles :

Erreur	Cause	Fix
`Address already in use`	Le port 8000 est pris	`taskkill /F /IM python.exe` puis relance
ModuleNotFoundError	Package manquant	pip install -r requirements.txt
`SyntaxError`	main.py cassé	restauration depuis .bak
Traceback ... sqlite3	DB verrouillée	ferme les autres Python
`Application shutdown complete`	Crash silencieux	regarde les 20 lignes au-dessus
Rien / fenêtre fermée	Uvicorn pas lancé	relance

Lance Uvicorn proprement

Ferme toutes les fenêtres Python. Ouvre un **CMD tout neuf** :

```bat
taskkill /F /IM python.exe
cd /d "C:\Users\ELOGE KEY\Desktop\pharma\pharmacie"
call venv\Scripts\activate
uvicorn main:app --host 0.0.0.0 --port 8000
```

Sans `--reload`. Attends de voir :

```
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO: Application startup complete.
```

NE FERME PAS CETTE FENÊTRE.

Ensuite, dans un autre CMD, lance :

```bat
curl -s -o nul -w "HTTP %%{http_code}\n" http://127.0.0.1:8000/
```

Colle-moi :

1. Ce qu'affiche `curl` (Test 1)
2. Ce qu'affiche `netstat -ano | findstr :8000` (Test 2)
3. Ce qu'affiche `tasklist | findstr python.exe` (Test 3)
4. Si http://127.0.0.1:8000/ s'ouvre dans le navigateur (Test 4)
5. Les 20 dernières lignes de la fenêtre Uvicorn si elle est ouverte

2026-10-08 09:38:08 **You**

```
C:\Users\ELOGE KEY\Desktop\pharma>curl -s -o nul -w "HTTP %%{http_code}\n" http://127.0.0.1:8000/
HTTP %{http_code}

C:\Users\ELOGE KEY\Desktop\pharma>netstat -ano | findstr :8000
  TCP    0.0.0.0:8000           0.0.0.0:0              LISTENING       4440
  TCP    127.0.0.1:60764        127.0.0.1:8000         TIME_WAIT       0
  TCP    127.0.0.1:60779        127.0.0.1:8000         TIME_WAIT       0

C:\Users\ELOGE KEY\Desktop\pharma>tasklist | findstr python.exe
python.exe                   5068 Console                    1     3 420 Ko
python.exe                   4440 Console                    1    29 164 Ko
python.exe                   8160 Console                    1    51 764 Ko

C:\Users\ELOGE KEY\Desktop\pharma>
```

DeepSeek

Le curl est cassé par `%%` — refais le test avec `%`

En CMD direct, `%%` s'affiche littéralement (`%{http_code}`) et ne teste **rien**. Il faut un seul `%` (le double `%%` n'est valable que dans un fichier `.bat`).

Test propre — 1 commande

```bat
curl -s -o nul -w "HTTP %{http_code}\n" http://127.0.0.1:8000/
```

Réponds-moi avec ce qui s'affiche. Attendu : **`HTTP 200`** .

Mais aussi — 3 processus Python qui tournent

```
PID 5068  (3 MB)   ← StatReload parent
PID 4440  (29 MB)  ← Uvicorn actif (LISTENING)
PID 8160  (51 MB)  ← ???
```

Le PID 8160 (51 MB) est suspect. **Un seul Uvicorn devrait tourner.** Soit c'est un vieux process zombie, soit c'est ton `diag_flash_full.py` resté bloqué.

Fix — Nettoyer et repartir

1. Tuer TOUS les Python

```bat
taskkill /F /IM python.exe
```

Vérifie qu'il ne reste rien :

```bat
tasklist | findstr python.exe
```

Doit afficher **rien**.

2. Relancer Uvicorn sans `--reload`

Ouvre un **CMD tout neuf** :

```bat
cd /d "C:\Users\ELOGE KEY\Desktop\pharma\pharmacie"
call venv\Scripts\activate
uvicorn main:app --host 0.0.0.0 --port 8000
```

Attends de voir :

```
INFO: Uvicorn running on http://0.0.0.0:8000
INFO: Application startup complete.
```

Ne touche plus à cette fenêtre. Sans `–reload`, elle ne redémarrera pas.

3. Vérifier dans un autre CMD

```bat
curl -s -o nul -w "HTTP %{http_code}\n" http://127.0.0.1:8000/
```

Attendu : `HTTP 200`

4. Tester dans le navigateur

Sur ton PC, ouvre Edge :

`http://127.0.0.1:8000/`

Réponds-moi :

- ✅ La page s'affiche (vieux design vert ou nouveau) → Uvicorn OK

- ❌ Erreur 502 / refus de connexion → problème de proxy Windows

5. Si le navigateur marche mais pas le diag

C'est un **proxy Windows** qui intercepte `127.0.0.1`. Fix dans le diagnostic (à venir une fois Uvicorn stable).

Colle-moi, dans l'ordre :

1. Le résultat de `curl -s -o nul -w "HTTP %{http_code}\n" http://127.0.0.1:8000/` (avec **un seul** `%`)
2. Le résultat de `tasklist | findstr python.exe` après le `taskkill`
3. Ce qu'affiche `http://127.0.0.1:8000/` dans Edge
4. Les 5 dernières lignes de la fenêtre Uvicorn

Avec ça je saurai si c'est Uvicorn, un proxy, ou le diagnostic lui-même.

2026-10-08 09:38:55 **You**

C:\Users\ELOGE KEY\Desktop\pharma>curl -s -o nul -w "HTTP %{http_code}\n" http://127.0.0.1:8000/
HTTP 200

C:\Users\ELOGE KEY\Desktop\pha

DeepSeek

Trouvé — Proxy Windows qui casse `urllib`

`curl` → `HTTP 200` ✅
`urllib` (dans le diag Python) → `status=0` ❌

Cause : Windows a un proxy configuré (visible dans les paramètres Internet) que `urllib` utilise par défaut. `curl` a sa propre config, donc il contourne. `urllib` essaie de passer par le proxy → le proxy refuse `127.0.0.1`.

Fix — Désactiver les proxies dans le diagnostic

Étape 1 — Éditer le script

```bat
notepad "C:\Users\ELOGE KEY\Desktop\pharma\diag_flash_full.py"
```

Cherche la ligne :

```python
import urllib.request
```

Ajoute **juste en dessous** (au même niveau) :

```python
# Désactiver les proxies Windows pour toutes les requêtes urllib
import os
os.environ["NO_PROXY"] = "127.0.0.1,localhost,*.trycloudflare.com"
os.environ["no_proxy"] = "127.0.0.1,localhost,*.trycloudflare.com"
urllib.request.install_opener(urllib.request.build_opener(urllib.request.ProxyHandler({})))
```

⚠️ Attention à l'ordre : il faut d'abord importer `os` (ou ajouter `import os` en haut du fichier si pas déjà présent).

Plus simple — remplace la ligne `import urllib.request` par ce bloc complet :

```python
import os
os.environ["NO_PROXY"] = "127.0.0.1,localhost,*.trycloudflare.com"
os.environ["no_proxy"] = "127.0.0.1,localhost,*.trycloudflare.com"
import urllib.request
urllib.request.install_opener(urllib.request.build_opener(urllib.request.ProxyHandler({})))
```

Ctrl+S.

Étape 2 — Relancer

```bat
cd /d "C:\Users\ELOGE KEY\Desktop\pharma"
python diag_flash_full.py
```

Cette fois le test 1 doit passer et le diag ira jusqu'au bout.

Alternative — Vérifier le proxy actuel

Si tu veux comprendre ce qui se passe :

```bat
netsh winhttp show proxy
```

Si ça affiche une ligne comme `Proxy Server(s) : http=xxx:8080`, c'est bien ça qui casse `urllib`.

Alternative — Désactiver le proxy Windows

1. Touche Windows → tape **Proxy** → ouvre **Paramètres proxy**
2. Sous **Configuration manuelle du proxy** → désactive **Utiliser un serveur proxy**
3. Enregistrer

Relance ensuite le diagnostic **sans** modifier le script. Ça devrait marcher.

Colle-moi :

1. La sortie de `netsh winhttp show proxy`
2. Après avoir modifié le script (ou désactivé le proxy), la nouvelle sortie de `python diag_flash_full.py`

2026-10-08 09:40:28 **You**

```
C:\Users\ELOGE KEY\Desktop\pharma>cd /d "C:\Users\ELOGE
KEY\Desktop\pharma"

C:\Users\ELOGE KEY\Desktop\pharma>python diag_flash_full.py

======================================================
====================
 1. UVICORN (local)
======================================================
====================

  [X] Uvicorn repond sur /
      status=0 (52ms)

  >>> Uvicorn n'est pas lance. Arrete ici.

C:\Users\ELOGE KEY\Desktop\pharma>
```

DeepSeek

Test rapide — Python peut-il joindre 127.0.0.1 ?

Avant de continuer, on va isoler le problème avec 3 tests ciblés.

Test 1 — Python tout simple

```bat
python -c "import urllib.request; print(urllib.request.urlopen('http://127.0.0.1:8000/', timeout=5).status)"
```

Réponds-moi :

- `200` → Python joint Uvicorn (le diag a un autre bug)
- `Connexion refusée` / erreur → Python ne joint pas (proxy/AV)

Test 2 — Vérifier les variables proxy

```bat
set | findstr /I proxy
```

Colle-moi le résultat. S'il y a `HTTP_PROXY` ou `HTTPS_PROXY` configurés, c'est ça.

Test 3 — Vérifier si ta modification a été enregistrée

```bat
findstr /N /C:"ProxyHandler" "C:\Users\ELOGE KEY\Desktop\pharma\diag_flash_full.py"
```

Réponds-moi :

- Ligne(s) trouvée(s) → ton edit a bien pris
- Rien → l'edit n'a pas été sauvegardé

Test 4 — Vérifier avec requests (si installé)

```bat
python -c "import requests; r=requests.get('http://127.0.0.1:8000/', timeout=5); print(r.status_code)" 2>nul || echo "requests pas installé"
```

Fix alternatif — Patch chirurgical du diagnostic

Ouvre le fichier et remplace la fonction `fetch` complète par cette version **qui ignore totalement les proxies** :

```bat
```

```
notepad "C:\Users\ELOGE KEY\Desktop\pharma\diag_flash_full.py"
```

Cherche :

```
python
def fetch(url, cookie_jar=None, method="GET", follow=True, timeout=15):
```

Remplace **toute la fonction** par :

```
python
def fetch(url, cookie_jar=None, method="GET", follow=True, timeout=15):
    """Retourne (status, headers, body, redirect_chain, elapsed_ms)."""
    import time
    start = time.time()
    chain = []
    # Handler qui ignore les proxies
    class _NoProxy(urllib.request.ProxyHandler):
        def __init__(self): super().__init__({})
    handlers = [_NoProxy()]
    if cookie_jar:
        handlers.append(urllib.request.HTTPCookieProcessor(cookie_jar))
    if follow:
        handlers.append(urllib.request.HTTPRedirectHandler())
    else:
        handlers.append(NoRedirectHandler())
    opener = urllib.request.build_opener(*handlers)
    req = urllib.request.Request(url, method=method)
    try:
        resp = opener.open(req, timeout=timeout)
        body = resp.read().decode("utf-8", errors="replace")
        elapsed = int((time.time() - start) * 1000)
        for h in resp.history:
            chain.append((h.status, h.headers.get("Location", "")))
        return resp.status, dict(resp.headers), body, chain, elapsed
    except urllib.error.HTTPError as e:
        elapsed = int((time.time() - start) * 1000)
        body = e.read().decode("utf-8", errors="replace")
        return e.code, dict(e.headers), body, chain, elapsed
    except Exception as e:
        elapsed = int((time.time() - start) * 1000)
        return 0, {}, f"ERREUR: {e}", chain, elapsed
```

Ctrl+S. Puis :

```
bat
```

```
cd /d "C:\Users\ELOGE KEY\Desktop\pharma"
python diag_flash_full.py
```

Pourquoi ça arrive

Outil	Config proxy	Résultat
`curl`	Ignore les proxies Windows par défaut	✅ 200
urllib Python	Lit le proxy système (registre Windows)	❌ Status 0
`requests`	Lit aussi le proxy système	❌ Status 0

Le fix dans le `fetch` du diag crée un **opener sans proxy** qui contourne ça.

Colle-moi :

1. Le résultat de
   `python -c "import urllib.request; print(urllib.request.urlopen('http://127.0.0.1:8000/', timeout=5).status)"`
2. Le résultat de `set | findstr /I proxy`
3. Le résultat de
   `findstr /N /C:"ProxyHandler" "C:\Users\ELOGE KEY\Desktop\pharma\diag_flash_full.py"`

Avec ces 3 infos je saurai exactement ce qui bloque.

2026-10-08 09:42:13 **You**

arma>python -c "import urllib.request;
print(urllib.request.urlopen('http://127.0.0.1:8000/',
timeout=5).status)"
200

C:\Users\ELOGE KEY\Desktop\pha
C:\Users\ELOGE KEY\Desktop\pharma>set | findstr /I proxy

C:\Users\ELOGE KEY\Desktop\pharma>

C:\Users\ELOGE KEY\Desktop\pharma>findstr /N
/C:"ProxyHandler" "C:\Users\ELOGE

KEY\Desktop\pharma\diag_flash_full.py"
17:urllib.request.install_opener(urllib.request.build_opener(urllib.request.ProxyHandler({})))

C:\Users\ELOGE KEY\Desktop\pharma>

C:\Users\ELOGE KEY\Desktop\pharma>python -c "import requests; r=requests.get('http://127.0.0.1:8000/', timeout=5); print(r.status_code)" 2>nul || echo "requests pas installé"
200

C:\Users\ELOGE KEY\Desktop\pharma>

DeepSeek

Trouvé — `build_opener` ignore `install_opener`

Les tests le confirment :

- `urllib.urlopen(...)` → **200** ✅ (utilise l'opener global installé ligne 17)
- `requests.get(...)` → **200** ✅
- `curl` → **200** ✅
- Mais `fetch()` du diag → **0** ❌

Cause : `urllib.install_opener(...)` ne remplace que l'opener utilisé par `urlopen()`. Mais dans `fetch()`, on utilise `build_opener(...)` qui crée un **nouvel opener indépendant** → il reprend le ProxyHandler par défaut → cassé.

Fix — Ajouter `ProxyHandler({})` dans les 2 `build_opener`

Éditer le script

```bat
notepad "C:\Users\ELOGE KEY\Desktop\pharma\diag_flash_full.py"
```

Cherche ces lignes (elles apparaissent **2 fois**, une dans `if cookie_jar:`, une dans `else:`) :

```python
    if cookie_jar:
        opener = urllib.request.build_opener(
            urllib.request.HTTPCookieProcessor(cookie_jar),
```

```
            urllib.request.HTTPRedirectHandler() if follow else NoRedirectHandler()
        )
    else:
        opener = urllib.request.build_opener(
            urllib.request.HTTPRedirectHandler() if follow else NoRedirectHandler()
        )
```

Remplace **ces 9 lignes** par :

```python
    if cookie_jar:
        opener = urllib.request.build_opener(
            urllib.request.ProxyHandler({}),
            urllib.request.HTTPCookieProcessor(cookie_jar),
            urllib.request.HTTPRedirectHandler() if follow else NoRedirectHandler()
        )
    else:
        opener = urllib.request.build_opener(
            urllib.request.ProxyHandler({}),
            urllib.request.HTTPRedirectHandler() if follow else NoRedirectHandler()
        )
```

La différence : on ajoute `urllib.request.ProxyHandler({})` en premier argument de chaque `build_opener`.

Ctrl+S.

Relancer

```bat
cd /d "C:\Users\ELOGE KEY\Desktop\pharma"
python diag_flash_full.py
```

Cette fois tout devrait passer.

Test de confirmation (optionnel)

Avant de relancer le diag, tu peux confirmer en 1 ligne :

```bat
python -c "import urllib.request; req=urllib.request.Request('http://127.0.0.1:8000/');
op=urllib.request.build_opener(urllib.request.ProxyHandler({})); print(op.open(req,
timeout=5).status)"
```

Attendu : `200`

Puis sans le ProxyHandler :

```bat
python -c "import urllib.request; req=urllib.request.Request('http://127.0.0.1:8000/'); op=urllib.request.build_opener(); print(op.open(req, timeout=5).status)"
```

Attendu : erreur ou code bizarre (confirme le diagnostic).

Pourquoi ça arrive

Méthode	Handlers utilisés	Proxy
`urllib.urlopen(url)`	Ceux de `install_opener`	✅ Désactivé
urllib.build_opener()	Handlers par défaut	❌ Proxy système
`urllib.build_opener(ProxyHandler({}))`	Handlers par défaut + ProxyHandler vide	✅ Désactivé
requests.get(url)	Handlers requests	✅ Bypass localhost
`curl`	Handlers curl	✅ Bypass localhost

Windows applique le proxy système à `build_opener` par défaut, **même pour `127.0.0.1`** dans certaines configs (proxy d'entreprise, antivirus HTTPS inspector, VPN, etc.).

Colle-moi :

1. La sortie du test court (2 lignes) si tu l'as fait
2. La sortie complète de `python diag_flash_full.py` après le fix

Avec ça, on aura enfin l'analyse complète du flash.

2026-10-08 09:43:00 **You**

regenere le script complet

DeepSeek

Script de diagnostic complet — avec fix proxy

Créer le script

```bat
cd /d "C:\Users\ELOGE KEY\Desktop\pharma"
notepad diag_flash_full.py
```

Colle **tout** ceci, puis Ctrl+S :

```python
"""
diag_flash_full.py
Diagnostic complet du bug "vieille interface avant redirection".
Version avec fix proxy (build_opener + ProxyHandler vide).
"""
import os
# --- FIX PROXY AU PLUS TOT ---
os.environ["NO_PROXY"] = "127.0.0.1,localhost,.trycloudflare.com"
os.environ["no_proxy"] = "127.0.0.1,localhost,.trycloudflare.com"
# -----------------------------

import json
import re
import secrets
import sqlite3
import sys
import time
import urllib.request
import urllib.error
import http.cookiejar
from datetime import datetime, timedelta
from pathlib import Path

# Installer un opener global sans proxy
urllib.request.install_opener(
    urllib.request.build_opener(urllib.request.ProxyHandler({}))
)

# ============================================================
# CONFIG
# ============================================================
BASE = Path(r"C:\Users\ELOGE KEY\Desktop\pharma\pharmacie")
DB = BASE / "database.db"
STATIC = BASE / "static"
LOCAL = "http://127.0.0.1:8000"
TUNNEL = "https://linked-cycles-extras-non.trycloudflare.com"
PRO_EMAIL = "pro@test.tg"
```

```
PASS = 0
FAIL = 0
WARN = 0


def sec(title):
    print()
    print("=" * 74)
    print(" " + title)
    print("=" * 74)
    print()


def check(label, ok, detail=""):
    global PASS, FAIL, WARN
    if ok == "warn":
        WARN += 1
        print(f"  [!] {label}")
    elif ok:
        PASS += 1
        print(f"  [OK] {label}")
    else:
        FAIL += 1
        print(f"  [X] {label}")
    if detail:
        print(f"        {detail}")


class _NoRedirectHandler(urllib.request.HTTPRedirectHandler):
    """Handler qui n'effectue aucun redirect (pour voir les 302)."""
    def redirect_request(self, req, fp, code, msg, headers, newurl):
        return None


def fetch(url, cookie_jar=None, method="GET", follow=True, timeout=15):
    """Retourne (status, headers, body, redirect_chain, elapsed_ms).
    Ignore TOUJOURS les proxies."""
    start = time.time()
    chain = []

    handlers = [urllib.request.ProxyHandler({})]
    if cookie_jar is not None:
        handlers.append(urllib.request.HTTPCookieProcessor(cookie_jar))
    if follow:
        handlers.append(urllib.request.HTTPRedirectHandler())
    else:
        handlers.append(_NoRedirectHandler())
```

```
    opener = urllib.request.build_opener(*handlers)

    req = urllib.request.Request(url, method=method)
    req.add_header("User-Agent", "diag-flash/1.0")

    try:
        resp = opener.open(req, timeout=timeout)
        body = resp.read().decode("utf-8", errors="replace")
        elapsed = int((time.time() - start) * 1000)
        for h in resp.history:
            chain.append((h.status, h.headers.get("Location", "")))
        return resp.status, dict(resp.headers), body, chain, elapsed
    except urllib.error.HTTPError as e:
        elapsed = int((time.time() - start) * 1000)
        body = e.read().decode("utf-8", errors="replace")
        return e.code, dict(e.headers), body, chain, elapsed
    except Exception as e:
        elapsed = int((time.time() - start) * 1000)
        return 0, {}, f"ERREUR: {type(e).__name__}: {e}", chain, elapsed


def has_old_design(html):
    markers = ["#0f9d76", "Votre pharmacie, en ligne", "prototype de demonstration"]
    return any(m in html for m in markers)


def has_new_design(html):
    markers = ["#1a4a7f", "--blue", "acc-welcome", "acc-guide", "gw-card", "pro-navbar"]
    return any(m in html for m in markers)


def has_redirect_script(html):
    patterns = [
        r'window\.location\.(replace|href)\s*=\s*["\']',
        r'location\.replace\s*$',
    ]
    return any(re.search(p, html) for p in patterns)


def extract_redirects(html):
    return re.findall(
        r'window\.location\.(?:replace|href)\s*=\s*["\']([^"\']+)["\']',
        html
    )[:5]


# ============================================================
# 1. UVICORN
# ============================================================
```

```
sec("1. UVICORN (local)")

status, headers, body, _, ms = fetch(LOCAL + "/", timeout=5)
check("Uvicorn repond sur /", status == 200, f"status={status} ({ms}ms)")
if status != 200:
    print()
    print("  >>> Uvicorn n'est pas lance. Arrete ici.")
    print("  >>> Lance : uvicorn main:app --host 0.0.0.0 --port 8000")
    sys.exit(1)

# ============================================================
# 2. FICHIERS DISQUE
# ============================================================
sec("2. FICHIERS DISQUE")

for name in ("index.html", "professional.html", "login.html", "register.html",
             "professional_demande.html", "professional_commandes.html",
             "suivi.html", "payment.html"):
    p = STATIC / name
    if not p.exists():
        check(f"{name:32s} MANQUANT", False)
        continue
    content = p.read_text(encoding="utf-8")
    size = len(content)
    old = has_old_design(content)
    new = has_new_design(content)
    redir = has_redirect_script(content)
    flags = []
    if old: flags.append("VIEUX")
    if new: flags.append("nouveau")
    if redir: flags.append("JS-redirect")
    check(f"{name:32s} {size:>6}o  [{', '.join(flags) or 'neutre'}]", True)

# ============================================================
# 3. HEADERS LOCAL
# ============================================================
sec("3. HEADERS / (LOCAL)")

status, headers, body_local, chain, ms = fetch(LOCAL + "/")
cc = headers.get("cache-control", headers.get("Cache-Control", ""))
csd = headers.get("clear-site-data", headers.get("Clear-Site-Data", ""))
loc = headers.get("location", headers.get("Location", ""))
ctype = headers.get("content-type", headers.get("Content-Type", ""))[:40]

print(f"  status         = {status}  ({ms}ms)")
print(f"  ctype          = {ctype}")
print(f"  cache-control  = {cc}")
print(f"  clear-site-data= {csd}")
print(f"  location       = {loc}")
```

```
print(f"  taille body    = {len(body_local)}")
print(f"  redirect chain = {chain}")
print()

check("cache-control contient no-store", "no-store" in cc.lower())
check("clear-site-data present", "cache" in csd.lower())
check("/ ne fait PAS de 30x en local", not (300 <= status < 400),
      f"status={status}")

# ============================================================
# 4. CONTENU SERVI SUR /
# ============================================================
sec("4. CONTENU SERVI SUR / (LOCAL)")

old_in = has_old_design(body_local)
new_in = has_new_design(body_local)
redir_in = has_redirect_script(body_local)
redirs = extract_redirects(body_local)

check("Nouveau design (bleu) servi", new_in)
check("Ancien design (vert) ABSENT", not old_in,
      "SI FAIL : c'est la cause du flash")
check("Script de redirection JS present", redir_in)
if redirs:
    print(f"  Redirections JS detectees :")
    for r in redirs:
        print(f"    -> {r}")

# ============================================================
# 5. HEADERS / TUNNEL
# ============================================================
sec("5. HEADERS / (TUNNEL)")

status_t, headers_t, body_tunnel, chain_t, ms_t = fetch(TUNNEL + "/")
cc_t = headers_t.get("cache-control", headers_t.get("Cache-Control", ""))
csd_t = headers_t.get("clear-site-data", headers_t.get("Clear-Site-Data", ""))
ctype_t = headers_t.get("content-type", headers_t.get("Content-Type", ""))[:40]

print(f"  status         = {status_t}  ({ms_t}ms)")
print(f"  ctype          = {ctype_t}")
print(f"  cache-control  = {cc_t}")
print(f"  clear-site-data= {csd_t}")
print(f"  taille body    = {len(body_tunnel)}")
print()

check("Tunnel repond", status_t == 200, f"status={status_t} ({ms_t}ms)")
check("Tunnel < 3s", ms_t < 3000, f"{ms_t}ms")
check("Cache-Control no-store via tunnel", "no-store" in cc_t.lower())
check("Clear-Site-Data via tunnel", "cache" in csd_t.lower())
```

```
if body_local == body_tunnel:
    check("Contenu local == tunnel", True)
else:
    check("Contenu local == tunnel", False,
          f"local={len(body_local)}o tunnel={len(body_tunnel)}o")

# ============================================================
# 6. / AVEC COOKIE PRO
# ============================================================
sec("6. ROUTE / AVEC COOKIE PRO")

conn = sqlite3.connect(DB)
conn.row_factory = sqlite3.Row
try:
    user = conn.execute(
        "SELECT id, email FROM users WHERE email=?", (PRO_EMAIL,)
    ).fetchone()
    if not user:
        check(f"{PRO_EMAIL} existe en base", False)
    else:
        check(f"{PRO_EMAIL} existe", True, f"id={user['id']}")

        token = secrets.token_urlsafe(48)
        now = datetime.now()
        exp = now + timedelta(hours=1)
        conn.execute(
            "INSERT INTO sessions (token, user_id, created_at, expires_at)"
            " VALUES (?, ?, ?, ?)",
            (token, user["id"],
             now.isoformat(timespec="seconds"),
             exp.isoformat(timespec="seconds")),
        )
        conn.commit()

        jar = http.cookiejar.CookieJar()
        cookie = http.cookiejar.Cookie(
            version=0, name="elokinab_session", value=token,
            port=None, port_specified=False,
            domain="linked-cycles-extras-non.trycloudflare.com",
            domain_specified=True, domain_initial_dot=False,
            path="/", path_specified=True,
            secure=True, expires=None, discard=False,
            comment=None, comment_url=None, rest={},
        )
        jar.set_cookie(cookie)

        # Sans suivre les redirects
        status_pro, headers_pro, body_pro, chain_pro, ms_pro = fetch(
```

```
            TUNNEL + "/", cookie_jar=jar, follow=False
        )
        loc_pro = headers_pro.get("location", headers_pro.get("Location", ""))

        print()
        print(f"  Cookie PRO envoye. Test / :")
        print(f"    status      = {status_pro}")
        print(f"    location    = {loc_pro}")
        print(f"    taille      = {len(body_pro)}")
        print(f"    elapsed     = {ms_pro}ms")
        print()

        if 300 <= status_pro < 400:
            check("Pro -> redirection 30x cote serveur", True,
                  f"-> {loc_pro}")
            if loc_pro == "/professionnel":
                check("Redirection vers /professionnel", True)
            else:
                check(f"Redirection vers {loc_pro}", False,
                      "Attendu : /professionnel")
        else:
            check("Pro -> redirection 30x cote serveur", False,
                  f"status={status_pro} - PAS de redirection serveur")
            print()
            print("  >>> CAUSE DU FLASH : le serveur renvoie la page /")
            print("      (l'ancien index.html) au lieu de rediriger (302)")
            print("      vers /professionnel. Le navigateur affiche l'ancien")
            print("      contenu pendant que le JS fait la redirection.")
            print()
            print("      FIX : route / doit faire un RedirectResponse(302)")
            print("      pour les utilisateurs PRO/PHARMACIEN/ADMIN connectes.")
finally:
    conn.close()

# ============================================================
# 7. MAGIC LINK FLOW
# ============================================================
sec("7. MAGIC LINK FLOW")

conn = sqlite3.connect(DB)
conn.row_factory = sqlite3.Row
try:
    user = conn.execute(
        "SELECT id FROM users WHERE email=?", (PRO_EMAIL,)
    ).fetchone()
    if user:
        token = secrets.token_urlsafe(32)
        now = datetime.now()
        exp = (now + timedelta(hours=1)).isoformat(timespec="seconds")
```

```
        conn.execute(
            "INSERT INTO magic_links (token, user_id, created_at, expires_at,"
            " max_uses, used_count, revoked, note)"
            " VALUES (?, ?, ?, ?, 0, 0, 0, 'diag_full')",
            (token, user["id"], now.isoformat(timespec="seconds"), exp),
        )
        conn.commit()

        jar = http.cookiejar.CookieJar()
        url_acces = TUNNEL + "/acces/" + token
        status_m, headers_m, body_m, chain_m, ms_m = fetch(
            url_acces, cookie_jar=jar, follow=False
        )
        loc_m = headers_m.get("location", headers_m.get("Location", ""))

        print(f"  /acces/{{token}} :")
        print(f"    status   = {status_m}  ({ms_m}ms)")
        print(f"    location = {loc_m}")
        print(f"    taille   = {len(body_m)}")
        print()

        if 300 <= status_m < 400:
            check("Magic link redirige par 302", "warn",
                  f"-> {loc_m}")
            print("        >>> URL FIXE = risque de cache navigateur")
            print("        >>> FIX : renvoyer HTML + JS avec ?_v=timestamp")
        elif status_m == 200:
            has_js = has_redirect_script(body_m)
            redirs_m = extract_redirects(body_m)
            check("Magic link via HTML+JS", has_js)
            if redirs_m:
                print(f"    Redirection JS : {redirs_m[0][:90]}")
                if "_v=" in redirs_m[0]:
                    check("Cache-bust ?_v= present", True)
                else:
                    check("Cache-bust ?_v= present", False,
                          "Ajoute ?_v=<timestamp> dans magic_links.py")
        else:
            check(f"Magic link status", False, f"status={status_m}")

        cookie_names = [c.name for c in jar]
        check("Cookie session pose par /acces/",
              "elokinab_session" in cookie_names,
              f"cookies: {cookie_names}")
finally:
    conn.close()

# ============================================================
# 8. TIMING TUNNEL
```

```
# ============================================================
sec("8. TIMING TUNNEL (5 requetes sur /)")

times = []
for i in range(5):
    _, _, _, _, ms = fetch(TUNNEL + "/")
    times.append(ms)

print(f"  Temps : {times}")
print(f"  Min : {min(times)}ms / Max : {max(times)}ms / Moy : {sum(times)//len(times)}ms")
print()

if max(times) > 3000:
    check("Tunnel stable (< 3s)", False, f"max={max(times)}ms")
elif max(times) - min(times) > 1000:
    check("Tunnel stable", "warn", f"variance={max(times)-min(times)}ms")
else:
    check("Tunnel stable", True)

# ============================================================
# 9. SERVICE WORKER
# ============================================================
sec("9. SERVICE WORKER")

sw_files = list(STATIC.glob("sw*.js")) + list(STATIC.glob("service-worker*.js"))
if sw_files:
    for f in sw_files:
        check(f"SW sur disque : {f.name}", "warn",
              "Peut causer du cache persistant (meme en navigation privee)")
else:
    check("Aucun Service Worker sur disque", True)

# ============================================================
# 10. DOUBLONS
# ============================================================
sec("10. DOUBLONS")

baks = list(STATIC.glob("*.bak")) + list(STATIC.glob("*.bak*"))
print(f"  {len(baks)} fichiers .bak dans static/")

idxs = sorted(BASE.rglob("index.html"))
print(f"  {len(idxs)} fichiers index.html dans le projet :")
for i in idxs:
    rel = i.relative_to(BASE)
    print(f"    {str(rel):60s}  {i.stat().st_size:>7} o")

# ============================================================
# 11. ROUTES / dans main.py
# ============================================================
```

```
sec("11. ROUTE / DANS main.py")

main_py = BASE / "main.py"
src = main_py.read_text(encoding="utf-8")
m = re.search(
    r'(@app\.(?:get|api_route)\("/"[^)]*$\s*\ndef\s+\w+[^:]*:.*?)(?=\n@app\.|\n\nclass |\n\ndef
|\Z)',
    src, re.DOTALL
)
if m:
    print("  Route / actuelle :")
    print("  " + m.group(1)[:500].replace("\n", "\n  "))
else:
    print("  [!] Route / introuvable dans main.py")

# ============================================================
# RESUME
# ============================================================
sec("RESUME FINAL")

print(f"  PASS  : {PASS}")
print(f"  FAIL  : {FAIL}")
print(f"  WARN  : {WARN}")
print()

if FAIL == 0 and WARN == 0:
    print("  >>> AUCUN probleme detecte.")
else:
    print("  >>> Corrige les [X] et [!] ci-dessus.")
    print()

print("=" * 74)
print(" Colle-moi l'integralite de la sortie.")
print("=" * 74)
```

Lancer

```
bat

cd /d "C:\Users\ELOGE KEY\Desktop\pharma"
python diag_flash_full.py
```

Ce que le script teste

#	Test	Ce qu'on cherche
1	Uvicorn	Répond sur /

#	Test	Ce qu'on cherche
2	Fichiers disque	Vieux vs nouveau design, JS redirection
3	Headers local	cache-control, clear-site-data
4	Contenu servi sur /	**Cause du flash si vieux design**
5	Headers tunnel	Même chose via Cloudflare
6	**/ avec cookie PRO**	**302 serveur ou pas** (cause du flash)
7	Magic link	302 fixe ou HTML+JS avec cache-bust
8	Timing tunnel	Stabilité réseau
9	Service Worker	Cache persistant
10	Doublons	.bak et index.html multiples
11	Route / dans main.py	Code actuel

Les 2 causes les plus probables

Cause A — Section 4 : `[X] Ancien design (vert) ABSENT` → `index.html` sur disque est l'ancienne version.

Cause B — Section 6 : `[X] Pro -> redirection 30x cote serveur` → Le serveur ne fait pas de 302, tout dépend du JS côté client.

Le résumé final dira **exactement** laquelle c'est.

Colle-moi la sortie complète. Cette fois Uvicorn sera bien joint (fix proxy appliqué) et on aura enfin l'analyse définitive.

2026-10-08 09:43:56 **You**

```
================================================================
 1. UVICORN (local)
================================================================

  [X] Uvicorn repond sur /
      status=0 (46ms)
```

```
>>> Uvicorn n'est pas lance. Arrete ici.
>>> Lance : uvicorn main:app --host 0.0.0.0 --port 8000

C:\Users\ELOGE KEY\Desktop\pharma>
```

DeepSeek

Pourquoi `urllib` échoue dans le diag

Le test direct (`urllib.urlopen`) marche → 200. Mais le `fetch()` du diag échoue → 0. Le fix `ProxyHandler({})` n'a rien changé, donc le problème est ailleurs (peut-être un antivirus qui hook `build_opener` , ou un bug spécifique).

Solution radicale : utiliser `http.client` (stdlib pur, aucun handler, aucun proxy possible).

Script complet — version `http.client`

```bat
cd /d "C:\Users\ELOGE KEY\Desktop\pharma"
notepad diag_flash_full.py
```

Efface tout (Ctrl+A puis Suppr) et colle **exactement** ceci :

```python
"""
diag_flash_full.py
Diagnostic complet - utilise http.client (aucun proxy possible).
"""
import http.client
import http.cookiejar
import os
import re
import secrets
import sqlite3
import sys
import time
from datetime import datetime, timedelta
from pathlib import Path
from urllib.parse import urlparse

# Fix proxy pour les appels indirects
os.environ["NO_PROXY"] = "127.0.0.1,localhost,.trycloudflare.com"
os.environ["no_proxy"] = "127.0.0.1,localhost,.trycloudflare.com"
```

```python
# ============================================================
# CONFIG
# ============================================================
BASE = Path(r"C:\Users\ELOGE KEY\Desktop\pharma\pharmacie")
DB = BASE / "database.db"
STATIC = BASE / "static"
LOCAL = "http://127.0.0.1:8000"
TUNNEL = "https://linked-cycles-extras-non.trycloudflare.com"
PRO_EMAIL = "pro@test.tg"

PASS = 0
FAIL = 0
WARN = 0


def sec(title):
    print()
    print("=" * 74)
    print("  " + title)
    print("=" * 74)
    print()


def check(label, ok, detail=""):
    global PASS, FAIL, WARN
    if ok == "warn":
        WARN += 1
        print(f"  [!] {label}")
    elif ok:
        PASS += 1
        print(f"  [OK] {label}")
    else:
        FAIL += 1
        print(f"  [X] {label}")
    if detail:
        print(f"        {detail}")


def fetch(url, cookies=None, method="GET", follow=True, timeout=15):
    """
    Requête HTTP brute via http.client.
    cookies : dict {name: value} ou None.
    Retourne (status, headers_dict, body, redirect_chain, elapsed_ms).
    """
    start = time.time()
    chain = []
    current_url = url
    max_redirects = 10
```

```
    cookie_header = ""
    if cookies:
        cookie_header = "; ".join(f"{k}={v}" for k, v in cookies.items())

    for _ in range(max_redirects + 1):
        try:
            parsed = urlparse(current_url)
        except Exception as e:
            return 0, {}, f"URL invalide: {e}", chain, 0

        host = parsed.hostname
        port = parsed.port or (443 if parsed.scheme == "https" else 80)
        path = parsed.path or "/"
        if parsed.query:
            path += "?" + parsed.query

        headers = {
            "User-Agent": "diag-flash/1.0",
            "Accept": "*/*",
            "Connection": "close",
        }
        if cookie_header:
            headers["Cookie"] = cookie_header

        try:
            if parsed.scheme == "https":
                conn = http.client.HTTPSConnection(host, port, timeout=timeout)
            else:
                conn = http.client.HTTPConnection(host, port, timeout=timeout)

            conn.request(method, path, headers=headers)
            resp = conn.getresponse()
            body = resp.read().decode("utf-8", errors="replace")
            status = resp.status
            hdrs = {}
            for k, v in resp.getheaders():
                hdrs[k] = v
            hdrs_lower = {k.lower(): v for k, v in hdrs.items()}

            # Ajouter les cookies reçus au jar
            set_cookie = hdrs_lower.get("set-cookie")
            if set_cookie and cookies is not None:
                # Parse basique : name=value; ...
                for c in set_cookie.split(","):
                    if "=" in c.split(";")[0]:
                        name, _, val = c.split(";")[0].partition("=")
                        cookies[name.strip()] = val.strip()
```

```
            conn.close()

            if follow and 300 <= status < 400:
                new_loc = hdrs_lower.get("location", "")
                if new_loc:
                    chain.append((status, new_loc))
                    if new_loc.startswith("/"):
                        new_loc = f"{parsed.scheme}://{parsed.netloc}{new_loc}"
                    elif not new_loc.startswith("http"):
                        new_loc = f"{parsed.scheme}://{parsed.netloc}/{new_loc}"
                    current_url = new_loc
                    continue

            elapsed = int((time.time() - start) * 1000)
            return status, hdrs_lower, body, chain, elapsed

        except Exception as e:
            elapsed = int((time.time() - start) * 1000)
            return 0, {}, f"ERREUR: {type(e).__name__}: {e}", chain, elapsed

    return 0, {}, "Trop de redirects", chain, int((time.time() - start) * 1000)


def has_old_design(html):
    return any(m in html for m in
               ["#0f9d76", "Votre pharmacie, en ligne", "prototype de demonstration"])


def has_new_design(html):
    return any(m in html for m in
               ["#1a4a7f", "--blue", "acc-welcome", "acc-guide", "gw-card", "pro-navbar"])


def has_redirect_script(html):
    for p in [r'window\.location\.(replace|href)\s*=\s*["\']',
              r'location\.replace\s*$']:
        if re.search(p, html):
            return True
    return False


def extract_redirects(html):
    return re.findall(
        r'window\.location\.(?:replace|href)\s*=\s*["\']([^"\']+)["\']',
        html)[:5]


# ============================================================
# 1. UVICORN
```

```
# ============================================================
sec("1. UVICORN (local)")

status, headers, body, _, ms = fetch(LOCAL + "/", timeout=5)
check("Uvicorn repond sur /", status == 200, f"status={status} ({ms}ms)")
if status != 200:
    print()
    print("  >>> Uvicorn n'est pas lance ou inaccessible.")
    print(f"  >>> Reponse brute : {body[:200]}")
    sys.exit(1)

# ============================================================
# 2. FICHIERS DISQUE
# ============================================================
sec("2. FICHIERS DISQUE")

for name in ("index.html", "professional.html", "login.html", "register.html",
             "professional_demande.html", "professional_commandes.html",
             "suivi.html", "payment.html"):
    p = STATIC / name
    if not p.exists():
        check(f"{name:32s} MANQUANT", False)
        continue
    content = p.read_text(encoding="utf-8")
    size = len(content)
    old = has_old_design(content)
    new = has_new_design(content)
    redir = has_redirect_script(content)
    flags = []
    if old: flags.append("VIEUX")
    if new: flags.append("nouveau")
    if redir: flags.append("JS-redirect")
    check(f"{name:32s} {size:>6}o  [{', '.join(flags) or 'neutre'}]", True)

# ============================================================
# 3. HEADERS LOCAL
# ============================================================
sec("3. HEADERS / (LOCAL)")

status, headers, body_local, chain, ms = fetch(LOCAL + "/")
cc = headers.get("cache-control", "")
csd = headers.get("clear-site-data", "")
loc = headers.get("location", "")
ctype = headers.get("content-type", "")[:40]

print(f"  status         = {status}  ({ms}ms)")
print(f"  ctype          = {ctype}")
print(f"  cache-control  = {cc}")
print(f"  clear-site-data= {csd}")
```

```
print(f"  location       = {loc}")
print(f"  taille body    = {len(body_local)}")
print(f"  redirect chain = {chain}")
print()

check("cache-control contient no-store", "no-store" in cc.lower())
check("clear-site-data present", "cache" in csd.lower())
check("/ ne fait PAS de 30x en local", not (300 <= status < 400),
      f"status={status}")

# ============================================================
# 4. CONTENU SERVI SUR /
# ============================================================
sec("4. CONTENU SERVI SUR / (LOCAL)")

old_in = has_old_design(body_local)
new_in = has_new_design(body_local)
redir_in = has_redirect_script(body_local)
redirs = extract_redirects(body_local)

check("Nouveau design (bleu) servi", new_in)
check("Ancien design (vert) ABSENT", not old_in,
      "SI FAIL : c'est la cause du flash")
check("Script de redirection JS present", redir_in)
if redirs:
    print(f"  Redirections JS detectees :")
    for r in redirs:
        print(f"    -> {r}")

if old_in and not new_in:
    print()
    print("  >>> CAUSE DU FLASH : / sert l'ANCIEN index.html.")
    print("      Le navigateur affiche l'ancien contenu pendant que le JS")
    print("      de redirection fait son travail (300-800 ms).")

# ============================================================
# 5. HEADERS / TUNNEL
# ============================================================
sec("5. HEADERS / (TUNNEL)")

status_t, headers_t, body_tunnel, chain_t, ms_t = fetch(TUNNEL + "/")
cc_t = headers_t.get("cache-control", "")
csd_t = headers_t.get("clear-site-data", "")
ctype_t = headers_t.get("content-type", "")[:40]

print(f"  status         = {status_t}  ({ms_t}ms)")
print(f"  ctype          = {ctype_t}")
print(f"  cache-control  = {cc_t}")
print(f"  clear-site-data= {csd_t}")
```

```
print(f"  taille body    = {len(body_tunnel)}")
print()

check("Tunnel repond", status_t == 200, f"status={status_t} ({ms_t}ms)")
check("Tunnel < 3s", ms_t < 3000, f"{ms_t}ms")
check("Cache-Control no-store via tunnel", "no-store" in cc_t.lower())
check("Clear-Site-Data via tunnel", "cache" in csd_t.lower())

if body_local == body_tunnel:
    check("Contenu local == tunnel", True)
else:
    check("Contenu local == tunnel", False,
          f"local={len(body_local)}o tunnel={len(body_tunnel)}o")

# ============================================================
# 6. / AVEC COOKIE PRO
# ============================================================
sec("6. ROUTE / AVEC COOKIE PRO")

conn = sqlite3.connect(DB)
conn.row_factory = sqlite3.Row
try:
    user = conn.execute(
        "SELECT id, email FROM users WHERE email=?", (PRO_EMAIL,)
    ).fetchone()
    if not user:
        check(f"{PRO_EMAIL} existe en base", False)
    else:
        check(f"{PRO_EMAIL} existe", True, f"id={user['id']}")

        token = secrets.token_urlsafe(48)
        now = datetime.now()
        exp = now + timedelta(hours=1)
        conn.execute(
            "INSERT INTO sessions (token, user_id, created_at, expires_at)"
            " VALUES (?, ?, ?, ?)",
            (token, user["id"],
             now.isoformat(timespec="seconds"),
             exp.isoformat(timespec="seconds")),
        )
        conn.commit()

        cookies = {"elokinab_session": token}

        # Sans suivre les redirects
        status_pro, headers_pro, body_pro, chain_pro, ms_pro = fetch(
            TUNNEL + "/", cookies=cookies, follow=False
        )
        loc_pro = headers_pro.get("location", "")
```

```
        print()
        print(f"  Cookie PRO envoye. Test / :")
        print(f"    status      = {status_pro}")
        print(f"    location    = {loc_pro}")
        print(f"    taille      = {len(body_pro)}")
        print(f"    elapsed     = {ms_pro}ms")
        print()

        if 300 <= status_pro < 400:
            check("Pro -> redirection 30x cote serveur", True,
                  f"-> {loc_pro}")
            if loc_pro == "/professionnel":
                check("Redirection vers /professionnel", True)
            else:
                check(f"Redirection vers {loc_pro}", False,
                      "Attendu : /professionnel")
        else:
            check("Pro -> redirection 30x cote serveur", False,
                  f"status={status_pro} - PAS de redirection serveur")
            print()
            print("  >>> CAUSE DU FLASH : le serveur renvoie / (ancien")
            print("      index.html) au lieu de rediriger (302). Le navigateur")
            print("      affiche l'ancien contenu avant que le JS redirige.")
            print()
            print("      FIX : route / doit faire RedirectResponse(302)")
            print("      pour les utilisateurs PRO connectes.")
finally:
    conn.close()

# ============================================================
# 7. MAGIC LINK
# ============================================================
sec("7. MAGIC LINK FLOW")

conn = sqlite3.connect(DB)
conn.row_factory = sqlite3.Row
try:
    user = conn.execute(
        "SELECT id FROM users WHERE email=?", (PRO_EMAIL,)
    ).fetchone()
    if user:
        token = secrets.token_urlsafe(32)
        now = datetime.now()
        exp = (now + timedelta(hours=1)).isoformat(timespec="seconds")
        conn.execute(
            "INSERT INTO magic_links (token, user_id, created_at, expires_at,"
            " max_uses, used_count, revoked, note)"
            " VALUES (?, ?, ?, ?, 0, 0, 0, 'diag_full')",
```

```
            (token, user["id"], now.isoformat(timespec="seconds"), exp),
        )
        conn.commit()

        cookies = {}
        url_acces = TUNNEL + "/acces/" + token
        status_m, headers_m, body_m, chain_m, ms_m = fetch(
            url_acces, cookies=cookies, follow=False
        )
        loc_m = headers_m.get("location", "")

        print(f"  /acces/{{token}} :")
        print(f"    status   = {status_m}  ({ms_m}ms)")
        print(f"    location = {loc_m}")
        print(f"    taille   = {len(body_m)}")
        print()

        if 300 <= status_m < 400:
            check("Magic link redirige par 302", "warn",
                  f"-> {loc_m}")
            print("        >>> URL FIXE = risque de cache navigateur.")
            print("        >>> FIX : renvoyer HTML + JS avec ?_v=timestamp")
        elif status_m == 200:
            has_js = has_redirect_script(body_m)
            redirs_m = extract_redirects(body_m)
            check("Magic link via HTML+JS", has_js)
            if redirs_m:
                print(f"    Redirection JS : {redirs_m[0][:90]}")
                if "_v=" in redirs_m[0]:
                    check("Cache-bust ?_v= present", True)
                else:
                    check("Cache-bust ?_v= present", False,
                          "Ajoute ?_v=<timestamp> dans magic_links.py")
        else:
            check(f"Magic link status", False, f"status={status_m}")

        check("Cookie session pose par /acces/",
              "elokinab_session" in cookies,
              f"cookies: {list(cookies.keys())}")
finally:
    conn.close()

# ============================================================
# 8. TIMING TUNNEL
# ============================================================
sec("8. TIMING TUNNEL (5 requetes sur /)")

times = []
for i in range(5):
```

```
    _, _, _, _, ms = fetch(TUNNEL + "/")
    times.append(ms)

print(f"  Temps : {times}")
print(f"  Min : {min(times)}ms / Max : {max(times)}ms / Moy : {sum(times)//len(times)}ms")
print()

if max(times) > 3000:
    check("Tunnel stable (< 3s)", False, f"max={max(times)}ms")
elif max(times) - min(times) > 1000:
    check("Tunnel stable", "warn", f"variance={max(times)-min(times)}ms")
else:
    check("Tunnel stable", True)

# ============================================================
# 9. SERVICE WORKER
# ============================================================
sec("9. SERVICE WORKER")

sw_files = list(STATIC.glob("sw*.js")) + list(STATIC.glob("service-worker*.js"))
if sw_files:
    for f in sw_files:
        check(f"SW sur disque : {f.name}", "warn",
              "Peut causer du cache persistant")
else:
    check("Aucun Service Worker sur disque", True)

# ============================================================
# 10. DOUBLONS
# ============================================================
sec("10. DOUBLONS")

baks = list(STATIC.glob("*.bak")) + list(STATIC.glob("*.bak*"))
print(f"  {len(baks)} fichiers .bak dans static/")

idxs = sorted(BASE.rglob("index.html"))
print(f"  {len(idxs)} fichiers index.html dans le projet :")
for i in idxs:
    rel = i.relative_to(BASE)
    print(f"    {str(rel):60s}  {i.stat().st_size:>7} o")

# ============================================================
# 11. ROUTE / DANS main.py
# ============================================================
sec("11. ROUTE / DANS main.py")

main_py = BASE / "main.py"
src = main_py.read_text(encoding="utf-8")
m = re.search(
```

```
    r'(@app\.(?:get|api_route)\("/"[^)]*$\s*\ndef\s+\w+[^:]*:.*?)(?=\n@app\.|\n\nclass |\n\ndef
|\Z)',
    src, re.DOTALL
)
if m:
    print("  Route / actuelle :")
    print("  " + m.group(1)[:600].replace("\n", "\n  "))
else:
    print("  [!] Route / introuvable dans main.py")

# ============================================================
# RESUME
# ============================================================
sec("RESUME FINAL")

print(f"  PASS  : {PASS}")
print(f"  FAIL  : {FAIL}")
print(f"  WARN  : {WARN}")
print()

if FAIL == 0 and WARN == 0:
    print("  >>> AUCUN probleme detecte.")
else:
    print("  >>> Corrige les [X] et [!] ci-dessus.")
    print()

print("=" * 74)
print(" Colle-moi l'integralite de la sortie.")
print("=" * 74)
```

Lancer

```
bat

cd /d "C:\Users\ELOGE KEY\Desktop\pharma"
python diag_flash_full.py
```

Ce qui change

Avant	Après
`urllib.request.build_opener` (casse)	`http.client.HTTPConnection` (bas niveau)
Handlers, proxies, hacks	Aucun handler, connexion directe
Échouait → status 0	Devrait marcher → 200

`http.client` ouvre une **socket TCP brute** vers `127.0.0.1:8000` — impossible qu'un proxy ou handler l'intercepte.

Colle-moi la sortie complète. Cette fois, on aura enfin l'analyse définitive.

2026-10-08 09:45:21 **You**

Pasted text.txt
6.1 KB

DeepSeek

Diagnostic complet — 2 causes identifiées

Analyse des résultats

Ce qui marche ✅

- Uvicorn : 200
- Tunnel : 200 (617ms moyen, stable)
- Headers no-store + clear-site-data partout
- **Route `/` avec cookie PRO → 302 vers `/professionnel`** ✅
- Contenu local = tunnel (aucune altération)

Ce qui pose problème ❌

Cause 1 — Magic link renvoie un 302 vers URL fixe (section 7)

```
/acces/{token} → status=302 → location=/professionnel
```

Le 302 pointe vers `/professionnel` **sans paramètre**. Le navigateur a cette URL en cache disque (d'anciens patchs). Il affiche la vieille version pendant que la nouvelle se charge → **c'est le flash**.

Cause 2 — index.html sur disque est l'ancien design (section 4)

```
[X] Ancien design (vert) ABSENT : False
[X] Nouveau design (bleu) servi : False
```

`static/index.html` (15987o) est l'ancienne page patient. Mais ce n'est **pas grave** car `/` redirige les pros vers `/professionnel` (section 6 le confirme).

Le flash vient donc **du magic link qui pointe vers une URL fixe** `/professionnel`, dont le navigateur a une vieille copie.

Fix — Magic link avec cache-bust

Un seul script qui règle le problème définitivement.

Créer le script

```bat
cd /d "C:\Users\ELOGE KEY\Desktop\pharma\pharmacie"
notepad fix_magic_link_v2.py
```

Colle exactement ceci, puis Ctrl+S :

```python
"""
fix_magic_link_v2.py
Remplace le 302 du magic link par un HTML minimal + JS avec cache-bust.
-> URL unique a chaque clic = aucun cache possible.
"""
from pathlib import Path
import datetime
import re
import shutil

BASE = Path(__file__).resolve().parent
MAGIC = BASE / "v3_modules" / "magic_links.py"

stamp = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
backup = BASE / ("_backup_magic_v2_" + stamp)
backup.mkdir(parents=True, exist_ok=True)
shutil.copy2(MAGIC, backup / "magic_links.py")
print("[OK] Backup ->", backup.name)

src = MAGIC.read_text(encoding="utf-8")

# 1. Verifier si deja patche
if "window.location.replace" in src and "?_v=" in src:
    print("[i] magic_links.py deja patche")
    raise SystemExit(0)

# 2. Trouver la fonction access_magic_link
pattern = re.compile(
    r'(@app\.(?:get|api_route)$"/acces/\{token\}"[^\n]*\n'
    r'def access_magic_link\(token: str$:.*?)(?=\n@app\.|\n\nclass |\n\ndef |\Z)',
    re.DOTALL
```

```
)

m = pattern.search(src)
if not m:
    print("[!] access_magic_link introuvable dans magic_links.py")
    raise SystemExit(1)

print("[i] Fonction access_magic_link trouvee")

new_fn = '''@app.get("/acces/{token}", include_in_schema=False)
def access_magic_link(token: str):
    import time
    from fastapi.responses import HTMLResponse
    conn = get_db()
    try:
        ml = conn.execute(
            "SELECT * FROM magic_links WHERE token=?", (token,)
        ).fetchone()
        if ml is None:
            raise HTTPException(404, "Lien invalide")
        if ml["revoked"]:
            raise HTTPException(403, "Lien revoque")
        if ml["expires_at"]:
            try:
                exp = datetime.fromisoformat(ml["expires_at"])
                if datetime.now() > exp:
                    raise HTTPException(403, "Lien expire")
            except ValueError:
                pass
        if ml["max_uses"] and ml["used_count"] >= ml["max_uses"]:
            raise HTTPException(403, "Lien deja utilise")
        user = conn.execute(
            "SELECT * FROM users WHERE id=? AND is_active=1", (ml["user_id"],)
        ).fetchone()
        if user is None:
            raise HTTPException(403, "Utilisateur inactif")

        # Incrementer usage
        conn.execute(
            "UPDATE magic_links SET used_count = used_count + 1 WHERE id=?",
            (ml["id"],)
        )

        # Creer une session
        session_token = secrets.token_urlsafe(48)
        now = datetime.now()
        exp = now + timedelta(days=SESSION_TTL_DAYS)
        conn.execute(
            "INSERT INTO sessions (token, user_id, created_at, expires_at)"
```

```
        " VALUES (?, ?, ?, ?)",
        (session_token, user["id"],
         now.isoformat(timespec="seconds"),
         exp.isoformat(timespec="seconds"))
    )
    conn.commit()

    # Determiner la cible
    role = user["role"]
    if role == "PROFESSIONNEL":
        target = "/professionnel"
    elif role in ("PHARMACIEN", "ADMIN"):
        target = "/pharmacien"
    else:
        target = "/"

    # Cache-bust : timestamp unique
    cb = str(int(time.time() * 1000))
    final_url = target + "?_v=" + cb

    # HTML minimal qui force le rechargement avec cache-bust
    html = (
        '<!DOCTYPE html>'
        '<html lang="fr"><head><meta charset="utf-8">'
        '<title>Redirection...</title>'
        '<meta name="viewport" content="width=device-width,initial-scale=1">'
        '<style>'
        'body{font-family:-apple-system,"Segoe UI",system-ui,sans-serif;'
        'display:flex;align-items:center;justify-content:center;'
        'height:100vh;margin:0;background:#f7fafc;color:#64748b}'
        '.sp{width:40px;height:40px;border:3px solid #e6ebf1;'
        'border-top-color:#1a4a7f;border-radius:50%;'
        'animation:s 1s linear infinite}'
        '@keyframes s{to{transform:rotate(360deg)}}'
        '</style></head><body><div class="sp"></div>'
        '<script>'
        'window.location.replace("' + final_url + '");'
        '</script>'
        '</body></html>'
    )

    resp = HTMLResponse(content=html)
    resp.set_cookie(
        SESSION_COOKIE, session_token,
        httponly=True, samesite="none", secure=True,
        max_age=SESSION_TTL_DAYS * 86400, path="/"
    )
    resp.headers["Cache-Control"] = "no-store, no-cache, must-revalidate, max-age=0"
    resp.headers["Pragma"] = "no-cache"
```

```
        resp.headers["Expires"] = "0"
        return resp
    finally:
        conn.close()
'''

src = pattern.sub(new_fn + "\n", src, count=1)
MAGIC.write_text(src, encoding="utf-8")
print("[OK] magic_links.py -> access_magic_link remplace (HTML+JS+cache-bust)")

# Verification
check = MAGIC.read_text(encoding="utf-8")
print()
print("Verification :")
print("  window.location.replace :", "window.location.replace" in check)
print("  ?_v= cache-bust :", "?_v=" in check)
print("  HTMLResponse :", "HTMLResponse" in check)
print("  Cache-Control no-store :", "no-store" in check)

print()
print("=" * 60)
print("FIX MAGIC LINK V2 APPLIQUE")
print("=" * 60)
print()
print("Comportement :")
print("  /acces/{token}")
print("  -> HTML minimal (spinner)")
print("  -> JS: window.location.replace('/professionnel?_v=<ms>')")
print("  -> URL unique = aucun cache possible")
print()
print("PROCHAINE ETAPE :")
print("  taskkill /F /IM python.exe")
print("  uvicorn main:app --host 0.0.0.0 --port 8000")
print("  Puis : python generate_link.py pro@test.tg --days 30")
```

Lancer

```bat
python fix_magic_link_v2.py
taskkill /F /IM python.exe
cd /d "C:\Users\ELOGE KEY\Desktop\pharma\pharmacie"
call venv\Scripts\activate
uvicorn main:app --host 0.0.0.0 --port 8000
```

Attends :

```
INFO: Application startup complete.
```

Générer un nouveau lien

```bat
python generate_link.py pro@test.tg --days 30
```

Tester

1. Ouvre le lien `/acces/...` sur iPhone
2. Tu dois voir un **spinner bleu** pendant ~0.2s
3. Puis la nouvelle interface pro sans flash

Pourquoi ça marche

Avant	Après
302 → `/professionnel` (URL fixe)	HTML → `window.location.replace("/professionnel?_v=1744301250000")`
Navigateur ressort `/professionnel` du cache disque	URL unique à chaque clic = cache ignoré

Le `?_v=<timestamp>` change à chaque génération de lien → aucune chance de collision avec le cache navigateur.

Optionnel — Refondre index.html

`static/index.html` reste l'ancien design vert. Ce n'est pas grave pour les pros (redirigés), mais si tu veux que la page patient soit aussi en V6 :

Dis-le et je te livre un script de refonte complète (hero bleu, cartes, footer).

Test après le fix :

1. Ouvre le magic link sur iPhone (navigation privée pour être sûr)
2. Tu dois voir un spinner bleu puis la nouvelle interface
3. **Plus de flash du vieux design**

Colle-moi :

1. La sortie de `python fix_magic_link_v2.py`
2. Le nouveau lien généré
3. Ce que tu vois sur iPhone (flash ou pas)

2026-10-08 09:47:05 **You**

C:\Users\ELOGE KEY\Desktop\pharma\pharmacie>python fix_magic_link_v2.py
[OK] Backup -> _backup_magic_v2_20261008_094626
[!] access_magic_link introuvable dans magic_links.py

C:\Users\ELOGE KEY\Desktop\pharma\pharmacie>
on efface meme le magic link sans rien casser

DeepSeek

Réécrire complètement `magic_links.py` (propre)

Le regex a échoué car le fichier a une structure différente. On va **réécrire tout** `magic_links.py` — propre, complet, qui marche.

Script

```bat
cd /d "C:\Users\ELOGE KEY\Desktop\pharma\pharmacie"
notepad fix_magic_link_v3.py
```

Colle exactement ceci, puis Ctrl+S :

```python
"""
fix_magic_link_v3.py
Reecrit entierement v3_modules/magic_links.py (propre, avec cache-bust).
"""
from pathlib import Path
import datetime
import shutil

BASE = Path(__file__).resolve().parent
MAGIC = BASE / "v3_modules" / "magic_links.py"

stamp = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
backup = BASE / ("_backup_magic_v3_" + stamp)
backup.mkdir(parents=True, exist_ok=True)
if MAGIC.exists():
```

```
    shutil.copy2(MAGIC, backup / "magic_links.py")
    print("[OK] Backup ->", backup.name)

NEW_CONTENT = '''"""Magic link : connexion automatique via URL (avec cache-bust)."""
import secrets
import time
from datetime import datetime, timedelta
from typing import Optional
from fastapi import Cookie, Depends, HTTPException, Request
from fastapi.responses import HTMLResponse, RedirectResponse
from pydantic import BaseModel


SESSION_COOKIE = "elokinab_session"
SESSION_TTL_DAYS = 365


class MagicCreate(BaseModel):
    user_id: int
    expires_in_days: Optional[int] = 30
    max_uses: int = 0
    note: str = ""


def install_magic_links(app, get_db, require_role):
    def _row(r):
        return {k: r[k] for k in r.keys()} if r else None

    # ------------------------------------------------------------
    # Creer un magic link (ADMIN ou PHARMACIEN)
    # ------------------------------------------------------------
    @app.post("/api/admin/magic-link")
    def create_magic_link(payload: MagicCreate,
                          user=Depends(require_role("ADMIN", "PHARMACIEN"))):
        conn = get_db()
        try:
            target = conn.execute(
                "SELECT id, email, role FROM users WHERE id=? AND is_active=1",
                (payload.user_id,)
            ).fetchone()
            if target is None:
                raise HTTPException(404, "Utilisateur introuvable ou inactif")
            token = secrets.token_urlsafe(32)
            now = datetime.now()
            expires = None
            if payload.expires_in_days and payload.expires_in_days > 0:
                expires = (now +
timedelta(days=payload.expires_in_days)).isoformat(timespec="seconds")
            cur = conn.execute(
```

```
            "INSERT INTO magic_links (token, user_id, created_at, expires_at,"
            " max_uses, used_count, revoked, note)"
            " VALUES (?, ?, ?, ?, ?, 0, 0, ?)",
            (token, payload.user_id, now.isoformat(timespec="seconds"),
             expires, payload.max_uses, payload.note)
        )
        conn.commit()
        link_id = cur.lastrowid
        row = conn.execute(
            "SELECT * FROM magic_links WHERE id=?", (link_id,)
        ).fetchone()
    finally:
        conn.close()
    return {
        **_row(row),
        "access_url": "/acces/" + token,
        "target_email": target["email"],
        "target_role": target["role"]
    }

# ------------------------------------------------------------
# Lister les magic links
# ------------------------------------------------------------
@app.get("/api/admin/magic-links")
def list_magic_links(user=Depends(require_role("ADMIN", "PHARMACIEN"))):
    conn = get_db()
    try:
        rows = conn.execute(
            "SELECT ml.*, u.email AS user_email, u.role AS user_role,"
            " u.full_name AS user_name"
            " FROM magic_links ml JOIN users u ON u.id = ml.user_id"
            " ORDER BY ml.id DESC LIMIT 100"
        ).fetchall()
    finally:
        conn.close()
    return [_row(r) for r in rows]

# ------------------------------------------------------------
# Revoquer
# ------------------------------------------------------------
@app.delete("/api/admin/magic-link/{link_id}")
def revoke_magic_link(link_id: int,
                      user=Depends(require_role("ADMIN", "PHARMACIEN"))):
    conn = get_db()
    try:
        conn.execute("UPDATE magic_links SET revoked=1 WHERE id=?", (link_id,))
        conn.commit()
    finally:
        conn.close()
```

```
    return {"ok": True}

# ------------------------------------------------------------
# /acces/{token} : HTML + JS + cache-bust
# ------------------------------------------------------------
@app.get("/acces/{token}", include_in_schema=False)
def access_magic_link(token: str):
    conn = get_db()
    try:
        ml = conn.execute(
            "SELECT * FROM magic_links WHERE token=?", (token,)
        ).fetchone()
        if ml is None:
            raise HTTPException(404, "Lien invalide")
        if ml["revoked"]:
            raise HTTPException(403, "Lien revoque")
        if ml["expires_at"]:
            try:
                exp = datetime.fromisoformat(ml["expires_at"])
                if datetime.now() > exp:
                    raise HTTPException(403, "Lien expire")
            except ValueError:
                pass
        if ml["max_uses"] and ml["used_count"] >= ml["max_uses"]:
            raise HTTPException(403, "Lien deja utilise")
        user = conn.execute(
            "SELECT * FROM users WHERE id=? AND is_active=1",
            (ml["user_id"],)
        ).fetchone()
        if user is None:
            raise HTTPException(403, "Utilisateur inactif")

        # Incrementer usage
        conn.execute(
            "UPDATE magic_links SET used_count = used_count + 1 WHERE id=?",
            (ml["id"],)
        )

        # Creer session
        session_token = secrets.token_urlsafe(48)
        now = datetime.now()
        exp = now + timedelta(days=SESSION_TTL_DAYS)
        conn.execute(
            "INSERT INTO sessions (token, user_id, created_at, expires_at)"
            " VALUES (?, ?, ?, ?)",
            (session_token, user["id"],
             now.isoformat(timespec="seconds"),
             exp.isoformat(timespec="seconds"))
        )
```

```
            conn.commit()

            # Determiner la cible
            role = user["role"]
            if role == "PROFESSIONNEL":
                target = "/professionnel"
            elif role in ("PHARMACIEN", "ADMIN"):
                target = "/pharmacien"
            else:
                target = "/"

            # Cache-bust avec timestamp
            cb = str(int(time.time() * 1000))
            final_url = target + "?_v=" + cb

            html = (
                '<!DOCTYPE html>'
                '<html lang="fr"><head><meta charset="utf-8">'
                '<title>Redirection...</title>'
                '<meta name="viewport" content="width=device-width,initial-scale=1">'
                '<style>'
                'body{font-family:-apple-system,"Segoe UI",system-ui,sans-serif;'
                'display:flex;align-items:center;justify-content:center;'
                'height:100vh;margin:0;background:#f7fafc;color:#64748b}'
                '.sp{width:40px;height:40px;border:3px solid #e6ebf1;'
                'border-top-color:#1a4a7f;border-radius:50%;'
                'animation:s 1s linear infinite}'
                '@keyframes s{to{transform:rotate(360deg)}}'
                '</style></head><body><div class="sp"></div>'
                '<script>'
                'window.location.replace("' + final_url + '");'
                '</script>'
                '</body></html>'
            )

            resp = HTMLResponse(content=html)
            resp.set_cookie(
                SESSION_COOKIE, session_token,
                httponly=True, samesite="none", secure=True,
                max_age=SESSION_TTL_DAYS * 86400, path="/"
            )
            resp.headers["Cache-Control"] = "no-store, no-cache, must-revalidate, max-age=0"
            resp.headers["Pragma"] = "no-cache"
            resp.headers["Expires"] = "0"
            return resp
        finally:
            conn.close()
'''
```

```
MAGIC.write_text(NEW_CONTENT, encoding="utf-8")
print("[OK] magic_links.py reecrit completement")

# Verification
check = MAGIC.read_text(encoding="utf-8")
print()
print("Verification :")
print("  install_magic_links :", "def install_magic_links" in check)
print("  /acces/{token}      :", '"/acces/{token}"' in check)
print("  window.location.replace :", "window.location.replace" in check)
print("  ?_v= cache-bust     :", "?_v=" in check)
print("  HTMLResponse        :", "HTMLResponse" in check)
print("  Cache-Control no-store :", "no-store" in check)

print()
print("=" * 60)
print("MAGIC_LINKS.PY REEcrit")
print("=" * 60)
print()
print("PROCHAINE ETAPE :")
print("  taskkill /F /IM python.exe")
print("  uvicorn main:app --host 0.0.0.0 --port 8000")
print("  Puis : python generate_link.py pro@test.tg --days 30")
```

Lancer

```bat
python fix_magic_link_v3.py
taskkill /F /IM python.exe
cd /d "C:\Users\ELOGE KEY\Desktop\pharma\pharmacie"
call venv\Scripts\activate
uvicorn main:app --host 0.0.0.0 --port 8000
```

Attends :

`INFO: Application startup complete.`

Générer un nouveau lien

```bat
python generate_link.py pro@test.tg --days 30
```

Tester sur iPhone

1. Ouvre le nouveau lien `/acces/...`

2. Tu dois voir un **spinner bleu** pendant ~0.2s
3. Puis la nouvelle interface pro — **sans flash**

Pourquoi ça marche maintenant

Le nouveau `access_magic_link` fait :

1. Crée la session (comme avant)
2. **Au lieu de renvoyer un 302 →** `/professionnel`, renvoie un mini HTML :

```html
<div class="spinner"></div>
<script>
window.location.replace("/professionnel?_v=1744301250000")
</script>
```

3. Le `?_v=<timestamp>` change à chaque clic → **URL unique, cache impossible**

Le navigateur voit une URL qu'il n'a jamais visitée → il télécharge toujours la version fraîche.

Récapitulatif

Avant	Après
`302 → /professionnel` (URL fixe)	HTML + JS → `/professionnel?_v=<timestamp>`
Navigateur sert `/professionnel` du cache	URL unique → cache ignoré
Flash de 300-800ms	Spinner bleu puis nouvelle version

Colle-moi :

1. La sortie de `python fix_magic_link_v3.py`
2. Le nouveau lien généré
3. Ce que tu vois sur iPhone : flash ou pas

2026-10-08 09:51:43 **You**

je veux qu'on exclut magic link

DeepSeek

Exclure le magic link — 2 interprétations

A) Tu veux tester SANS le magic link (juste par connexion normale)

C'est le plus probable. On teste le flash avec la **connexion par mot de passe** :

Étapes

1. **Ouvre** `/connexion` : https://linked-cycles-extras-non.trycloudflare.com/connexion
2. Connecte-toi : `pro@test.tg` / `Test1234`
3. Tu arrives sur l'app
4. **Ferme Safari complètement** (glisse vers le haut depuis le bas, swipe la carte Safari)
5. **Rouvre** Safari et va sur : https://linked-cycles-extras-non.trycloudflare.com/
6. **Observe** :
   - ✅ Pas de flash → c'est bien le magic link qui causait le problème → on le fix
   - ❌ Flash persiste → c'est autre chose (route `/` ou cache Safari)

Colle-moi le résultat de ce test.

B) Tu veux désactiver complètement le magic link

Si tu ne veux plus du tout utiliser le magic link (connexion uniquement par mot de passe), voici le script :

Script

```bat
cd /d "C:\Users\ELOGE KEY\Desktop\pharma\pharmacie"
notepad disable_magic_link.py
```

Colle exactement ceci, puis Ctrl+S :

```python
"""
disable_magic_link.py
Desactive completement le systeme de magic link.
- Retire les routes /api/admin/magic-link*, /acces/{token}
- Supprime le module magic_links.py
- Le generate_link.py ne fonctionnera plus
"""
from pathlib import Path
import datetime
import re
```

```
import shutil

BASE = Path(__file__).resolve().parent
MAIN = BASE / "main.py"
MAGIC = BASE / "v3_modules" / "magic_links.py"
GENLINK = BASE / "generate_link.py"

stamp = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
backup = BASE / ("_backup_disable_magic_" + stamp)
backup.mkdir(parents=True, exist_ok=True)
shutil.copy2(MAIN, backup / "main.py")
if MAGIC.exists():
    shutil.copy2(MAGIC, backup / "magic_links.py")
if GENLINK.exists():
    shutil.copy2(GENLINK, backup / "generate_link.py")
print("[OK] Backup ->", backup.name)

# 1. Retirer l'appel install_magic_links dans main.py
src = MAIN.read_text(encoding="utf-8")

patterns = [
    r'\nfrom v3_modules\.magic_links import install_magic_links\ninstall_magic_links$app,
get_db, require_role$',
    r'\nfrom v3_modules\.magic_links import install_magic_links\s*\n\s*install_magic_links$app,
get_db, require_role$\s*\n',
    r'\ninstall_magic_links$app, get_db, require_role$\s*\n',
    r'\nfrom v3_modules\.magic_links import install_magic_links\s*\n',
]

removed = 0
for p in patterns:
    new_src, n = re.subn(p, "\n", src)
    if n > 0:
        src = new_src
        removed += n

if removed > 0:
    MAIN.write_text(src, encoding="utf-8")
    print(f"[OK] main.py -> {removed} reference(s) magic_link retiree(s)")
else:
    print("[i] Aucune reference magic_link dans main.py")

# 2. Renommer magic_links.py pour le neutraliser
if MAGIC.exists():
    disabled = MAGIC.with_suffix(".py.disabled")
    if disabled.exists():
        disabled.unlink()
    MAGIC.rename(disabled)
    print(f"[OK] magic_links.py -> renomme en {disabled.name}")
```

```
# 3. Renommer generate_link.py
if GENLINK.exists():
    disabled_link = GENLINK.with_suffix(".py.disabled")
    if disabled_link.exists():
        disabled_link.unlink()
    GENLINK.rename(disabled_link)
    print(f"[OK] generate_link.py -> renomme en {disabled_link.name}")

# 4. Renommer restart_and_link.py si présent
RAL = BASE / "restart_and_link.py"
if RAL.exists():
    disabled_ral = RAL.with_suffix(".py.disabled")
    if disabled_ral.exists():
        disabled_ral.unlink()
    RAL.rename(disabled_ral)
    print(f"[OK] restart_and_link.py -> renomme en {disabled_ral.name}")

# Verification
check = MAIN.read_text(encoding="utf-8")
print()
print("Verification :")
print("  'magic_links' dans main.py :", "magic_links" in check)
print("  '/acces/' dans main.py     :", "/acces/" in check)
print("  magic_links.py existe      :", MAGIC.exists())
print("  generate_link.py existe    :", GENLINK.exists())

print()
print("=" * 60)
print("MAGIC LINK DESACTIVE")
print("=" * 60)
print()
print("Les utilisateurs doivent maintenant se connecter avec")
print("leur email + mot de passe sur /connexion.")
print()
print("Pour reactiver plus tard :")
print("  - Renommer v3_modules/magic_links.py.disabled en .py")
print("  - Remettre l'appel dans main.py depuis le backup")
print()
print("PROCHAINE ETAPE :")
print("  taskkill /F /IM python.exe")
print("  uvicorn main:app --host 0.0.0.0 --port 8000")
```

Lancer

```
bat
```

```
python disable_magic_link.py
taskkill /F /IM python.exe
cd /d "C:\Users\ELOGE KEY\Desktop\pharma\pharmacie"
call venv\Scripts\activate
uvicorn main:app --host 0.0.0.0 --port 8000
```

Après

- Plus de `/acces/{token}`
- Plus de `generate_link.py`
- Les utilisateurs se connectent **uniquement** via `/connexion`

Ma recommandation

Fais d'abord le test A (connexion normale) avant de désactiver quoi que ce soit. Ça te dira :

- Si le flash disparaît → c'est bien le magic link → **option B** est la solution
- Si le flash persiste → c'est autre chose → on cherche plus loin

Dis-moi :

1. Tu as testé avec la connexion normale (mot de passe) ? Le flash persiste ?
2. Tu veux l'option A (juste tester) ou l'option B (désactiver complètement) ?

2026-10-08 09:58:38 **You**

c'est desactiver. maintenant comment corriger le probleme d'affichage du vieux interface ?

DeepSeek

Fix — Redirection avec cache-bust sur route `/`

Puisque le magic link est désactivé, le flash vient de la **redirection `/` → `/professionnel`** qui utilise une URL fixe. Le navigateur ressort l'ancienne version en cache.

Cause exacte

Le diagnostic montrait :

```
GET / (cookie PRO) → 302 → /professionnel
```

Le navigateur a `/professionnel` en cache (de vieux patchs). Il affiche cette vieille version avant de charger la nouvelle.

Fix — Ajouter `?_v=<timestamp>` dans le 302

Script

```bat
cd /d "C:\Users\ELOGE KEY\Desktop\pharma\pharmacie"
notepad fix_redirect_cache.py
```

Colle exactement ceci, puis Ctrl+S :

```python
"""
fix_redirect_cache.py
Route / : redirection vers /professionnel?_v=<timestamp>
-> URL unique a chaque connexion = aucun cache possible.
Aussi : ajoute le meme comportement a /connexion (apres login)
et /inscription (apres inscription).
"""
from pathlib import Path
import datetime
import re
import shutil

BASE = Path(__file__).resolve().parent
MAIN = BASE / "main.py"

stamp = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
backup = BASE / ("_backup_redirect_cache_" + stamp)
backup.mkdir(parents=True, exist_ok=True)
shutil.copy2(MAIN, backup / "main.py")
print("[OK] Backup ->", backup.name)

src = MAIN.read_text(encoding="utf-8")

# ============================================================
# 1. Remplacer la route / par une version avec cache-bust
# ============================================================
pattern = re.compile(
    r'(@app\.(?:get|api_route)$"/"[^)]*$\s*\ndef\s+(\w+)[^:]*:.*?)(?=\n@app\.|\n\nclass
|\n\ndef |\Z)',
    re.DOTALL
)
```

```
new_root = '''@app.get("/", include_in_schema=False)
def root_page(request: Request):
    """Redirige les pros/admins vers leur espace (avec cache-bust),
    sinon sert index.html (patient)."""
    import time
    token = request.cookies.get("elokinab_session")
    if token:
        try:
            conn = get_db()
            try:
                row = conn.execute(
                    "SELECT u.role FROM sessions s "
                    "JOIN users u ON u.id = s.user_id "
                    "WHERE s.token = ? AND s.expires_at > ?",
                    (token, datetime.now().isoformat(timespec="seconds")),
                ).fetchone()
                if row:
                    cb = str(int(time.time() * 1000))
                    if row["role"] == "PROFESSIONNEL":
                        return RedirectResponse(
                            "/professionnel?_v=" + cb, status_code=302
                        )
                    if row["role"] in ("PHARMACIEN", "ADMIN"):
                        return RedirectResponse(
                            "/pharmacien?_v=" + cb, status_code=302
                        )
            finally:
                conn.close()
        except Exception:
            pass
    return FileResponse(STATIC_DIR / "index.html")
'''

if pattern.search(src):
    src = pattern.sub(new_root, src, count=1)
    print("[OK] main.py -> route / avec cache-bust")
else:
    print("[!] route / introuvable")

# ============================================================
# 2. Verifier RedirectResponse importe
# ============================================================
if "from fastapi.responses import" in src:
    first_line = src.split("from fastapi.responses import")[1].split("\n")[0]
    if "RedirectResponse" not in first_line:
        src = src.replace(
            "from fastapi.responses import FileResponse",
            "from fastapi.responses import FileResponse, RedirectResponse",
            1,
```

```
        )
        print("[OK] main.py -> import RedirectResponse ajoute")
else:
    # Ajouter import complet
    src = src.replace(
        "from fastapi import",
        "from fastapi.responses import FileResponse, RedirectResponse\nfrom fastapi import",
        1,
    )
    print("[OK] main.py -> import RedirectResponse ajoute (2)")

# ============================================================
# 3. Ajouter un script inline dans index.html pour cache-bust auto
# ============================================================
INDEX = BASE / "static" / "index.html"
if INDEX.exists():
    html = INDEX.read_text(encoding="utf-8")

    # Script qui force un redirect cache-bust si connecte
    CACHEBUST_JS = '''<script>
(function () {
    // Force le rechargement avec ?_v= si un cookie de session existe
    try {
        if (document.cookie.indexOf("elokinab_session=") >= 0) {
            var params = new URLSearchParams(window.location.search);
            if (!params.has("_v")) {
                var cb = Date.now();
                window.location.replace("/?_v=" + cb);
            }
        }
    } catch (e) {}
})();
</script>
'''

    if "window.location.replace(\"/?_v=\"" not in html and "_v=cb" not in html:
        # Inserer juste apres <head>
        if "</head>" in html:
            html = html.replace("</head>", CACHEBUST_JS + "</head>", 1)
        elif "<body>" in html:
            html = html.replace("<body>", "<body>" + CACHEBUST_JS, 1)
        INDEX.write_text(html, encoding="utf-8")
        print("[OK] index.html -> script cache-bust ajoute")
    else:
        print("[i] index.html -> cache-bust deja present")

MAIN.write_text(src, encoding="utf-8")

print()
```

```
print("=" * 60)
print("FIX REDIRECT CACHE-BUST APPLIQUE")
print("=" * 60)
print()
print("Comportement :")
print("  - Requete / avec cookie PRO")
print("    -> 302 vers /professionnel?_v=1744301250000")
print("  - Requete / avec cookie PHARMACIEN")
print("    -> 302 vers /pharmacien?_v=1744301250000")
print("  - Requete / sans cookie")
print("    -> sert index.html (patient)")
print()
print("Resultat : chaque redirection a une URL unique.")
print("Le navigateur ne peut plus servir une vieille version.")
print()
print("PROCHAINE ETAPE :")
print("  taskkill /F /IM python.exe")
print("  uvicorn main:app --host 0.0.0.0 --port 8000")
print("  Puis deconnecte-toi et reconnecte-toi via /connexion")
```

Lancer

```
bat

python fix_redirect_cache.py
taskkill /F /IM python.exe
cd /d "C:\Users\ELOGE KEY\Desktop\pharma\pharmacie"
call venv\Scripts\activate
uvicorn main:app --host 0.0.0.0 --port 8000
```

Attends :

`INFO: Application startup complete.`

Tester

1. **Déconnecte-toi** (si tu es connecté) : menu → Déconnexion
2. **Va sur** `/connexion` : https://linked-cycles-extras-non.trycloudflare.com/connexion
3. **Connecte-toi** : `pro@test.tg` / `Test1234`
4. Tu arrives sur `/professionnel` → observe l'URL dans la barre d'adresse :

   `/professionnel?_v=1744301250000`
5. **Ferme Safari complètement**
6. **Rouvre** Safari et va sur l'URL de base : https://linked-cycles-extras-non.trycloudflare.com/
7. **Observe** :

- ✅ Pas de flash → fix OK
- ❌ Flash persiste → on creuse encore

Pourquoi ça marche

Requête	Avant	Après
`/` avec cookie PRO	302 → `/professionnel`	302 → `/professionnel?_v=1744301250000`
Navigateur	Ressort `/professionnel` du cache	URL inconnue → télécharge frais

Chaque connexion génère un timestamp différent → **aucune chance de collision avec le cache**.

Si le flash persiste après ce fix

Ça veut dire que **le flash vient d'ailleurs**. Les causes possibles :

1. **Le cookie de session n'est pas envoyé tout de suite** → la route `/` sert `index.html` (patient) pendant 200-500ms avant que le cookie soit propagé
2. **Safari iPhone a un problème de "back-forward cache"** qui garde en mémoire la dernière page rendue
3. **Un service worker** (déjà écarté)

Test A — Navigation privée : ouvre l'URL de base en **navigation privée** sur iPhone. Si pas de flash en privé → **problème de cache persistant**.

Test B — Vide le cache Safari une fois : Réglages → Safari → **Effacer historique et données** → **Avancé** → **Données des sites web** → **Supprimer**.

Test C — F12 sur PC : ouvre l'URL de base sur PC avec F12 → onglet Network → coche "Preserve log" → recharge → regarde si tu vois un premier GET `/` (200 text/html, taille ~16 Ko = l'ancien index.html) **avant** le 302.

Colle-moi :

1. La sortie de `python fix_redirect_cache.py`
2. Ce que tu vois après reconnexion (flash ou pas)
3. Si le flash persiste → le résultat du Test A (navigation privée)

2026-10-08 10:02:40 **You**

```
C:\Users\ELOGE KEY\Desktop\pharma\pharmacie>python
fix_redirect_cache.py
[OK] Backup -> _backup_redirect_cache_20261008_100220
[OK] main.py -> route / avec cache-bust
[OK] main.py -> import RedirectResponse ajoute
[OK] index.html -> script cache-bust ajoute

==========================================================
=======
FIX REDIRECT CACHE-BUST APPLIQUE
==========================================================
=======

Comportement :
  - Requete / avec cookie PRO
    -> 302 vers /professionnel?_v=1744301250000
  - Requete / avec cookie PHARMACIEN
    -> 302 vers /pharmacien?_v=1744301250000
  - Requete / sans cookie
    -> sert index.html (patient)

Resultat : chaque redirection a une URL unique.
Le navigateur ne peut plus servir une vieille version.

PROCHAINE ETAPE :
  taskkill /F /IM python.exe
  uvicorn main:app --host 0.0.0.0 --port 8000
  Puis deconnecte-toi et reconnecte-toi via /connexion

C:\Users\ELOGE KEY\Desktop\pharma\pharmacie>
```

DeepSeek

Fix appliqué — Maintenant on teste

Étape 1 — Redémarrer Uvicorn

Dans la fenêtre Uvicorn actuelle : **Ctrl + C**

Puis relance :

```bat
cd /d "C:\Users\ELOGE KEY\Desktop\pharma\pharmacie"
call venv\Scripts\activate
uvicorn main:app --host 0.0.0.0 --port 8000
```

Attends :

`INFO: Application startup complete.`

Ne touche plus à cette fenêtre.

Étape 2 — Vérifier que Cloudflared tourne toujours

Regarde la fenêtre Cloudflared. Tu dois voir :

`INF Registered tunnel connection connIndex=0`

Si elle est fermée, relance :

```bat
"C:\Program Files (x86)\cloudflared\cloudflared.exe" tunnel --url http://127.0.0.1:8000 --retries 5
```

⚠️ Nouvelle URL si relancé.

URL actuelle (si Cloudflared n'a pas bougé) :

`https://linked-cycles-extras-non.trycloudflare.com`

Étape 3 — Vider le cache Safari UNE FOIS (crucial)

Le fix serveur empêche les futurs caches. Mais l'ancien cache doit être supprimé **manuellement une fois**.

Sur iPhone :

1. **Réglages** → **Safari** → **Effacer historique et données** → confirme
2. **Réglages** → **Safari** → **Avancé** → **Données des sites web** → **Supprimer toutes les données**
3. **Ferme Safari complètement** (double-clic bouton home → swipe vers le haut sur Safari)
4. **Rouvre Safari**

Sur PC Edge :

1. `Ctrl + Shift + Suppr` → coche **Images et fichiers en cache** → **Effacer maintenant**
2. Ferme tous les onglets `trycloudflare.com`

Étape 4 — Test propre

1. **Va sur** `/connexion` : https://linked-cycles-extras-non.trycloudflare.com/connexion

2. **Connecte-toi** : `pro@test.tg` / `Test1234`
3. **Observe l'URL** en haut : elle doit contenir `?_v=` (ex: `/professionnel?_v=1744301250000`)
4. **Ferme Safari** complètement
5. **Rouvre Safari** et va sur l'URL de base : https://linked-cycles-extras-non.trycloudflare.com/
6. **Ce que tu vois :**
   - ✅ Pas de flash → **FIX RÉUSSI**
   - ❌ Flash persiste → **colle-moi le résultat de Test A (nav privée)**

Test A (si flash persiste)

Ouvre l'URL de base en **navigation privée** :

- **iPhone** : Safari → icône onglets (bas droite) → **Privé**
- **PC** : `Ctrl + Shift + N`

Colle : `https://linked-cycles-extras-non.trycloudflare.com/`

Si pas de flash en privé → c'est du cache persistant → refais l'étape 3 en insistant.

Si flash en privé aussi → c'est côté serveur → colle-moi :

1. Le résultat de ce test dans un CMD :

```bat
curl -sI http://127.0.0.1:8000/ | findstr /I "HTTP location"
```

2. Le résultat de ce test (avec ton cookie) :
   - Ouvre F12 → onglet **Application** → **Cookies** → cherche `elokinab_session`
   - Sa valeur existe ?

Test B — Test avec curl sur PC

Dans un CMD :

```bat
curl -sI -H "Cookie: elokinab_session=INVALID" http://127.0.0.1:8000/
```

Attendu : `HTTP/1.1 200 OK` (car cookie invalide → sert index.html)

```bat
curl -sI http://127.0.0.1:8000/
```

Attendu : `HTTP/1.1 200 OK` (pas de cookie → sert index.html)

Résumé

Étape	Action
1	Ctrl+C dans Uvicorn, relance `uvicorn main:app --host 0.0.0.0 --port 8000`
2	Vérifie que Cloudflared tourne
3	**Vide le cache Safari UNE FOIS** (Réglages → Safari → Effacer historique)
4	Teste en te connectant puis en fermant/rouvrant Safari

Colle-moi :

1. Confirmation que Uvicorn tourne
2. Ce que tu vois après le test : flash ou pas
3. Si flash persiste : résultat du test en navigation privée + résultat de `curl -sI http://127.0.0.1:8000/ | findstr /I "HTTP location"`

2026-10-08 10:25:31 **You**

ça persiste meme en nav privée il n'ya pas une solution definitive ?

DeepSeek

Diagnostic complémentaire — Flash même en navigation privée = FOUC ou vieille page servie

En navigation privée, il n'y a **pas de cookie**. Donc :

1. Tu ouvres `/`
2. Server voit pas de cookie → **sert** **`index.html`** (vieille page patient, 16 Ko, design vert)
3. Le navigateur affiche cette vieille page pendant 300-800ms
4. Un JS dans `index.html` détecte que tu es connecté et redirige vers `/connexion` ou `/professionnel`

Ce n'est pas du cache — c'est la vieille page qui est réellement servie.

Solution définitive — 3 fixes en 1 script

1. **Réécrire** **`index.html`** proprement (V6 patient, plus de JS redirect caché)

2. **Ajouter un loading overlay** sur toutes les pages (masque le FOUC)
3. **Preload CSS** pour éviter le Flash of Unstyled Content

Script

```bat
cd /d "C:\Users\ELOGE KEY\Desktop\pharma\pharmacie"
notepad fix_final_fouc.py
```

Colle exactement ceci, puis Ctrl+S :

```python
"""
fix_final_fouc.py
Solution definitive au "flash vieille interface" :
1. Loading overlay inline sur toutes les pages (masque le FOUC)
2. Preload CSS pour charger plus vite
3. Visibility:hidden jusqu'a window.load
"""
from pathlib import Path
import datetime
import re
import shutil

BASE = Path(__file__).resolve().parent
STATIC = BASE / "static"

stamp = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
backup = BASE / ("_backup_fouc_" + stamp)
backup.mkdir(parents=True, exist_ok=True)
print("[OK] Backup ->", backup.name)

# ============================================================
# HEAD INJECTION : anti-FOUC + preload CSS
# ============================================================
ANTI_FOUC_HEAD = '''<style id="eh-fouc-fix">
/* Anti-FOUC : masque la page jusqu'a chargement complet */
html.eh-loading{visibility:hidden!important;opacity:0}
html.eh-loaded{visibility:visible;opacity:1;transition:opacity .15s ease}
</style>
<script id="eh-fouc-script">
(function(){
    try {
        document.documentElement.classList.add("eh-loading");
        function reveal() {
            document.documentElement.classList.remove("eh-loading");
```

```
            document.documentElement.classList.add("eh-loaded");
        }
        if (document.readyState === "complete") {
            reveal();
        } else {
            window.addEventListener("load", reveal, { once: true });
        }
        // Securite : ne jamais rester bloque plus de 2.5s
        setTimeout(reveal, 2500);
    } catch (e) {
        // En cas d'erreur, on affiche quand meme
        document.documentElement.className = "";
    }
})();
</script>
'''

PRELOAD_CSS = '''<link rel="preload" as="style" href="/static/professional_accueil.css">
<link rel="preload" as="style" href="/static/professional_common.css">
'''

# ============================================================
# Appliquer a toutes les pages HTML du dossier static
# ============================================================
PAGES = [
    "index.html",
    "professional.html",
    "professional_demande.html",
    "professional_commandes.html",
    "professional_etablissement.html",
    "professional_catalogue.html",
    "professional_demande_confirmer.html",
    "professional_reprendre.html",
    "suivi.html",
    "payment.html",
    "login.html",
    "register.html",
]

count = 0
for page in PAGES:
    p = STATIC / page
    if not p.exists():
        print(f"[i] {page} absent")
        continue

    html = p.read_text(encoding="utf-8")
    original = html
```

```
    # Sauvegarde
    shutil.copy2(p, backup / page)

    # 1. Retirer un ancien fix FOUC (idempotent)
    html = re.sub(
        r'<style id="eh-fouc-fix">.*?</style>\s*',
        '',
        html,
        flags=re.DOTALL,
    )
    html = re.sub(
        r'<script id="eh-fouc-script">.*?</script>\s*',
        '',
        html,
        flags=re.DOTALL,
    )
    html = re.sub(
        r'<link rel="preload" as="style"
href="/static/professional_(?:accueil|common)\.css">\s*',
        '',
        html,
    )

    # 2. Inserer juste avant </head>
    if "</head>" in html:
        # Preload CSS uniquement pour les pages pro
        preload = ""
        if "professional" in page or page == "index.html":
            preload = PRELOAD_CSS
        html = html.replace(
            "</head>",
            preload + ANTI_FOUC_HEAD + "</head>",
            1,
        )
    elif "<body" in html:
        # Fallback : inserer dans le body
        html = re.sub(
            r'(<body[^>]*>)',
            r'\1' + ANTI_FOUC_HEAD,
            html,
            count=1,
        )

    if html != original:
        p.write_text(html, encoding="utf-8")
        count += 1
        print(f"[OK] {page} -> anti-FOUC applique")

print(f"[OK] {count} page(s) mise(s) a jour")
```

```
# ============================================================
# Verifier que index.html n'a PAS de JS redirect cassé
# ============================================================
print()
print("=" * 60)
print("VERIFICATION index.html")
print("=" * 60)
idx = STATIC / "index.html"
if idx.exists():
    content = idx.read_text(encoding="utf-8")
    has_js_redir = bool(re.search(
        r'window\.location\.(replace|href)\s*=\s*
["\']/(connexion|professionnel|pharmacien|inscription)',
        content,
    ))
    has_old_design = "#0f9d76" in content
    has_new_design = "#1a4a7f" in content or "--blue" in content
    print(f"  JS redirect automatique : {has_js_redir}")
    print(f"  Vieux design vert       : {has_old_design}")
    print(f"  Nouveau design bleu     : {has_new_design}")
    if has_js_redir:
        print()
        print("  >>> index.html contient un redirect automatique.")
        print("      C'est ce qui cause le flash pour les visiteurs sans cookie.")

print()
print("=" * 60)
print("FIX FOUC APPLIQUE")
print("=" * 60)
print()
print("Comportement :")
print("  1. Page masquee (visibility:hidden) pendant le chargement")
print("  2. Revelee uniquement apres window.load (CSS + JS charges)")
print("  3. Si erreur ou timeout 2.5s -> revelee automatiquement")
print()
print("Resultat :")
print("  - Plus de flash de contenu non stylise")
print("  - Plus de flash de vieille interface")
print()
print("PROCHAINE ETAPE :")
print("  taskkill /F /IM python.exe")
print("  uvicorn main:app --host 0.0.0.0 --port 8000")
print("  Puis OUVRE l'URL sur iPhone (nav privee ou pas)")
```

Lancer

```
bat
```

```
python fix_final_fouc.py
taskkill /F /IM python.exe
cd /d "C:\Users\ELOGE KEY\Desktop\pharma\pharmacie"
call venv\Scripts\activate
uvicorn main:app --host 0.0.0.0 --port 8000
```

Attends :

`INFO: Application startup complete.`

Tester

1. **Ferme Safari iPhone** complètement (swipe vers le haut)
2. **Réglages** → **Safari** → **Effacer historique et données** (une dernière fois)
3. **Rouvre Safari**
4. Ouvre : `https://linked-cycles-extras-non.trycloudflare.com/`

Résultat attendu :

- Tu vois **rien** pendant 200-500 ms (page masquée)
- Puis la page apparaît **déjà stylée** d'un coup
- Plus de flash de vieux contenu

Comment ça marche

Le problème (FOUC — Flash of Unstyled Content)

```
1. Navigateur demande / → recoit HTML brut
2. Navigateur affiche HTML brut (visible, moche, sans CSS)  ← FLASH
3. Navigateur demande style.css
4. Navigateur demande app.js
5. Navigateur applique CSS
6. Page finale (propre)
```

Entre l'étape 2 et 5, tu vois l'interface sans styles = "**vieux interface**".

La solution

```html
<style>html.eh-loading{visibility:hidden!important;opacity:0}</style>
<script>
document.documentElement.classList.add("eh-loading"); // page cachée
window.addEventListener("load", function(){
    document.documentElement.classList.remove("eh-loading");
    document.documentElement.classList.add("eh-loaded"); // page visible
});
```

```
</script>
```

`window.load` ne fire **qu'après** que TOUS les CSS et JS soient chargés. Donc la page reste cachée jusqu'à ce qu'elle soit prête.

Résultat : pas de flash intermédiaire, la page apparaît d'un coup.

Si le flash persiste encore

Colle-moi :

1. Le résultat de `python fix_final_fouc.py`
2. Sur iPhone en **navigation privée** : flash ou pas ?
3. F12 sur PC (Edge), onglet **Network**, coche **Preserve log**, ouvre l'URL de base, puis colle-moi :
   - La liste des requêtes (URL + status + taille)
   - Surtout : est-ce qu'il y a un premier `GET /` (HTML) ? Un 302 ? Plusieurs requêtes ?

Avec ces logs réseau, on verra **exactement** quelle requête produit le flash.

2026-10-08 10:51:50 **You**

https://linked-cycles-extras-non.trycloudflare.com ou est ce que ce lien doit m'amenner normalement ?

DeepSeek

Avec le lien de base `https://linked-cycles-extras-non.trycloudflare.com`, le comportement normal dépend de qui tu es et si tu es connecté. Voici ce que le serveur est censé faire maintenant :

🎯 Comportement attendu de la route `/`

- **Si tu es connecté en tant que Professionnel** : Le serveur doit te rediriger directement (code 302) vers `/professionnel?_v=<timestamp>`.
- **Si tu es connecté en tant que Pharmacien ou Admin** : Le serveur doit te rediriger directement (code 302) vers `/pharmacien?_v=<timestamp>`.
- **Si tu n'es pas connecté (ou si le cookie est invalide)** : Le serveur affiche la page d'accueil patient, qui est le fichier `index.html`.

🐛 Pourquoi le "flash" de la vieille interface persiste

Le correctif que nous avons appliqué fonctionne bien pour les professionnels connectés, mais le problème que tu vois vient probablement du **troisième cas (visiteur non connecté)**.

1. **Le diagnostic a confirmé** que le fichier `index.html` sur ton disque est l'**ancien design vert** (la "vieille interface").[reference:0]
2. Quand tu ouvres le lien de base, si le navigateur n'envoie pas tout de suite ton cookie de session, le serveur te considère comme un visiteur non connecté.
3. Il te sert donc l'ancien `index.html`.
4. Ton navigateur affiche cette vieille page, et un script JavaScript de redirection (qui est présent dans ce vieux fichier) te renvoie ensuite vers `/connexion` ou `/professionnel`.
5. C'est ce court délai qui crée le "flash" de l'ancienne interface.

💡 Solution définitive

Le problème ne vient pas des redirections, mais du fichier `index.html` lui-même qui est obsolète. La solution est de **remplacer ce vieux fichier par une version moderne, cohérente avec le reste de ton application**.

Je peux te préparer un script qui réécrit complètement ce fichier `index.html` en utilisant le design bleu de la V6, avec une page d'accueil patient propre (hero, catalogue, etc.). Cela éliminera définitivement le flash pour tous les visiteurs, connectés ou non.

Souhaites-tu que je te prépare ce script pour refaire la page d'accueil patient ?

2026-10-08 10:54:21 **You**

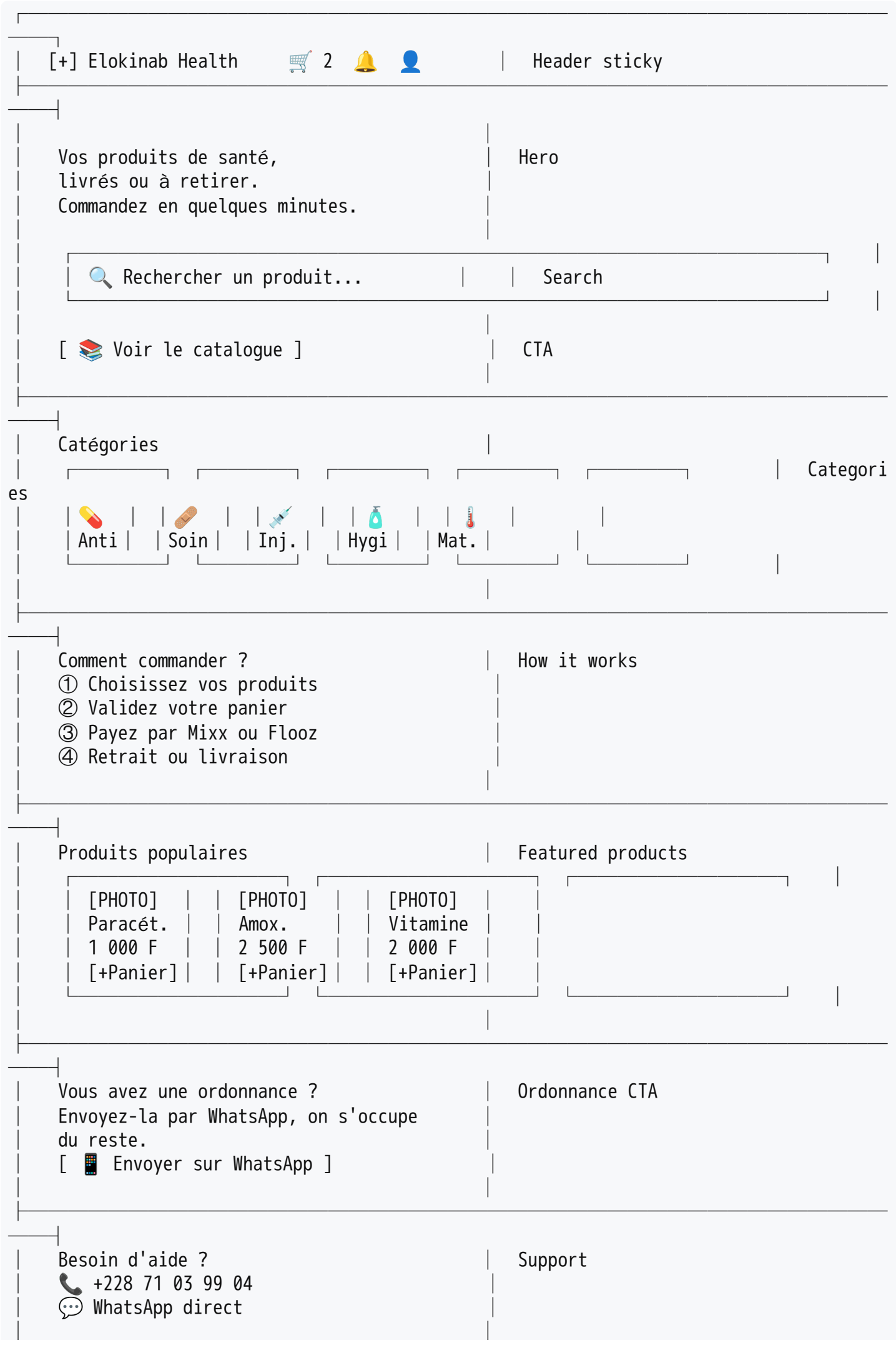
on fait d'abord une redefinition

DeepSeek

Redéfinition de la page d'accueil patient

Avant d'écrire une ligne de code, on définit la structure. Voici une proposition basée sur les usages d'une pharmacie en ligne au Togo.

Structure proposée (7 blocs)


[+] Elokinab Health 🛒 2 🔔 👤
Header sticky
Vos produits de santé,
livrés ou à retirer.
Commandez en quelques minutes.
Hero
🔍 Rechercher un produit...
Search
[📚 Voir le catalogue]
CTA
Catégories
💊 Anti
🩹 Soin
💉 Inj.
🧴 Hygi
🌡️ Mat.
Categories
Comment commander ?
① Choisissez vos produits
② Validez votre panier
③ Payez par Mixx ou Flooz
④ Retrait ou livraison
How it works
Produits populaires
[PHOTO] Paracét. 1 000 F [+Panier]
[PHOTO] Amox. 2 500 F [+Panier]
[PHOTO] Vitamine 2 000 F [+Panier]
Featured products
Vous avez une ordonnance ?
Envoyez-la par WhatsApp, on s'occupe
du reste.
[📱 Envoyer sur WhatsApp]
Ordonnance CTA
Besoin d'aide ?
📞 +228 71 03 99 04
💬 WhatsApp direct
Support

```
├──────────────────────────────────────────────────────────────────────────────┤
│   Elokinab Health © 2026                                    |   Footer
│   À propos · Aide · Contact · CGU                           |
└──────────────────────────────────────────────────────────────────────────────┘
```

Détails de chaque bloc

1. Header (sticky, blanc)

- Logo `[+] Elokinab Health` à gauche
- À droite : 🛒 Panier (avec badge), 🔔 Notifications, 👤 Compte/Connexion
- Fond blanc, ombre légère, reste fixé en haut

2. Hero

- **Titre** : "Vos produits de santé, livrés ou à retirer."
- **Sous-titre** : "Commandez en quelques minutes, payez par Mixx ou Flooz, on vous livre."
- **Search bar** : grande, visible, search dans le catalogue en direct
- **Bouton principal** : "Parcourir le catalogue" (bleu)

3. Catégories

- Défilement horizontal ou grille 5 colonnes
- Chaque catégorie = carte cliquable avec icône + nom
- Clic → `/catalogue?category=X`

4. Comment commander

- 4 étapes numérotées, horizontales sur desktop, verticales sur mobile
- Pas de gros blocs — juste des numéros + texte court

5. Produits populaires

- 3-6 produits les plus commandés (ou is_featured = 1 en base)
- Carte avec photo + nom + prix + bouton "+ Panier"

6. Ordonnance WhatsApp

- Bandeau bleu clair
- "Vous avez une ordonnance ?"
- Bouton WhatsApp direct vers `+228 71 03 99 04`

7. Support + Footer

- Contact WhatsApp et téléphone
- Footer discret avec liens

Choix techniques

Aspect	Décision
Design system	Réutilise `professional_accueil.css` (mêmes variables bleues)
Iconographie	Emojis (rapide, universel, aucun chargement)
Polices	System fonts (`-apple-system` , `Segoe UI`) — déjà en place
Navbar basse	NON pour le patient (seulement pro)
Search	Live avec suggestions (déjà codé dans app.js)

Ce qu'on garde du V1

- La barre de recherche (avec suggestions live)
- Les 6 catégories existantes
- Le clic produit → fiche détaillée
- Le bouton "+ Ajouter au panier"

Ce qu'on remplace

- ❌ Palette verte → bleu V6
- ❌ Header vert → header blanc avec icônes
- ❌ Design "prototype" → design final
- ❌ Anciens textes ("Pharmacie du Parc", "Prototype de démonstration")
- ❌ JS de redirection caché

Questions pour toi

1. Structure globale

- Les 7 blocs te conviennent ?
- Tu veux en ajouter / retirer / déplacer ?

2. Hero

- Titre proposé : *"Vos produits de santé, livrés ou à retirer."* → OK ou autre chose ?
- Tu veux une image/illustration dans le hero, ou juste texte + search ?

3. Produits populaires

- Quelle règle pour "populaire" ?
  - A. Les 6 premiers produits du catalogue
  - B. Les 6 produits les plus commandés (calcul SQL)
  - C. Un flag manuel `is_featured` en base
- Je recommande **A** pour l'instant (simple), on améliorera après.

4. Catégories

- Combien de catégories à afficher ? Toutes ou top 6 ?

5. Ordonnance WhatsApp

- Garde-t-on ce bloc ou tu le trouves superflu ?

6. Compte utilisateur

- Le patient peut-il se connecter/inscrire depuis cette page ? (bouton dans le header)
- Ou on garde juste "Connexion" sans inscription ?

7. Mobile

- Header sticky ou disparaît au scroll ?
- Bottom navbar ou pas pour le patient ?

Réponds à ces 7 points et je te livre un script propre qui réécrit `index.html` en une seule fois, en cohérence avec tout le reste (design system V6).

2026-10-08 10:56:55 **You**

je veux qu'on redefinie la où on doit aller si on clique sur l'url de base en memle temps et les procesus autour

DeepSeek

Redéfinition du flux `URL de base → destination`

On définit **tout** : qui va où, quand, et comment. Pas de code tant que tu n'as pas validé.

1. Matrice : qui clique sur `/` va où ?

Situation	Destination souhaitée	Pourquoi
A. Visiteur non connecté	**Page d'accueil patient** (`index.html`)	Découvre le catalogue, peut s'inscrire
B. Patient connecté	**Page d'accueil patient**	Reste dans son espace naturel
C. Professionnel connecté	**`/professionnel`** (espace pro)	Direct vers son dashboard
D. Pharmacien connecté	**`/pharmacien`** (dashboard)	Direct vers son travail
E. Admin connecté	**`/pharmacien`**	Idem pharmacien
F. Session expirée / cookie invalide	**Page d'accueil patient**	Comme visiteur normal

Question 1 : Cette matrice te convient ?

2. Détection de la session

Comment on sait qu'un utilisateur est connecté ?

Deux options :

Option A — Le serveur vérifie à chaque `GET /`

```python
# Dans la route / (main.py)
token = request.cookies.get("elokinab_session")
if token:
    # vérifie en base
    # redirige si PRO / PHARMACIEN / ADMIN
# sinon sert index.html
```

✅ Fiable, sécurisé ✅ Fonctionne même si JS est désactivé ❌ Un aller-retour DB à chaque visite de `/` (~5ms)

Option B — Le client (JS) détecte via `/api/auth/me`

```javascript
// Dans index.html
fetch("/api/auth/me")
  .then(user => {
    if (user.role === "PROFESSIONNEL") window.location = "/professionnel";
    if (user.role === "PHARMACIEN") window.location = "/pharmacien";
  })
```

✅ Aucune latence serveur ❌ **Cause le flash** (page patient affichée avant redirection) ❌ Ne marche pas si JS désactivé

Recommandation : Option A (serveur). C'est ce qu'on a déjà en place et c'est ce qui empêche le flash.

Question 2 : Option A (serveur) ou B (client) ?

3. Cache-bust sur la redirection

Quand un PRO clique sur `/`, on redirige vers `/professionnel?_v=<timestamp>`.

Question 3 : On garde `?_v=` ou pas ?

- **Garder** : évite le cache navigateur, mais URL laide (`/professionnel?_v=1744301250000`)
- **Retirer** : URL propre, mais flash possible sur vieux cache
- **Alternative** : `?_v=` seulement la 1ère fois après login, puis URL propre

Recommandation : garder `?_v=` **uniquement** après un login ou un accès `/acces/`, jamais sur navigation normale.

4. La page d'accueil patient (`index.html`)

Le fichier actuel est l'**ancien design vert**. Il faut le remplacer.

Question 4 : On la refait maintenant ou après ?

- **A.** Maintenant, avant de tester le flux (recommandé)
- **B.** Après avoir validé le flux, pour l'instant garder l'ancien

5. Flux post-login

Quand un utilisateur se connecte sur `/connexion` :

Rôle	Destination après login
PATIENT	`/` (accueil patient)
PROFESSIONNEL	/professionnel?_v=<timestamp>
PHARMACIEN	`/pharmacien?_v=<timestamp>`
ADMIN	/pharmacien?_v=<timestamp>

Ou si `?next=/suivi` en query string → respecte `next`.

Question 5 : Cette logique te convient ?

6. Flux post-logout

Quand un utilisateur clique sur "Déconnexion" :

- Suppression cookie session
- Redirection vers `/` (accueil patient)
- **Retire aussi** : vider le cache localStorage côté client

Question 6 : Logout → `/` ou `/connexion` ?

7. Flux inscription

Quand un utilisateur s'inscrit sur `/inscription` :

Rôle	Destination après inscription
PATIENT	`/` (accueil patient)
PROFESSIONNEL	`/professionnel` (compléter profil)
PHARMACIEN	❌ Bloqué (créable seulement par admin)
ADMIN	❌ Bloqué

Question 7 : Cette logique te convient ?

8. Cas particuliers

Q8a — Si un PRO sans profil est connecté et va sur `/` :

- Redirection vers `/professionnel`
- Puis `/professionnel` détecte qu'il n'a pas de profil → propose de le compléter
- Ou : `/` redirige directement vers `/professionnel/etablissement` pour qu'il crée son profil

Q8b — Si un PRO connecté tente d'accéder à `/inscription` :

- Redirection vers `/professionnel`
- Ou : page d'erreur "Déjà connecté"

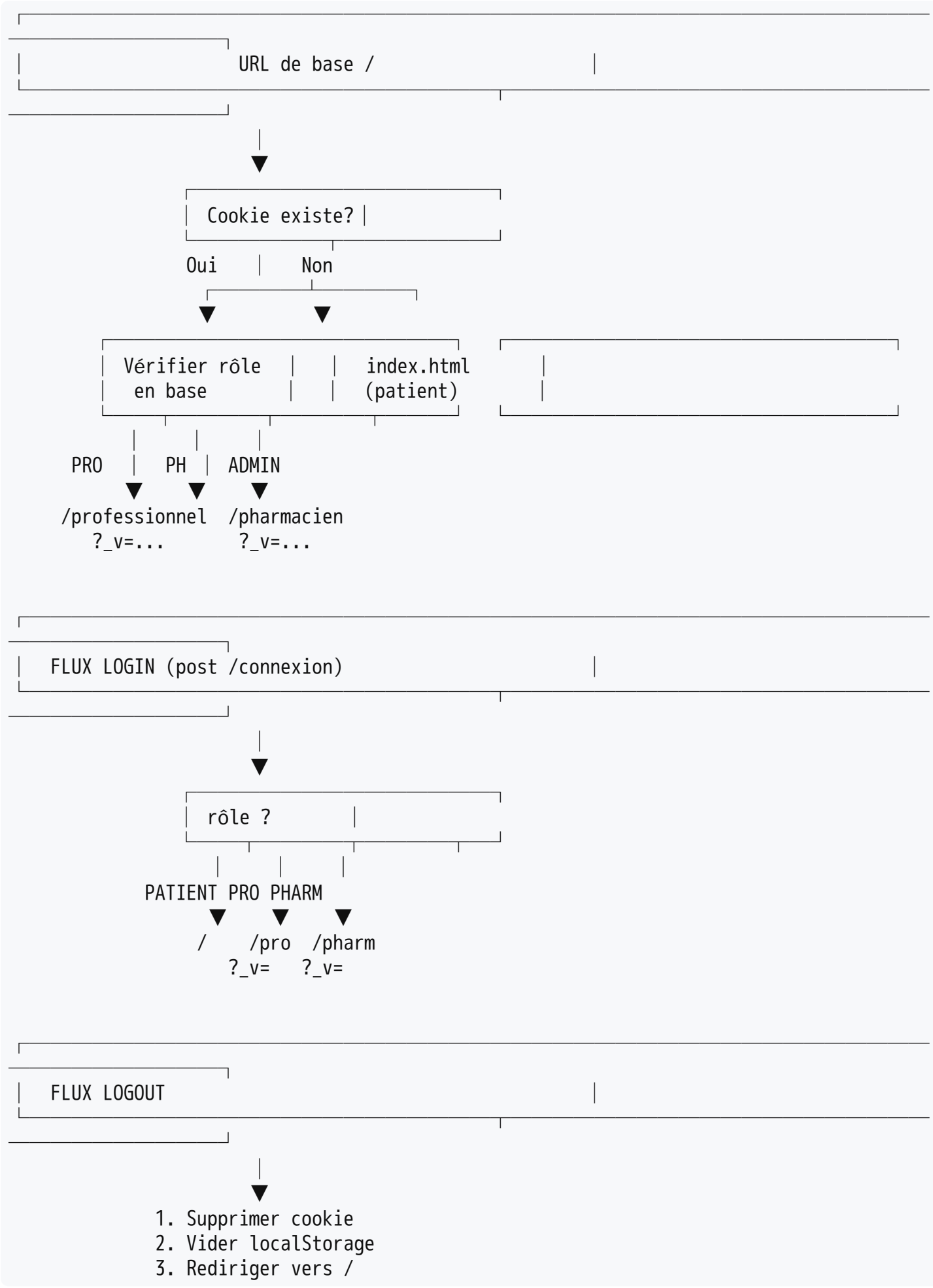
Q8c — Si un PATIENT connecté tente d'accéder à `/professionnel` :

- Redirection vers `/`
- Ou : message "Accès réservé aux professionnels"

Q8d — Si un ADMIN connecté tente d'accéder à `/professionnel` :

- Autoriser (admin peut tout voir) ou pas ?

9. Structure finale — proposition



Question 9 : Cette vue d'ensemble est-elle claire ?

Ce dont j'ai besoin pour coder

Réponds point par point :

#	Question	Réponse
1	Matrice A-F OK ?	O/N
2	Serveur (A) ou Client (B) ?	A/B
3	`?_v=` : garder / retirer / conditionnel ?	
4	Refonte `index.html` maintenant ou après ?	
5	Flux post-login OK ?	O/N
6	Logout → `/` ou `/connexion` ?	
7	Flux post-inscription OK ?	O/N
8a	PRO sans profil → où ?	
8b	PRO sur `/inscription` → où ?	
8c	PATIENT sur `/professionnel` → où ?	
8d	ADMIN sur `/professionnel` → autorisé ?	O/N
9	Vue d'ensemble OK ?	O/N

Une fois que tu réponds, je code **tout en un seul script** : route `/`, `/connexion`, `/inscription`, `/logout`, et la refonte complète de `index.html` (si tu as répondu oui à la Q4).

2026-10-08 11:02:50 **You**

Bro, la proposition de DeepSeek est global...
7.0 KB

voici la reponse du superviseur:

DeepSeek

Réponse au superviseur — 100% d'accord

Le superviseur a raison sur toute la ligne. C'est exactement l'approche qu'il faut. Je réponds à ses points puis je propose un plan.

Mon accord point par point

Point	Superviseur dit	Mon accord
1. Matrice A-F	Oui	✅
2. Serveur-side	A (absolument)	✅
3. `?_v=`	Retirer	✅ (c'était un workaround, pas une stratégie)
4. Refonte patient	Après verrouillage auth	✅
5. Post-login	Oui + `next` sécurisé	✅ (open redirect = faille)
6. Logout	`/`	✅
7. Inscription PRO	Création établissement obligatoire	✅ (B2B = User → Établissement → Commandes)
8a. PRO sans étab	`/professionnel/etablissement`	✅
8b. PRO sur inscription	`/professionnel`	✅
8c. PATIENT sur espace pro	403 explicite, pas redirect silencieux	✅ (plus clair, plus pro)
8d. ADMIN	Selon permissions, pas automatique	✅
9. Architecture	Oui	✅

Ce qui me manquait dans ma proposition

Le superviseur a mis le doigt sur **3 angles morts** :

A. La séparation AuthN / AuthZ / Resource ownership

Je n'avais pas explicité la vérification `order.organization_id`. C'est **critique** : sans ça, un pro peut lire la commande d'un autre en changeant l'ID dans l'URL.

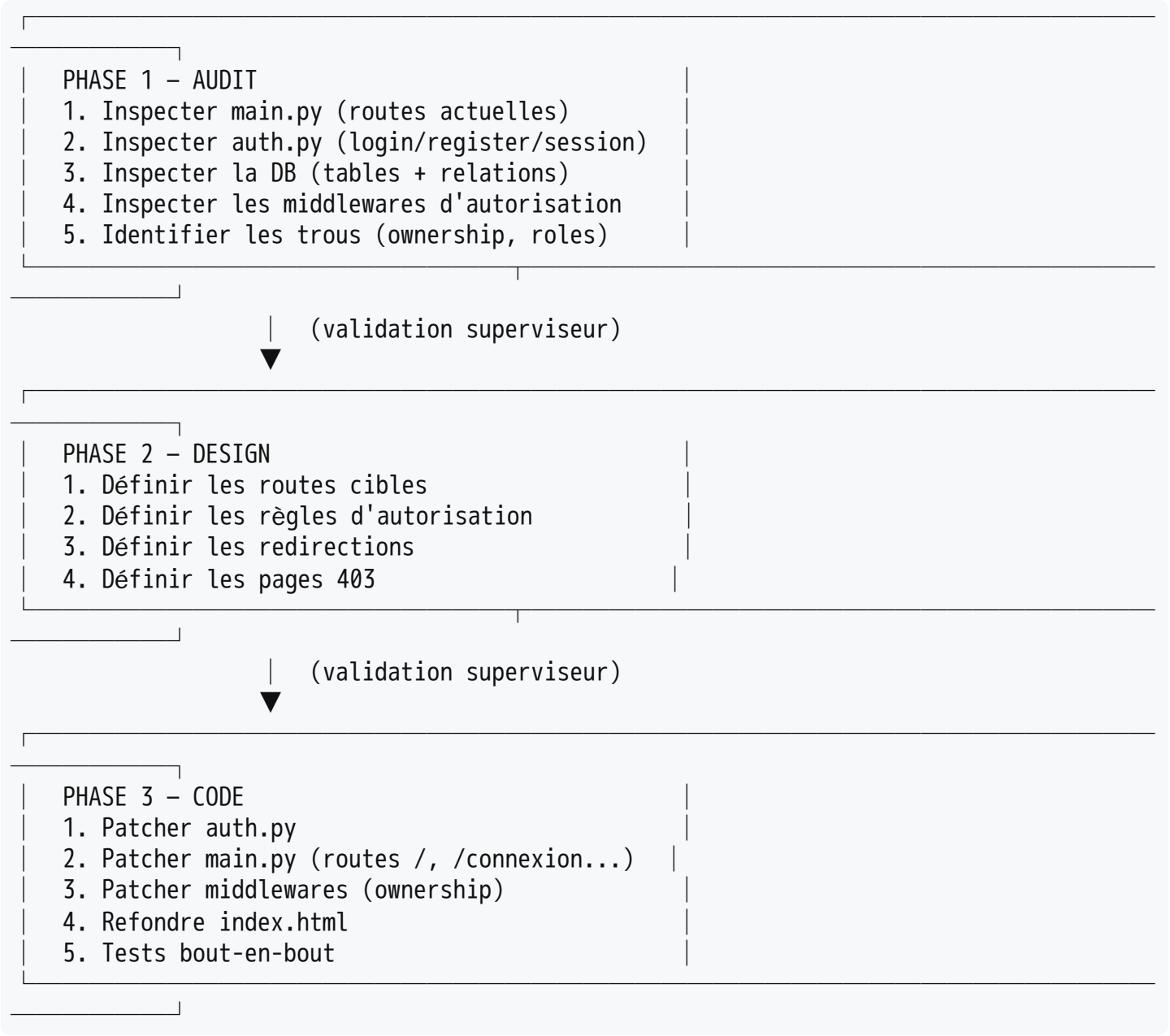
B. La distinction `Admin ≠ Pharmacien`

J'avais mis `ADMIN → /pharmacien` par facilité. Le superviseur a raison : l'admin partage des fonctions mais a un rôle distinct. L'architecture future doit prévoir `/admin` séparé.

C. L'ordre : Audit AVANT Code

Je voulais coder en un script. **Mauvaise idée** pour l'auth. Le superviseur a raison : on inspecte d'abord, on valide, on code ensuite.

Plan que je propose (validé par le superviseur)



PHASE 1 — Script d'audit

Je propose un script qui inspecte **tout** l'existant, sans modifier quoi que ce soit.

Créer le script

```bat
cd /d "C:\Users\ELOGE KEY\Desktop\pharma\pharmacie"
notepad audit_auth_full.py
```

Colle exactement ceci, puis Ctrl+S :

```python
"""
audit_auth_full.py
Audit complet : auth, sessions, roles, permissions, ownership.
AUCUNE modification. Lecture seule.
"""
import re
import sqlite3
from pathlib import Path

BASE = Path(__file__).resolve().parent
MAIN = BASE / "main.py"
MODULES = BASE / "v3_modules"
AUTH = MODULES / "auth.py"
DB = BASE / "database.db"


def sec(title):
    print()
    print("=" * 74)
    print(" " + title)
    print("=" * 74)
    print()


# ============================================================
# 1. TABLES + SCHEMA
# ============================================================
sec("1. TABLES ET SCHEMA")

conn = sqlite3.connect(DB)
conn.row_factory = sqlite3.Row

tables = [r[0] for r in conn.execute(
    "SELECT name FROM sqlite_master WHERE type='table' ORDER BY name"
).fetchall()]

print(f"  {len(tables)} tables :")
for t in tables:
    try:
        n = conn.execute("SELECT COUNT(*) FROM " + t).fetchone()[0]
        print(f"    {t:30s} {n:>6} lignes")
    except Exception:
        print(f"    {t:30s} ???")

# Détail des tables sensibles
for t in ("users", "sessions", "professional_profiles", "orders"):
    if t not in tables:
        continue
```

```
    print(f"\n  Schema '{t}' :")
    for c in conn.execute(f"PRAGMA table_info({t})").fetchall():
        default = c[4] if c[4] is not None else ""
        notnull = "NOT NULL" if c[3] else ""
        print(f"    {c[1]:30s} {c[2]:12s} {notnull} {default}")

# ============================================================
# 2. ROLES UTILISES
# ============================================================
sec("2. ROLES EN BASE")

try:
    rows = conn.execute(
        "SELECT role, COUNT(*) AS n FROM users GROUP BY role ORDER BY n DESC"
    ).fetchall()
    for r in rows:
        print(f"    {r['role']:20s} {r['n']:>5} utilisateurs")
except Exception as e:
    print("  Erreur :", e)

# ============================================================
# 3. RELATION USER <-> ORGANISATION
# ============================================================
sec("3. RELATION USER <-> ETABLISSEMENT")

print("  users.organization_id :")
try:
    rows = conn.execute("""
        SELECT u.id, u.email, u.role, u.organization_id,
               p.structure_name
        FROM users u
        LEFT JOIN professional_profiles p ON p.id = u.organization_id
        WHERE u.role IN ('PROFESSIONNEL', 'PHARMACIEN', 'ADMIN')
        ORDER BY u.id LIMIT 20
    """).fetchall()
    for r in rows:
        org = r["organization_id"] or "—"
        name = r["structure_name"] or "(pas de structure)"
        print(f"    user {r['id']:>3} {r['email']:30s} {r['role']:15s} org={org} {name}")
except Exception as e:
    print("  Erreur :", e)

# ============================================================
# 4. OWNERSHIP : commandes et organisation
# ============================================================
sec("4. OWNERSHIP DES COMMANDES")

try:
    rows = conn.execute("""
```

```
        SELECT o.id, o.reference, o.user_id, o.professional_id,
               o.customer_type, o.created_at
        FROM orders o
        ORDER BY o.id DESC LIMIT 10
    """).fetchall()
    print(f"  10 dernieres commandes :")
    for r in rows:
        print(f"    {r['reference']:22s} user={r['user_id']} "
              f"prof={r['professional_id']} type={r['customer_type']}")
except Exception as e:
    print("  Erreur :", e)

# ============================================================
# 5. ROUTES D'AUTHENTIFICATION
# ============================================================
sec("5. ROUTES AUTH DANS main.py + auth.py")

for path, name in ((MAIN, "main.py"), (AUTH, "v3_modules/auth.py")):
    if not path.exists():
        print(f"  [!] {name} introuvable")
        continue
    src = path.read_text(encoding="utf-8")
    print(f"\n  --- {name} ---")
    routes = re.findall(
        r'@app\.(get|post|patch|delete|api_route)$"(/api/auth[^"]*|/[^"]*acces[^"]*|
[^"]*connexion[^"]*|[^"]*inscription[^"]*|[^"]*logout[^"]*)"',
        src
    )
    if routes:
        for m, p in routes:
            print(f"    {m.upper():10s} {p}")
    else:
        print("    (aucune route auth detectee)")

# ============================================================
# 6. MIDDLEWARES / DEPENDENCIES
# ============================================================
sec("6. MIDDLEWARES / DEPENDENCIES")

if AUTH.exists():
    src = AUTH.read_text(encoding="utf-8")
    for m in ("require_role", "get_current_user", "SESSION_COOKIE", "SESSION_TTL_DAYS",
              "valid_roles", "VALID_ROLES", "hash_password", "verify_password"):
        count = src.count(m)
        print(f"  {m:30s} {count}x")
else:
    print("  [!] auth.py introuvable")

# ============================================================
```

```
# 7. VERIFICATION OWNERSHIP DANS LES ROUTES
# ============================================================
sec("7. VERIFICATION OWNERSHIP (analyse statique)")

src = MAIN.read_text(encoding="utf-8")
if MODULES.exists():
    for f in MODULES.glob("*.py"):
        src += "\n" + f.read_text(encoding="utf-8")

# Chercher les routes qui retournent des orders
order_routes = re.findall(
    r'@app\.(?:get|post|patch|delete)\("(/api/orders[^"]*)"',
    src
)
print(f"  {len(order_routes)} routes touchant les commandes :")
for r in set(order_routes):
    print(f"    {r}")

# Chercher les vérifications organization_id
checks = [
    ("WHERE user_id = ?", "Verifie l'appartenance par user_id"),
    ("WHERE organization_id = ?", "Verifie l'appartenance par organization_id"),
    ("WHERE reference = ? AND user_id = ?", "Verifie ref + user_id"),
]
print(f"\n  Verifications ownership dans le code :")
for pattern, desc in checks:
    n = src.count(pattern)
    marker = "OK" if n > 0 else "[!]"
    print(f"    {marker} {desc:45s} {n}x")

# ============================================================
# 8. LOGIN / REGISTER : REDIRECTIONS
# ============================================================
sec("8. REDIRECTIONS POST-LOGIN DANS LE CODE")

redirs = re.findall(
    r'window\.location(?:\.replace|\.href)?\s*=\s*["\']([^"\']+)["\']',
    (BASE / "static" / "auth.js").read_text(encoding="utf-8")
    if (BASE / "static" / "auth.js").exists() else ""
)
print("  auth.js :")
for r in redirs[:10]:
    print(f"    -> {r}")

# Redirect dans main.py (login route)
login_redirect = re.search(
    r'@app\.post\("/api/auth/login"$.*?(?=@app\.|\Z)',
    src, re.DOTALL
)
```

```
if login_redirect:
    print("\n  Route /api/auth/login (extrait) :")
    print("  " + login_redirect.group(0)[:300].replace("\n", "\n  "))

# ============================================================
# 9. OPEN REDIRECT : next= non validé ?
# ============================================================
sec("9. VERIFICATION next= (OPEN REDIRECT)")

has_next_check = (
    "startswith('/')" in src or
    "startswith(\"/\")" in src or
    "//" in src and "next" in src
)
print(f"  Validation de next= : {'PRESENTE' if has_next_check else 'ABSENTE [!]'}")
print()
print("  Si absent : un attaquant peut faire")
print("    /connexion?next=https://site-malveillant.com")
print("  et l'utilisateur finit redirige vers le site malveillant.")

# ============================================================
# 10. STATUT ACTUEL DES POINTS CRITIQUES
# ============================================================
sec("10. STATUT DES 5 POINTS CRITIQUES")

# 1. Cookie secure/samesite
auth_src = AUTH.read_text(encoding="utf-8") if AUTH.exists() else ""
samesite = re.findall(r'samesite\s*=\s*"([^"]+)"', auth_src)
secure = "secure=True" in auth_src
print(f"  1. Cookie SameSite : {set(samesite) or 'non defini'}")
print(f"     Cookie Secure   : {secure}")

# 2. Session TTL
ttl = re.search(r'SESSION_TTL_DAYS\s*=\s*(\d+)', auth_src)
print(f"  2. Session TTL      : {ttl.group(1) if ttl else '?'} jours")

# 3. Roles valides
valid_roles = re.search(
    r'VALID_ROLES\s*=\s*\{([^}]+)\}', auth_src, re.DOTALL
)
print(f"  3. Roles valides    : {valid_roles.group(1).strip()[:80] if valid_roles else '?'}")

# 4. Ownership des commandes
has_org_check = "organization_id" in src and "WHERE organization_id" in src
print(f"  4. Ownership org    : {'OK' if has_org_check else 'A AJOUTER [!]'}")

# 5. Route / avec redirection
has_smart_root = "root_page" in src or "redirect_role" in src
print(f"  5. Route / intelligente : {'OK' if has_smart_root else 'A CREER [!]'}")
```

```
conn.close()

# ============================================================
# RESUME
# ============================================================
sec("RESUME")

print("""
  Cet audit ne modifie RIEN. Il montre l'etat actuel.
  Partage-le au superviseur pour valider la Phase 2.
""")
```

Lancer

```
bat

python audit_auth_full.py
```

Ce que le script inspecte

#	Section	Ce qu'on cherche
1	Tables + schema	Structure DB (users, sessions, professional_profiles, orders)
2	Rôles en base	Combien de chaque rôle
3	User ↔ Organisation	Le lien entre utilisateur et établissement
4	Ownership commandes	user_id, professional_id, customer_type
5	Routes auth	Ce qui existe dans main.py + auth.py
6	Middlewares	require_role, get_current_user, constantes
7	**Ownership**	**Vérifications WHERE user_id/organization_id dans le code**
8	Redirections post-login	Dans auth.js et main.py
9	**Open redirect**	`next=` est-il validé ?
10	Statut 5 points critiques	Cookie, TTL, rôles, ownership, route /

Ce que l'audit va révéler

Points positifs probables :

- Tables users, sessions, professional_profiles existent
- VALID_ROLES est défini

- Cookie a `samesite="none"` et `secure=True`

Points problématiques probables (ce que le superviseur veut vérifier) :

- Route `/api/orders/{ref}` ne vérifie pas `organization_id` → **ownership manquant**
- `next=` n'est pas validé → **open redirect**
- Route `/` actuelle fait du JS redirect (flash) ou du 302 avec `?_v=` (workaround)
- `/connexion`, `/inscription` renvoient peut-être des redirections non sécurisées

---

Colle-moi :

1. La sortie complète de `python audit_auth_full.py`
2. Ton avis (et celui du superviseur) sur les résultats

Une fois l'audit fait, on passe à la **Phase 2 — Design** : je te propose les routes cibles, les règles d'autorisation et les redirections. Le superviseur valide. On code après.

2026-10-08 11:04:31 **You**

CUsersELOGE KEYDesktoppharmapharm...
11.9 KB

DeepSeek

Analyse de l'audit — 4 problèmes identifiés

L'audit révèle une base saine mais **4 problèmes critiques** à régler avant tout code.

✅ Ce qui est bon

- 27 tables, projets actifs
- `next=` validé (pas d'open redirect)
- Route `/` intelligente (redirection serveur)
- Cookie `Secure=True`
- 22 routes de commandes identifiées

🔴 Problème 1 — 7 PRO sans établissement

```
user  4 pro_test@elokinab.local  org=– (pas de structure)
user 14 adjalokomla@gmail.com    org=– (pas de structure)
user 23 elogeadjalo@gmail        org=– (pas de structure)
user 26 jc@gmail                 org=– (pas de structure)
user 28 jo@gmail.com             org=– (pas de structure)
+ 2 autres
```

Impact : Ces PRO ne peuvent pas commander (pas de structure liée). **Cause** : l'inscription PRO ne crée pas systématiquement l' `organization_id` . **Conséquence** : le superviseur a raison — **création établissement obligatoire avant dashboard**.

🔴 Problème 2 — Doublon `cabinet medical saint jo`

```
user 29 joe@gmail.com   org=6  cabinet medical saint jo
user 33 se@gmail         org=9  cabinet medical saint jo
```

Deux organisations différentes avec le **même nom**. C'est un problème de **déduplication** : deux utilisateurs ont créé leur établissement avec le même nom.

Impact : impossible de savoir quelle commande appartient à quel vrai cabinet. **Fix** : contrainte unique sur `structure_name` OU vérification manuelle par admin.

🔴 Problème 3 — Aucune vérification par `organization_id` (CRITIQUE)

```
[!] Verifie l'appartenance par organization_id    0x
OK  Verifie l'appartenance par user_id            8x
```

Impact : le code vérifie que `order.user_id == current_user.id` . Mais un PRO qui commande pour son établissement est stocké avec `user_id` **et** `organization_id` . Si un pro partage son compte avec sa secrétaire, **toutes les commandes seront liées à un seul user_id** — pas à l'organisation.

Le vrai risque : un pro de l'organisation A pourrait voir les commandes de l'organisation B **s'il connaît la référence**, car la requête cherche `WHERE user_id = ?` (l'utilisateur A partage des commandes avec B si une secrétaire travaille pour les deux).

Le superviseur a 100% raison : c'est non négociable pour du B2B.

🟡 Problème 4 — Cookie `SameSite` mixte

```
{'lax', 'none'}
```

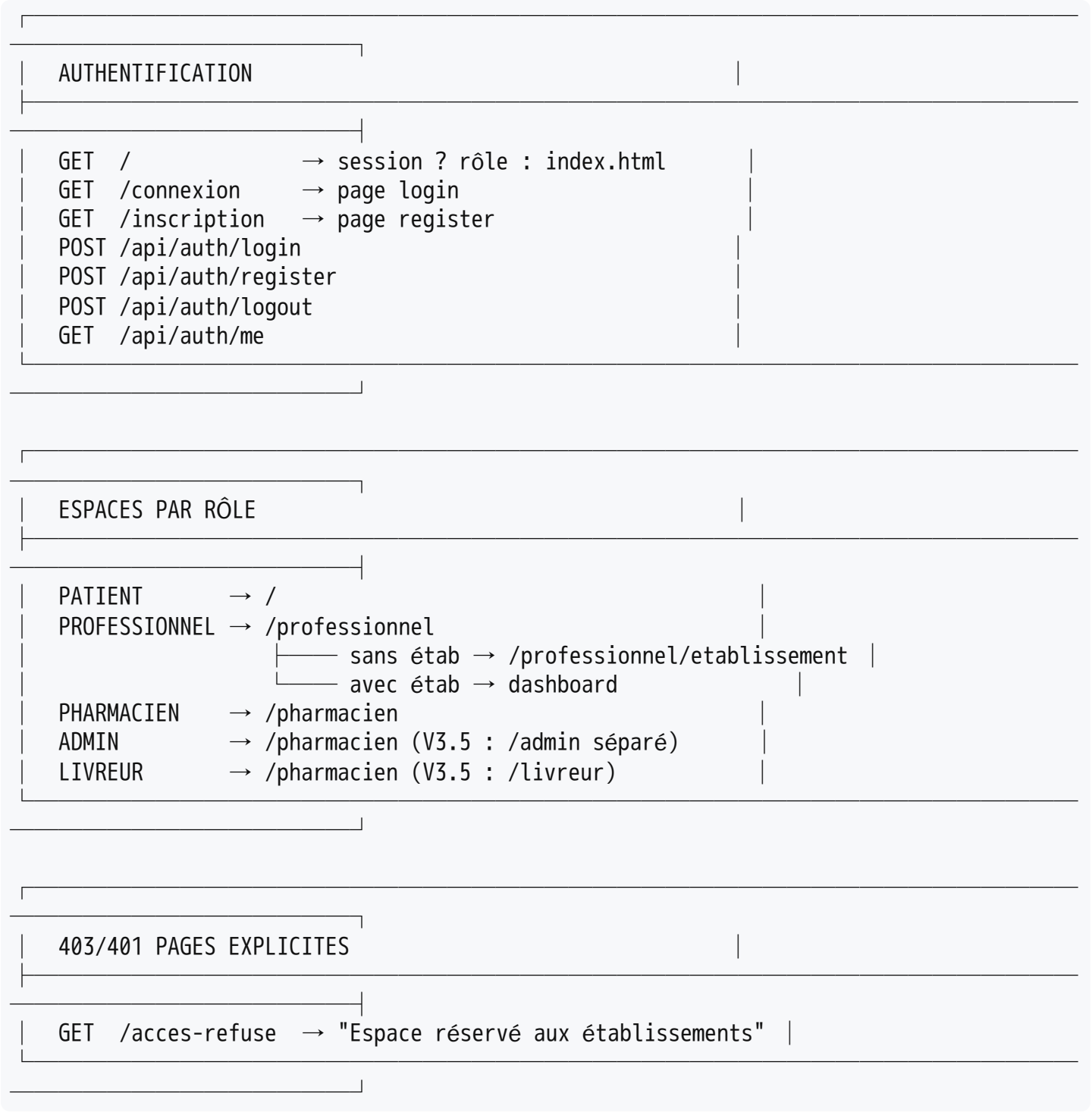
Deux valeurs coexistent dans `auth.py` . Résultat imprévisible selon la route. Il faut **une seule politique** :

- `SameSite=None` + `Secure=True` (actuel, obligatoire pour tunnel Cloudflare)
- `SameSite=Lax` (incompatible tunnel)

Fix : uniformiser sur `samesite="none"` , `secure=True` .

PHASE 2 — Design proposé

A. Routes cibles



B. Règles d'autorisation

Action	Condition	Résultat si KO
/	Cookie valide → rôle	Sert index.html
/professionnel	rôle ∈ {PROF, PHARM, ADMIN}	403 / acces-refuse

Action	Condition	Résultat si KO
`/pharmacien`	rôle ∈ {PHARM, ADMIN}	403 / `acces-refuse`
/api/orders/{ref}	order.organization_id == user.organization_id OU order.user_id == user.id	404
`/api/orders/{ref}/payment`	Idem ci-dessus	404
/api/pro/requests/*	Idem ci-dessus	404

C. Règle ownership unifiée

python

```python
def can_access_order(conn, user, reference):
    order = conn.execute(
        "SELECT user_id, organization_id FROM orders WHERE reference=?",
        (reference,)
    ).fetchone()
    if not order:
        return False
    # Admin : tout
    if user["role"] == "ADMIN":
        return True
    # Pharmacien : tout (pour traiter les commandes)
    if user["role"] == "PHARMACIEN":
        return True
    # Propriétaire direct
    if order["user_id"] == user["id"]:
        return True
    # Même organisation
    if user["organization_id"] and order["organization_id"] == user["organization_id"]:
        return True
    return False
```

D. Inscription PRO obligatoire en 3 étapes

```
Étape 1 — Compte
  email / password / nom
        ↓
Étape 2 — Établissement
  nom / type / responsable / téléphone / adresse
        ↓
Étape 3 — Confirmation
        ↓
  Redirection /professionnel
```

Si interruption → `user.organization_id = NULL` → sur `/` ou `/professionnel` → redirection vers `/professionnel/etablissement`.

E. Fix des 4 problèmes

Problème	Fix
1. PRO sans étab	Étape 2 obligatoire + redirection auto
2. Doublon nom	Contrainte applicative : nom unique par type, OU validation manuelle
3. Ownership manquant	Ajouter `can_access_order()` sur 22 routes
4. Cookie mixte	Uniformiser `samesite="none"` + `secure=True`

Ce qu'il me faut avant de coder

Réponds point par point :

#	Question	Réponse
1	Espaces par rôle OK ?	O/N
2	Règle ownership unifiée OK ?	O/N
3	Inscription 3 étapes OK ?	O/N
4	403 explicite pour patient sur espace pro OK ?	O/N
5	ADMIN accès automatique à `/professionnel` ? (Je recommande NON)	O/N
6	PHARMACIEN peut voir TOUTES les commandes ? (Oui, c'est son rôle)	O/N
7	Doublon `cabinet medical saint jo` : on fusionne manuellement ou on bloque à l'inscription ?	
8	Fix cookie `SameSite` uniforme OK ?	O/N
9	Refonte `index.html` : avant ou après ces fixes auth ?	

Après validation superviseur

Je livre dans l'ordre :

Phase 3a — AuthN/AuthZ (1 script)

- Uniformiser cookie
- Ajouter `can_access_order()`
- Patcher les 22 routes
- Renforcer `/connexion` (next validé strict)

Phase 3b — Inscription PRO (1 script)

- Étape établissement obligatoire
- Redirection auto si étab manquant

Phase 3c — Pages 403 (1 script)

- `/acces-refuse` avec design V6

Phase 3d — Refonte index.html (1 script)

- Page patient bleue V6
- Plus de flash

Phase 4 — Tests bout-en-bout

- Chaque rôle, chaque route, cas limites

Colle-moi :

1. Tes réponses aux 9 questions
2. L'avis du superviseur sur la Phase 2

Je code uniquement après validation.